

控制理论

从经典方法到现代前沿

涵盖经典控制、现代控制与非线性控制

魏舟

2026 年 7 月 24 日

目录

第 1 章 引言与数学基础	1
1.1 控制理论概述	1
1.1.1 控制理论的发展简史	1
1.1.2 开环控制与闭环控制	1
1.2 数学预备知识	2
1.2.1 复数与复变函数回顾	2
1.2.2 Laplace 变换	3
1.2.3 Laplace 逆变换与部分分式展开法	4
1.3 微分方程与线性系统	5
1.3.1 线性常微分方程	5
1.3.2 自由响应与强迫响应	5
1.3.3 系统的微分方程求解	6
1.4 传递函数概念	6
1.4.1 传递函数的定义	6
1.4.2 极点和零点	7
1.4.3 极点位置与系统响应关系	7
1.5 控制系统的基本要求	7
1.5.1 稳定性	7
1.5.2 快速性（动态性能）	8
1.5.3 准确性（稳态性能）	8
第 2 章 控制系统数学模型	11
2.1 微分方程建模	11
2.1.1 RLC 电路	11
2.1.2 质量-弹簧-阻尼系统	12
2.1.3 直流电机	12
2.2 传递函数与典型环节	13
2.2.1 典型环节的传递函数	13
2.3 框图及其化简	14
2.3.1 方框图的基本连接方式	14
2.3.2 框图等效变换规则	14
2.4 信号流图与 Mason 增益公式	16
2.4.1 信号流图的基本概念	16
2.4.2 Mason 增益公式	16
2.5 典型系统的传递函数	17
2.5.1 闭环控制系统的基本传递函数	17

第3章 时域分析	21
3.1 典型输入信号	21
3.2 一阶系统的时域分析	21
3.2.1 一阶系统的数学模型	21
3.2.2 一阶系统的单位阶跃响应	21
3.2.3 一阶系统的单位斜坡响应	22
3.3 二阶系统的时域分析	22
3.3.1 标准二阶系统的传递函数	22
3.3.2 欠阻尼二阶系统的单位阶跃响应	23
3.3.3 临界阻尼与过阻尼响应	23
3.4 时域性能指标	24
3.4.1 上升时间 t_r	24
3.4.2 峰值时间 t_p	24
3.4.3 超调量 $\sigma\%$	24
3.4.4 调节时间 t_s	24
3.5 稳态误差分析	25
3.5.1 终值定理与稳态误差	25
3.5.2 系统型别与静态误差系数	26
3.6 稳定性代数判据	27
3.6.1 稳定性概念与充要条件	27
3.6.2 Routh-Hurwitz 判据	27
3.6.3 Routh 表的构造	27
3.6.4 Routh 判据的特殊情况	28
第4章 根轨迹法	31
4.1 根轨迹的基本概念	31
4.1.1 问题的提出	31
4.1.2 开环传递函数与闭环极点的关系	31
4.2 幅值条件与相角条件	32
4.3 根轨迹的绘制规则	32
4.3.1 绘制规则的应用示例——二阶系统	34
4.3.2 三阶系统的根轨迹	34
4.4 参数根轨迹	37
4.4.1 非增益参数变化的根轨迹	37
4.5 基于根轨迹的系统性能分析	37
4.5.1 主导极点概念	37
4.5.2 附加极点对系统性能的影响	38
4.5.3 附加零点对系统性能的影响	38
4.6 基于根轨迹的补偿设计	38
4.6.1 超前补偿（相位超前网络）	39
4.6.2 滞后补偿（相位滞后网络）	39
4.6.3 设计流程	39
第5章 频域分析	41
5.1 频率特性的基本概念	41
5.1.1 频率特性的定义	41
5.1.2 频率特性的几何表示	41

5.2 典型环节的频率特性	42
5.2.1 比例环节	42
5.2.2 积分环节	42
5.2.3 微分环节	42
5.2.4 惯性环节	42
5.2.5 振荡环节	43
5.2.6 滞后环节	43
5.2.7 一阶微分环节	43
5.3 Bode 图	44
5.3.1 对数频率特性的坐标	44
5.3.2 系统辨识中的 Bode 图应用	44
5.4 Nyquist 图与稳定性判据	46
5.4.1 Nyquist 图的基本概念	46
5.4.2 Cauchy 定理与 Nyquist 稳定判据	46
5.4.3 Nyquist 图的绘制	47
5.5 稳定裕度	47
5.5.1 幅值裕度	47
5.5.2 相位裕度	48
5.5.3 从 Bode 图和 Nyquist 图求取稳定裕度	48
5.6 闭环频率特性	50
5.6.1 闭环频域指标	50
5.6.2 Nichols 图	50
5.7 频域特性与时域指标的关系	51
5.7.1 二阶系统的频域-时域关系	51
5.7.2 由频域指标设计系统	52
5.8 Python 频域分析仿真	52
5.9 本章小结	54
第 6 章 系统校正与 PID 控制	55
6.1 校正的基本概念	55
6.1.1 校正的分类	55
6.1.2 频率法校正的目标	55
6.2 超前校正	56
6.2.1 超前校正的传递函数	56
6.2.2 超前校正的频域特性	56
6.2.3 超前校正的设计步骤	57
6.3 滞后校正	59
6.3.1 滞后校正的传递函数	59
6.3.2 滞后校正的设计步骤	59
6.4 滞后-超前校正	60
6.5 PID 控制	60
6.5.1 PID 控制的基本原理	60
6.5.2 各环节的作用	61
6.5.3 位置式与增量式 PID	61
6.5.4 Ziegler-Nichols 整定方法	62
6.5.5 PID 参数对系统性能的影响	63

6.6	前馈-反馈复合校正	63
6.6.1	不变性原理	63
6.6.2	前馈-反馈复合控制	63
6.7	Smith 预估器	64
6.8	Python PID 控制器仿真	64
6.9	本章小结	66
第 7 章	状态空间基础	67
7.1	状态空间的基本概念	67
7.1.1	状态与状态变量	67
7.1.2	状态方程与输出方程	67
7.2	从传递函数到状态空间	68
7.2.1	能控标准型	69
7.2.2	能观标准型	69
7.2.3	对角标准型与 Jordan 标准型	69
7.3	从状态空间到传递函数	70
7.3.1	传递函数矩阵的推导	70
7.4	状态方程的解	71
7.4.1	状态转移矩阵	71
7.4.2	Laplace 变换法	72
7.4.3	Cayley-Hamilton 法	72
7.5	状态转移矩阵的性质	73
7.6	线性变换与特征结构	74
7.6.1	坐标变换	74
7.6.2	特征值与特征向量	75
7.6.3	对角化与模态分解	75
7.7	离散时间状态空间	75
7.7.1	离散状态空间模型	75
7.7.2	连续系统的离散化	76
7.7.3	离散系统的稳定性	76
7.8	Python 状态方程数值求解	76
7.9	本章小结	78
第 8 章	能控性与能观性	81
8.1	能控性的基本概念	81
8.1.1	能控性的定义	81
8.2	能控性判据	82
8.2.1	能控性矩阵与秩判据	82
8.2.2	PBH 判据	82
8.2.3	能控标准型与能控性	83
8.3	能观性的基本概念	83
8.3.1	能观性的定义	83
8.4	能观性判据	84
8.4.1	能观性矩阵与秩判据	84
8.4.2	PBH 能观性判据	84
8.5	对偶原理	85
8.5.1	对偶原理的应用	85

8.6	Kalman 分解	85
8.6.1	结构分解的基本思想	85
8.6.2	Kalman 分解的构造方法	86
8.7	最小实现	87
8.7.1	最小实现的定义	87
8.7.2	不可约传递函数	87
8.8	Python 能控性与能观性分析	88
8.9	本章小结	90
第 9 章	极点配置与状态反馈	91
9.1	状态反馈的基本概念	91
9.1.1	状态反馈的结构	91
9.1.2	闭环系统的特征值	92
9.2	极点配置定理	92
9.2.1	能控性与极点配置的等价性	92
9.2.2	多输入系统的极点配置	93
9.3	Ackermann 公式	93
9.3.1	公式推导	93
9.3.2	计算步骤与应用	94
9.4	状态观测器	95
9.4.1	开环与闭环观测器	95
9.4.2	观测器增益的设计	96
9.5	分离原理	96
9.5.1	控制器与观测器的独立设计	96
9.5.2	观测器增益的极点配置（对偶设计）	97
9.6	基于观测器的输出反馈	98
9.6.1	完整结构	98
9.6.2	传递函数的实现——二自由度结构	98
9.7	MATLAB 与 Python 实现	99
9.7.1	MATLAB 实现	99
9.7.2	Python 实现	100
9.8	小结	103
第 10 章	线性二次型最优控制	105
10.1	最优控制问题的提法	105
10.1.1	性能指标（代价函数）	105
10.1.2	Q 和 R 的物理意义	105
10.1.3	最优控制问题的 Hamiltonian	106
10.2	连续时间 LQR	106
10.2.1	Riccati 微分方程与代数方程	106
10.2.2	最优增益的推导	106
10.2.3	加权矩阵的选择与极点移动趋势	107
10.3	离散时间 LQR 与 LQG 问题	108
10.3.1	离散时间 LQR	108
10.3.2	LQG 问题的提法	108
10.4	Kalman 滤波器	109
10.4.1	问题描述与推导思路	109
10.4.2	离散 Kalman 滤波五大公式	109

10.4.3 Kalman 滤波的稳定性	109
10.4.4 连续时间 Kalman-Bucy 滤波器	110
10.5 扩展 Kalman 滤波器	110
10.5.1 非线性系统的线性化	110
10.6 Python 实现	112
10.6.1 LQR 求解与仿真	112
10.6.2 Kalman 滤波仿真	114
10.6.3 EKF 示例——非线性振荡器	116
10.6.4 离散 LQR 与 Kalman 结合的 LQG 仿真	118
10.7 从 LQR 到 MPC 的联系	119
10.8 小结	119
第 11 章 非线性控制	121
11.1 非线性系统的特性	121
11.1.1 非线性系统的基本描述	121
11.1.2 叠加原理的失效与多平衡点	121
11.1.3 极限环、混沌与分岔	122
11.2 Lyapunov 稳定性理论	123
11.2.1 自治系统的稳定性概念	123
11.2.2 Lyapunov 直接法（能量法）	123
11.2.3 Lyapunov 函数的构造方法	123
11.2.4 Lasalle 不变原理	124
11.3 滑模控制	124
11.3.1 滑模控制的基本原理	124
11.3.2 滑模面的设计	125
11.3.3 趋近律设计	125
11.3.4 抖振问题与边界层方法	125
11.4 反馈线性化	126
11.4.1 输入-状态线性化	126
11.4.2 Lie 导数与 Lie 括号	126
11.4.3 相对阶与输入-输出线性化	127
11.5 Backstepping 设计方法	127
11.5.1 Backstepping 的基本思想	127
11.5.2 严格反馈形式	128
11.5.3 二阶系统的 Backstepping 设计范例	128
11.5.4 自适应 Backstepping	128
11.6 非线性观测器	129
11.6.1 高增益观测器	129
11.6.2 滑模观测器	129
11.7 Python 实现：倒立摆的滑模控制仿真	130
11.8 小结	132
第 12 章 高级控制方法	133
12.1 模型预测控制	133
12.1.1 三大基本原理	133
12.1.2 MPC 的数学表述——二次规划形式	133
12.1.3 预测方程的推导	134
12.1.4 约束处理	134

12.1.5	可行性与稳定性	134
12.1.6	非线性 MPC 与分布式 MPC	135
12.2	鲁棒控制	135
12.2.1	不确定性的描述	135
12.2.2	H_∞ 控制问题	136
12.2.3	标准 H_∞ 框架与 Riccati 方法	136
12.2.4	μ 综合简介	136
12.3	自适应控制	137
12.3.1	模型参考自适应控制	137
12.3.2	自校正控制	137
12.3.3	MIT 规则与归一化自适应律	138
12.4	强化学习与控制	138
12.4.1	MDP 与最优控制的等价性	138
12.4.2	Policy Iteration 求解 LQR	139
12.4.3	自适应动态规划 (ADP)	139
12.5	智能控制	140
12.5.1	模糊控制	140
12.5.2	神经网络控制	141
12.6	前沿方向	141
12.6.1	学习型 MPC	141
12.6.2	Safe RL	141
12.6.3	基于控制理论的深度学习解释	142
12.7	Python 实现	143
12.7.1	简单 MPC 的 QP 求解	143
12.7.2	RL 求解 LQR	145
12.7.3	模糊控制器的简单实现	147
12.8	小结	148

引言与数学基础

控制理论研究动态系统的建模、分析与设计方法。本章介绍自动控制理论的发展历程、核心思想与必要的数学预备知识。

§1.1 控制理论概述

1.1.1 控制理论的发展简史

自动控制的起源可追溯至古代文明。公元前 3 世纪，希腊人 Ktesibios 发明的水钟利用浮子调节水位实现流量控制，是最早的反馈机构之一。然而，控制理论作为一门系统科学，其发展脉络主要集中在近现代：

- 工业革命时期（18–19 世纪）。James Watt 于 1788 年设计的离心式调速器是自动控制领域的里程碑。该装置利用飞球离心力调节蒸汽机进气阀门，构成速度反馈回路。然而，调速器在特定工况下出现的剧烈振荡——即所谓“猎振”（hunting）现象——引起了 J. C. Maxwell 的注意。1868 年，Maxwell 发表了《On Governors》一文，首次从微分方程角度分析了调速系统的稳定性，开创了控制理论的数学化研究。
- 经典控制时期（1930–1960）。20 世纪 30 年代，Bell 实验室的 H. Nyquist 提出了基于频率响应的稳定性判据，H. W. Bode 发展了相应的对数频率特性图（Bode 图）。二战期间，火炮伺服系统、雷达跟踪系统等军事需求极大地推动了控制理论的发展。N. Wiener 于 1948 年出版的《控制论》（Cybernetics）将反馈概念推广至通信、生物和社会系统。
- 现代控制时期（1960 至今）。1960 年，R. E. Kalman 提出了状态空间方法和著名的 Kalman 滤波器，将控制理论从频域的传递函数方法扩展到时域的状态空间框架。此后，最优控制（Pontryagin 极大值原理、Bellman 动态规划）、鲁棒控制（ H_∞ 理论）、自适应控制、非线性控制和非线性观测器等方向蓬勃发展。

1.1.2 开环控制与闭环控制

开环控制系统

开环控制系统是指控制器的输出仅取决于参考输入信号，而不依赖于系统实际输出信号的系统。其通用结构为：

参考输入 $\rightarrow$ 控制器 $\rightarrow$ 执行器 $\rightarrow$ 被控对象 $\rightarrow$ 输出

闭环控制系统（反馈控制系统）

闭环控制系统将系统输出量通过测量元件反馈至输入端，与参考输入比较形成偏差信号，控制器根据偏差产生控制作用。其核心在于“检测偏差，纠正偏差”。

【例题 1.1】 以室内温度控制为例：开环方式是根据经验设定加热功率，不检测室温；闭环方式则通过温度传感器实时测量室温，与设定温度比较后自动调节加热功率。若门窗意外开启，开环系统无法补偿热量散失，而闭环系统能自动增大加热功率以维持设定温度。

注. 反馈的引入使系统具备以下能力：(1) 抑制外部扰动和内部参数变化的影响；(2) 改善系统的动态响应特性；(3) 降低对元件精度的依赖。但反馈也可能带来稳定性问题，需要精心设计。

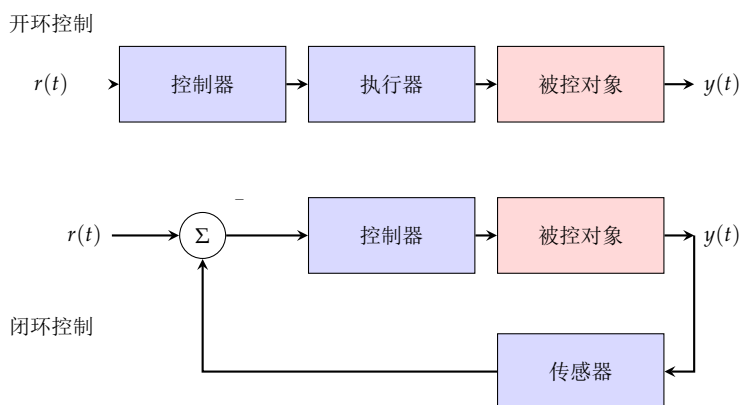


图 1.1: 开环控制系统（上）与闭环控制系统（下）结构对比

§ 1.2 数学预备知识

1.2.1 复数与复变函数回顾

复数 $s = \sigma + j\omega$ ，其中 $\sigma = \Re(s)$ 为实部， $\omega = \Im(s)$ 为虚部， $j = \sqrt{-1}$ 。复数的极坐标表示为 $s = |s|e^{j\theta}$ ，其中模 $|s| = \sqrt{\sigma^2 + \omega^2}$ ，辐角 $\theta = \arctan(\omega/\sigma)$ 。

Euler 公式是连接复指数与三角函数的桥梁：

$$e^{j\theta} = \cos \theta + j \sin \theta, \quad e^{-j\theta} = \cos \theta - j \sin \theta$$

由此可得：

$$\cos \theta = \frac{e^{j\theta} + e^{-j\theta}}{2}, \quad \sin \theta = \frac{e^{j\theta} - e^{-j\theta}}{2j}$$

解析函数

设复变函数 $F(s)$ 在区域 D 内每一点均可导，则称 $F(s)$ 在 D 内解析（analytic）。若 $F(s)$ 在 s_0 处不解析但在 s_0 的任意邻域内均解析，则称 s_0 为 $F(s)$ 的孤立奇点。

孤立奇点按 Laurent 展开中负幂项的最高次数分类：(1) 可去奇点——无负幂项；(2) m 阶极点——最高负幂为 $(s - s_0)^{-m}$ ；(3) 本性奇点——负幂项有无穷多项。

留数定理

设 C 为一条简单闭合正向曲线，函数 $F(s)$ 在 C 内部除有限个孤立奇点 $s_1, s_2, \dots, s_n$ 外解析，则

$$\oint_C F(s) ds = 2\pi j \sum_{k=1}^n \text{Res}[F(s), s_k]$$

其中 $\text{Res}[F(s), s_k]$ 为 $F(s)$ 在 s_k 处的留数。对于 m 阶极点 s_0 ：

$$\text{Res}[F(s), s_0] = \frac{1}{(m-1)!} \lim_{s \rightarrow s_0} \frac{d^{m-1}}{ds^{m-1}} [(s-s_0)^m F(s)]$$

1.2.2 Laplace 变换**Laplace 变换**

设函数 $f(t)$ 在 $t \geq 0$ 上有定义，若积分

$$F(s) = \mathcal{L}[f(t)] = \int_0^{\infty} f(t)e^{-st} dt$$

在复平面某区域内收敛，则称 $F(s)$ 为 $f(t)$ 的 Laplace 变换（或象函数）， $f(t)$ 为 $F(s)$ 的 Laplace 逆变换（或原函数）。

Laplace 变换将时域 $t \in [0, \infty)$ 中的函数映射到复频域 $s = \sigma + j\omega$ 中的函数，其核心价值在于将微分方程转化为代数方程。

Laplace 变换的存在条件：若存在常数 $M > 0$ 和 σ_0 使得对一切 $t \geq 0$ 有 $|f(t)| \leq Me^{\sigma_0 t}$ ，则 $F(s)$ 在半平面 $\Re(s) > \sigma_0$ 内存在。

常用 Laplace 变换的基本性质： (1.1)

$$\text{线性: } \mathcal{L}[af(t) + bg(t)] = aF(s) + bG(s) \quad (1.2)$$

$$\text{时域微分: } \mathcal{L}\left[\frac{df}{dt}\right] = sF(s) - f(0) \quad (1.3)$$

$$\text{二阶微分: } \mathcal{L}\left[\frac{d^2f}{dt^2}\right] = s^2F(s) - sf(0) - \dot{f}(0) \quad (1.4)$$

$$\text{时域积分: } \mathcal{L}\left[\int_0^t f(\tau) d\tau\right] = \frac{1}{s}F(s) \quad (1.5)$$

$$\text{时域平移: } \mathcal{L}[f(t-a)u(t-a)] = e^{-as}F(s) \quad (1.6)$$

$$\text{频域平移: } \mathcal{L}[e^{at}f(t)] = F(s-a) \quad (1.7)$$

$$\text{相似: } \mathcal{L}[f(at)] = \frac{1}{a}F\left(\frac{s}{a}\right), a > 0 \quad (1.8)$$

$$\text{卷积: } \mathcal{L}[f(t) * g(t)] = F(s)G(s) \quad (1.9)$$

$$\text{终值定理: } \lim_{t \rightarrow \infty} f(t) = \lim_{s \rightarrow 0} sF(s) \text{ (若极限存在)} \quad (1.10)$$

$$\text{初值定理: } \lim_{t \rightarrow 0^+} f(t) = \lim_{s \rightarrow \infty} sF(s) \quad (1.11)$$

常用函数的 Laplace 变换表： (1.12)

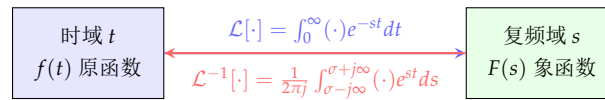
$$\mathcal{L}[\delta(t)] = 1, \quad \mathcal{L}[u(t)] = \frac{1}{s}, \quad \mathcal{L}[t] = \frac{1}{s^2} \quad (1.13)$$

$$\mathcal{L}\left[\frac{t^{n-1}}{(n-1)!}\right] = \frac{1}{s^n}, \quad n = 1, 2, \dots, \quad \mathcal{L}[e^{-at}] = \frac{1}{s+a} \quad (1.14)$$

$$\mathcal{L}[\sin \omega t] = \frac{\omega}{s^2 + \omega^2}, \quad \mathcal{L}[\cos \omega t] = \frac{s}{s^2 + \omega^2} \quad (1.15)$$

$$\mathcal{L}[e^{-at} \sin \omega t] = \frac{\omega}{(s+a)^2 + \omega^2}, \quad \mathcal{L}[e^{-at} \cos \omega t] = \frac{s+a}{(s+a)^2 + \omega^2} \quad (1.16)$$

$$\mathcal{L}[te^{-at}] = \frac{1}{(s+a)^2}, \quad \mathcal{L}[1 - e^{-at}] = \frac{a}{s(s+a)} \quad (1.17)$$



微分方程 $\rightarrow$ 代数方程

卷积 $\longleftrightarrow$ 乘积

图 1.2: Laplace 变换建立时域与复频域的桥梁

1.2.3 Laplace 逆变换与部分分式展开法

在控制理论中， $F(s)$ 通常为有理分式：

$$F(s) = \frac{B(s)}{A(s)} = \frac{b_ms^m + b_{m-1}s^{m-1} + \dots + b_1s + b_0}{s^n + a_{n-1}s^{n-1} + \dots + a_1s + a_0}, \quad m \leq n$$

反变换的核心方法是**部分分式展开法**，将复杂的 $F(s)$ 分解为若干简单分式之和，再逐项利用变换表求反变换。

部分分式展开步骤

1. 将分母多项式 $A(s)$ 因式分解，找出全部极点 $p_1, p_2, \dots, p_n$ 。
2. 根据极点类型（单实极点、重实极点、共轭复极点）选择相应的展开形式。
3. 计算待定系数（使用留数法、待定系数法或特殊值法）。
4. 利用 Laplace 变换表逐项求反变换。

情况一： $A(s)$ 有 n 个互异实极点 $p_1, p_2, \dots, p_n$ 。

$$F(s) = \frac{B(s)}{(s-p_1)(s-p_2)\dots(s-p_n)} = \frac{c_1}{s-p_1} + \frac{c_2}{s-p_2} + \dots + \frac{c_n}{s-p_n}$$

其中系数由留数公式给出：

$$c_k = (s-p_k)F(s)|_{s=p_k}, \quad k = 1, 2, \dots, n$$

于是反变换为：

$$f(t) = c_1 e^{p_1 t} + c_2 e^{p_2 t} + \dots + c_n e^{p_n t}$$

情况二： $A(s)$ 含有 r 重实极点 p_1 。

$$F(s) = \frac{B(s)}{(s-p_1)^r} = \frac{c_{1r}}{(s-p_1)^r} + \frac{c_{1,r-1}}{(s-p_1)^{r-1}} + \cdots + \frac{c_{11}}{s-p_1}$$

系数公式：

$$c_{1k} = \frac{1}{(r-k)!} \frac{d^{r-k}}{ds^{r-k}} [(s-p_1)^r F(s)] \Big|_{s=p_1}$$

情况三： $A(s)$ 含共轭复极点 $\alpha \pm j\beta$ 。此时可将相应部分写为：

$$\frac{cs+d}{(s-\alpha)^2+\beta^2}$$

对应的反变换为 $e^{\alpha t}(A \cos \beta t + B \sin \beta t)$ 的形式。

【例题 1.2】 求 $F(s) = \frac{s+3}{s^2+3s+2}$ 的反变换。

解：分母因式分解： $s^2+3s+2 = (s+1)(s+2)$ 。展开：

$$F(s) = \frac{s+3}{(s+1)(s+2)} = \frac{c_1}{s+1} + \frac{c_2}{s+2}$$

$$c_1 = \frac{s+3}{s+2} \Big|_{s=-1} = \frac{2}{1} = 2, \quad c_2 = \frac{s+3}{s+1} \Big|_{s=-2} = \frac{1}{-1} = -1$$

故 $F(s) = \frac{2}{s+1} - \frac{1}{s+2}$ ，反变换得：

$$f(t) = (2e^{-t} - e^{-2t})u(t)$$

§ 1.3

微分方程与线性系统

1.3.1 线性常微分方程

控制系统通常由 n 阶线性常系数 ODE 描述：

$$a_n y^{(n)} + a_{n-1} y^{(n-1)} + \cdots + a_1 \dot{y} + a_0 y = b_m u^{(m)} + b_{m-1} u^{(m-1)} + \cdots + b_1 \dot{u} + b_0 u$$

其中 $y(t)$ 为系统输出， $u(t)$ 为系统输入， a_i, b_j 为实常数。

对两端取 Laplace 变换（零初始条件），得代数方程：

$$(a_n s^n + \cdots + a_1 s + a_0)Y(s) = (b_m s^m + \cdots + b_1 s + b_0)U(s)$$

1.3.2 自由响应与强迫响应

线性系统的响应可以分解为两部分：

自由响应与强迫响应

系统在非零初始条件下的响应由两部分构成：

- 自由响应（零输入响应）——由非零初始条件引起，与输入无关，其形态由系统的特征根（极点）决定。
- 强迫响应（零状态响应）——由外部输入激励产生。

叠加原理：对于线性系统，若输入 $u_1(t)$ 产生响应 $y_1(t)$ ，输入 $u_2(t)$ 产生响应 $y_2(t)$ ，则输入 $\alpha u_1(t) + \beta u_2(t)$ 产生响应 $\alpha y_1(t) + \beta y_2(t)$ 。这一性质是时域分析中卷积方法和频域传递函数方法的基础。

1.3.3 系统的微分方程求解

以求一阶系统 $\dot{y} + ay = bu(t)$ 为例。通解由齐次解加特解构成：

- 齐次解： $y_h(t) = Ce^{-at}$ ，其中 C 由初始条件确定。
- 特解：依赖于 $u(t)$ 的形式。若 $u(t) = 1$ （阶跃输入），则特解 $y_p(t) = b/a$ 。
- 全解： $y(t) = y_h(t) + y_p(t) = Ce^{-at} + b/a$ 。

代入 $y(0) = y_0$ 得 $C = y_0 - b/a$ ，故：

$$y(t) = \left(y_0 - \frac{b}{a}\right)e^{-at} + \frac{b}{a}$$

第一项为自由响应（暂态），第二项为强迫响应（稳态）。

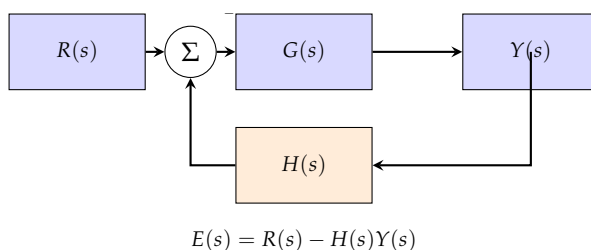


图 1.3: 标准单位反馈控制系统框图

§1.4 传递函数概念

1.4.1 传递函数的定义

传递函数

对于线性时不变系统，在零初始条件下，传递函数 $G(s)$ 定义为输出量 $Y(s)$ 与输入量 $U(s)$ 的 Laplace 变换之比：

$$G(s) = \frac{Y(s)}{U(s)} \Big|_{\text{零初始条件}} = \frac{b_ms^m + b_{m-1}s^{m-1} + \cdots + b_1s + b_0}{a_ns^n + a_{n-1}s^{n-1} + \cdots + a_1s + a_0}$$

传递函数具有以下重要性质：

- 传递函数仅取决于系统本身的结构和参数，与输入信号形式和初始条件无关。
- 传递函数是复变量 s 的有理真分式（物理可实现系统满足 $m \leq n$ ）。
- 传递函数的求解等价于系统微分方程在零初始条件下的 Laplace 变换。
- 传递函数不能反映系统内部状态——它只描述输入-输出关系。这也是经典控制理论的根本局限之一。

1.4.2 极点和零点

将传递函数写为因式分解形式：

$$G(s) = K \frac{(s - z_1)(s - z_2) \cdots (s - z_m)}{(s - p_1)(s - p_2) \cdots (s - p_n)}$$

- 零点 $z_1, z_2, \dots, z_m$ ——使 $G(s) = 0$ 的 s 值。
- 极点 $p_1, p_2, \dots, p_n$ ——使 $G(s) \rightarrow \infty$ 的 s 值。
- 增益 K ——系统的增益因子，传递函数根轨迹增益。

极点和零点在复平面上的分布完全决定了系统的动态特性。

特征方程

系统的特征方程为 $A(s) = 0$ ，即 $1 + G(s)H(s) = 0$ （闭环）或 $A(s) = 0$ （开环分母）。特征方程的根（即闭环极点）决定了系统自由响应的模态形式。

例如，若系统具有极点 $p = -\sigma \pm j\omega_d$ ($\sigma > 0$)，则自由响应包含 $e^{-\sigma t} \cos(\omega_d t)$ 形式的振荡衰减模态；若 $p = +j\omega$ （纯虚极点），则含等幅振荡；若 $p = \sigma_j > 0$ ，则含指数发散模态（不稳定）。

1.4.3 极点位置与系统响应关系

令 $s = \sigma + j\omega$ 。每个极点 $p_i = \sigma_i + j\omega_i$ 对应一个时域模态 $e^{p_i t} = e^{\sigma_i t}(\cos \omega_i t + j \sin \omega_i t)$ 。由此可得：

- 极点位于左半平面 ($\sigma_i < 0$)——响应衰减至零（稳定）。
- 极点位于虚轴上 ($\sigma_i = 0$)——响应等幅振荡（临界稳定）。
- 极点位于右半平面 ($\sigma_i > 0$)——响应发散（不稳定）。
- 极点实部绝对值越大——衰减越快，暂态过程越短。
- 极点虚部绝对值越大——振荡频率越高。

§ 1.5

控制系统的基本要求

对任何自动控制系统，通常从以下三个方面评价其性能：

1.5.1 稳定性

稳定性

若系统在初始扰动下产生的响应随 $t \rightarrow \infty$ 趋于零（即回到原平衡状态），则系统是（渐近）稳定的。数学上等价于所有闭环极点均位于 s 左半平面。

稳定性是控制系统工作的前提。一个不稳定的系统无法实际运行。线性时不变系统的稳定性完全由其极点位置决定，与输入信号无关。

1.5.2 快速性（动态性能）

快速性描述系统跟踪输入信号的瞬态响应品质，由以下指标衡量：

- 上升时间 t_r ——响应从稳态值的 10% 上升至 90%（或 0% 至 100%）所需时间。
- 峰值时间 t_p ——响应达到第一个峰值所需时间。
- 调节时间 t_s ——响应进入稳态值 $\pm(2\%$ 或 $5\%)$ 误差带后不再逸出的时刻。
- 超调量 $\sigma\%$ ——最大峰值超出稳态值的百分比。

1.5.3 准确性（稳态性能）

准确性即稳态精度，通常用稳态误差 $e_{ss} = \lim_{t \rightarrow \infty} [r(t) - y(t)]$ 衡量。

系统型别

设开环传递函数为

$$G(s)H(s) = \frac{K(\tau_1 s + 1)(\tau_2 s + 1) \cdots}{s^N (T_1 s + 1)(T_2 s + 1) \cdots}$$

其中 N 为积分环节的个数（即 $s = 0$ 处极点的重数），称系统为 N 型系统。 $N = 0, 1, 2$ 分别对应 0 型、I 型、II 型系统。

型别越高，系统跟踪多项式输入信号的能力越强（稳态误差越小），但稳定性设计也更困难。

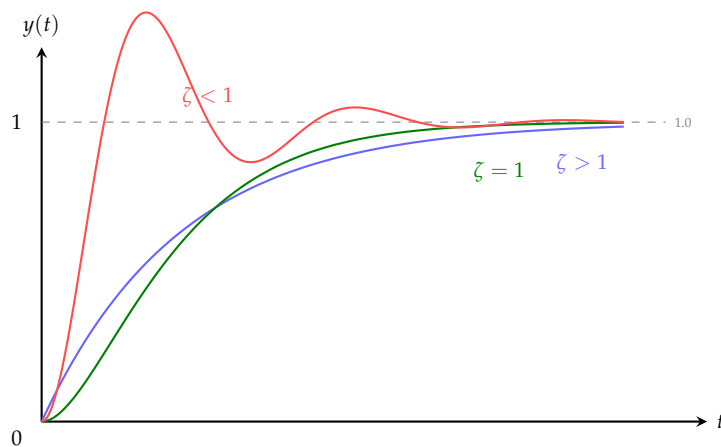


图 1.4: 不同阻尼比 ζ 下的典型二阶系统阶跃响应曲线

Python 示例：计算并绘制阶跃响应

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy import signal
4
5 # 定义二阶系统: G(s) = wn^2 / (s^2 + 2*zeta*wn*s + wn^2)
6 wn = 1.0 # 自然频率
7 zeta_values = [0.2, 0.5, 0.707, 1.0, 2.0]
8
9 t = np.linspace(0, 20, 1000)
10 plt.figure(figsize=(10, 6))
```

```
11
12 for zeta in zeta_values:
13     num = [wn**2]
14     den = [1, 2*zeta*wn, wn**2]
15     sys = signal.TransferFunction(num, den)
16     t_out, y = signal.step(sys, T=t)
17     plt.plot(t_out, y, linewidth=1.8, label=f'$\\zeta={zeta}$')
18
19 plt.xlabel('时间 $t$ (s)')
20 plt.ylabel('响应 $y(t)$')
21 plt.title('不同阻尼比下的二阶系统阶跃响应')
22 plt.legend()
23 plt.grid(True, alpha=0.3)
24 plt.axhline(y=1.0, color='gray', linestyle='--', linewidth=0.8)
25 plt.show()
```

注. 本章建立了控制理论所需的数学基础。Laplace 变换是最核心的工具，它将微分方程转化为代数方程，使传递函数概念成为可能。传递函数的极点和零点分析将在后续各章中贯穿始终。对控制系统稳定性、快速性和准确性的基本要求，构成了整个课程的评价框架。

控制系统数学模型

建立被控对象的数学模型是控制系统分析与设计的第一步。本章系统介绍微分方程建模、传递函数建立、框图化简与信号流图方法。

§2.1 微分方程建模

描述物理系统的数学模型通常是一组微分方程。以下通过几个典型实例说明从物理定律到微分方程的建模过程。

2.1.1 RLC 电路

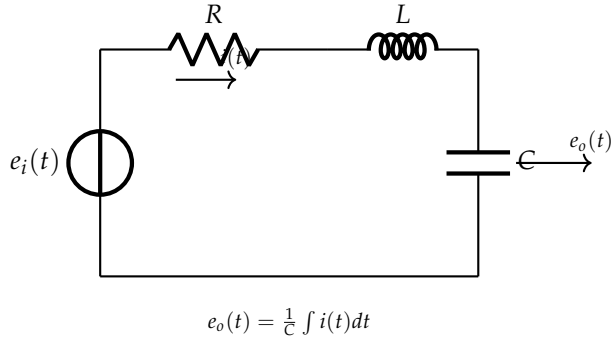


图 2.1: RLC 串联电路

由 Kirchhoff 电压定律:

$$e_i(t) = Ri(t) + L \frac{di}{dt} + \frac{1}{C} \int i(t) dt$$

输出为电容电压: $e_o(t) = \frac{1}{C} \int i(t) dt$ 。代入 $i = C \dot{e}_o$, 得到:

$$LC \frac{d^2 e_o}{dt^2} + RC \frac{de_o}{dt} + e_o = e_i$$

这是典型的二阶线性 ODE。物理参数 L' (电感)、 R (电阻)、 C (电容) 直接决定了系统的动态行为。标准化形式:

$$\ddot{e}_o + 2\zeta\omega_n \dot{e}_o + \omega_n^2 e_o = \omega_n^2 e_i$$

其中自然频率 $\omega_n = 1/\sqrt{LC}$, 阻尼比 $\zeta = \frac{R}{2} \sqrt{C/L}$ 。

取 Laplace 变换 (零初始条件) 得传递函数:

$$G(s) = \frac{E_o(s)}{E_i(s)} = \frac{1}{LCs^2 + RCs + 1} = \frac{\omega_n^2}{s^2 + 2\zeta\omega_n s + \omega_n^2}$$

2.1.2 质量-弹簧-阻尼系统

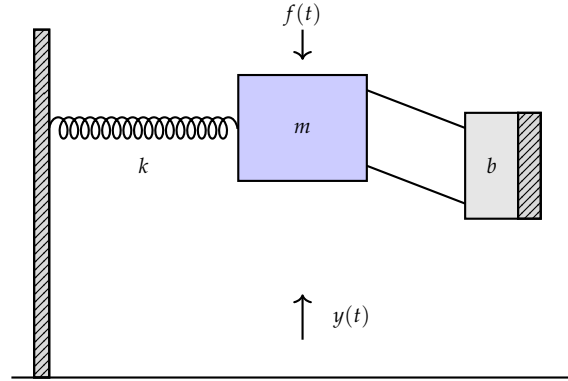


图 2.2: 质量-弹簧-阻尼机械系统

由 Newton 第二定律，受力分析：

$$f(t) - b \frac{dy}{dt} - ky = m \frac{d^2y}{dt^2}$$

整理得：

$$m\ddot{y} + b\dot{y} + ky = f(t)$$

标准化为：

$$\ddot{y} + \frac{b}{m}\dot{y} + \frac{k}{m}y = \frac{1}{m}f(t)$$

取 Laplace 变换得：

$$G(s) = \frac{Y(s)}{F(s)} = \frac{1}{ms^2 + bs + k} = \frac{1/m}{s^2 + (b/m)s + k/m}$$

自然频率 $\omega_n = \sqrt{k/m}$ ，阻尼比 $\zeta = \frac{b}{2\sqrt{mk}}$ 。值得注意的是，RLC 电路和质量-弹簧-阻尼系统具有完全相同的数学结构——这是物理系统之间的“对偶性”（duality），揭示了不同工程领域背后的统一数学框架。

2.1.3 直流电机

电枢控制的直流电机的动态方程为：

$$\begin{cases} L_a \frac{di_a}{dt} + R_a i_a + K_b \dot{\theta} = e_a(t) & (\text{电枢回路}) \\ J\ddot{\theta} + b\dot{\theta} = K_t i_a & (\text{机械负载}) \end{cases}$$

其中 e_a 为电枢电压（输入）， θ 为转角（输出）， K_b 为反电动势系数， K_t 为转矩系数。

忽略电感 L_a （对多数电机近似成立）时，消去 i_a 可得：

$$J\ddot{\theta} + \left(b + \frac{K_b K_t}{R_a}\right)\dot{\theta} = \frac{K_t}{R_a} e_a$$

传递函数为：

$$G(s) = \frac{\Theta(s)}{E_a(s)} = \frac{K_m}{s(\tau_m s + 1)}$$

其中电机增益 $K_m = K_t / (R_a b + K_b K_t)$ ，机电时间常数 $\tau_m = JR_a / (R_a b + K_b K_t)$ 。

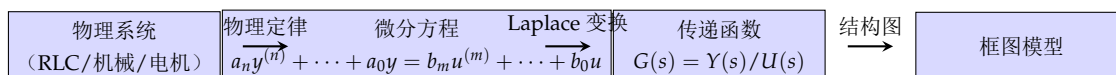


图 2.3: 从物理系统到数学模型的一般建模流程

注. 以上三个实例——RLC 电路、质量-弹簧-阻尼系统和直流电机——分别代表了电气、机械和机电三类对象。它们的建模过程遵循统一范式：物理定律 → 微分方程 → Laplace 变换 → 传递函数。这一范式贯穿整个经典控制理论。

§2.2 传递函数与典型环节

2.2.1 典型环节的传递函数

任何复杂的线性系统均可分解为以下几种基本环节的串联或并联组合：

1. **比例环节 (Proportional):** $G(s) = K$ 。其输出与输入成正比，无时间延迟。例如理想放大器、杠杆机构。阶跃响应为幅度 K 的阶跃。
2. **积分环节 (Integral):** $G(s) = 1/s$ 。输出为输入的积分。阶跃响应为斜率等于 1 的斜坡信号。积分环节具有“记忆”功能，能消除稳态误差，但会引入 -90° 的相位滞后。
3. **微分环节 (Derivative):** $G(s) = s$ 。输出为输入的微分。理想微分环节在物理上不可实现（分子阶次高于分母），实际中使用带滤波的近似微分环节。
4. **惯性环节 (First-order lag):** $G(s) = \frac{K}{Ts+1}$ 。一阶滞后环节，时间常数 T 反映响应速度。阶跃响应为 $y(t) = K(1 - e^{-t/T})$ 。
5. **振荡环节 (Second-order oscillation):** $G(s) = \frac{\omega_n^2}{s^2 + 2\zeta\omega_n s + \omega_n^2}$ 。当 $0 < \zeta < 1$ 时，阶跃响应为衰减振荡； $\zeta \geq 1$ 时为无超调单调过程。
6. **纯滞后环节 (Time delay):** $G(s) = e^{-\tau s}$ 。输出无畸变地滞后输入 τ 秒。在频域中滞后环节对相位的影响极大，是反馈系统设计中的不利因素。

六种典型环节的传递函数汇总：

(2.1)

比例: $G(s) = K$

积分: $G(s) = \frac{1}{s}$

理想微分: $G(s) = s$

惯性: $G(s) = \frac{K}{Ts+1}$

振荡: $G(s) = \frac{\omega_n^2}{s^2 + 2\zeta\omega_n s + \omega_n^2}$

纯滞后: $G(s) = e^{-\tau s}$

§ 2.3 框图及其化简

2.3.1 方框图的基本连接方式

控制系统的功能框图由方框、信号线、比较点和引出点四种基本元素组成。多个环节之间有三种基本连接方式：

串联连接

n 个环节串联时，总传递函数为各环节传递函数的乘积：

$$G(s) = G_1(s)G_2(s) \cdots G_n(s)$$

并联连接

n 个环节并联时，总传递函数为各环节传递函数之和：

$$G(s) = G_1(s) + G_2(s) + \cdots + G_n(s)$$

反馈连接

前向通道 $G(s)$ 与反馈通道 $H(s)$ 构成的闭环传递函数为：

$$\Phi(s) = \frac{Y(s)}{R(s)} = \frac{G(s)}{1 + G(s)H(s)}$$

分母中“+”对应负反馈（比较点处为减号），“-”对应正反馈。

负反馈闭环传函推导. 由框图： $E(s) = R(s) - B(s)$, $B(s) = H(s)Y(s)$, $Y(s) = G(s)E(s)$ 。消去 $E(s)$ 和 $B(s)$ ：

$$Y(s) = G(s)[R(s) - H(s)Y(s)] = G(s)R(s) - G(s)H(s)Y(s)$$

移项：

$$[1 + G(s)H(s)]Y(s) = G(s)R(s) \quad \Rightarrow \quad \Phi(s) = \frac{G(s)}{1 + G(s)H(s)}$$

□

2.3.2 框图等效变换规则

典型的框图化简遵循以下六条等价规则（表2.1）。

表 2.1: 框图等效变换规则

序号	原框图	等效框图
1 (比较点前移)	比较点从环节前移至环节后： 在反馈通道中串联该环节	比较点从环节后移至环节前： 在反馈通道中串联该环节的 倒数
2 (引出点前移)	引出点从环节后移至环节前： 引出通道中串联该环节	引出点从环节前移至环节后： 引出通道中串联该环节的倒 数
3 (串联化简)	$R \rightarrow [G_1] \rightarrow [G_2] \rightarrow Y$	$R \rightarrow [G_1 G_2] \rightarrow Y$
4 (并联化简)	$R \rightarrow \begin{matrix} [G_1] \\ [G_2] \end{matrix} \rightarrow \oplus \rightarrow Y$	$R \rightarrow [G_1 + G_2] \rightarrow Y$
5 (反馈化简)	前向 G , 反馈 H , 比较点 (减 号)	$R \rightarrow [\frac{G}{1+GH}] \rightarrow Y$
6 (单位反馈化)	非单位反馈 $\Phi = \frac{G}{1+GH}$	等价为单位反馈右乘 $1/H$: 或 变换为 $\frac{G}{1+GH} = \frac{1}{H} \cdot \frac{GH}{1+GH}$

框图化简的基本策略是：逐步移动比较点和引出点，消除交叉耦合，将内环化简，最终得到单一传递函数。

【例题 2.1】 化简图 2.4 所示的多回路系统。

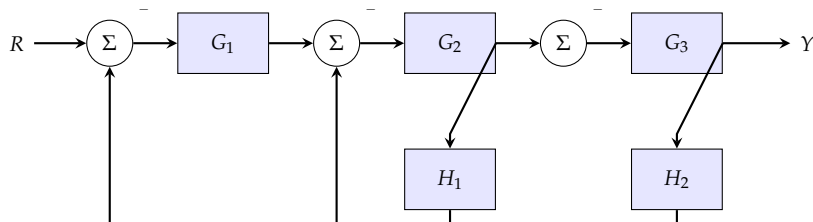


图 2.4: 多回路控制系统框图

解：从最内环开始化简。内环 1 (H_2 反馈环)：

$$G_{23,eq} = \frac{G_2}{1 + G_2 G_3 H_2}$$

化简后，外环变为 H_1 的反馈环，前向通道为 $G_1 G_{23,eq} G_3$ ：

$$\Phi(s) = \frac{G_1 \cdot \frac{G_2}{1+G_2 G_3 H_2} \cdot G_3}{1 + G_1 \cdot \frac{G_2}{1+G_2 G_3 H_2} \cdot G_3 \cdot H_1} = \frac{G_1 G_2 G_3}{1 + G_2 G_3 H_2 + G_1 G_2 G_3 H_1}$$

§ 2.4 信号流图与 Mason 增益公式

2.4.1 信号流图的基本概念

信号流图 (Signal Flow Graph, SFG) 是由 S. J. Mason 于 1953 年提出的表示线性代数方程组因果关系的有向图方法。与框图相比, 信号流图更简洁, 且 Mason 公式可直接给出输入-输出传递函数, 无需逐步化简。

信号流图的基本元素

- 节点——代表变量 (信号)。每个节点对应一个变量值, 等于流入该节点的所有信号之和。
- 支路——连接两个节点的有向线段, 带有增益 (传递函数)。信号沿支路方向单向传递并乘以支路增益。
- 源节点 (输入节点)——只有出支路的节点。
- 汇节点 (输出节点)——只有入支路的节点。
- 通路——顺支路方向连接的节点序列。
- 前向通路——从源节点到汇节点、中间不重复经过任何节点的通路。前向通路增益 P_k 为该通路上各支路增益之积。
- 回路——起点与终点为同一节点、且中间不重复经过任何节点的闭合路径。回路增益 L_j 为回路上各支路增益之积。

2.4.2 Mason 增益公式

Mason 增益公式

从源节点到汇节点的总增益 (传递函数) 为:

$$T = \frac{Y}{R} = \frac{\sum_k P_k \Delta_k}{\Delta}$$

其中:

- $\Delta = 1 - \sum L_a + \sum L_b L_c - \sum L_d L_e L_f + \cdots$ ——图的特征式。第一项为所有回路增益之和, 第二项为所有两两不接触回路增益乘积之和, 第三项为所有三三不接触回路增益乘积之和, 以此类推。
- P_k ——第 k 条前向通路的增益。
- Δ_k ——去掉与第 k 条前向通路接触的所有节点和支路后, 剩余子图的特征式 (即 Δ 中的与 P_k 接触项置零)。

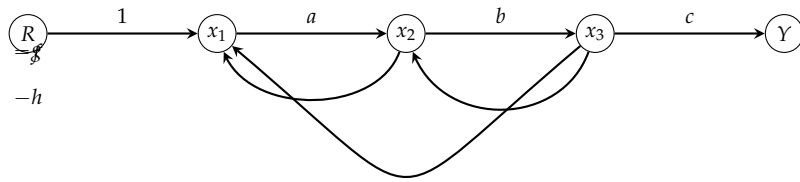


图 2.5: 信号流图示例

以图2.5为例说明 Mason 公式的应用。前向通路： $P_1 = abc$ （只有一条）。回路： $L_1 = -af$ （ $x_1 \rightarrow x_2 \rightarrow x_1$ ）， $L_2 = -bg$ （ $x_2 \rightarrow x_3 \rightarrow x_2$ ）， $L_3 = -ach$ （ $x_1 \rightarrow x_2 \rightarrow x_3 \rightarrow x_1$ ）。互不接触回路对： L_1 与 L_2 互不接触（无公共节点），乘积为 $(-af)(-bg) = abfg$ 。

特征式：

$$\Delta = 1 - (L_1 + L_2 + L_3) + (L_1L_2) = 1 + af + bg + ach + abfg$$

所有回路均与 P_1 接触，故 $\Delta_1 = 1$ 。由 Mason 公式：

$$T = \frac{P_1 \Delta_1}{\Delta} = \frac{abc}{1 + af + bg + ach + abfg}$$

【例题 2.2】将框图2.4转换为信号流图并用 Mason 公式验证结果。

节点设为 R, x_1, x_2, x_3, Y ，其中 x_1 为 G_1 输出， x_2 为 G_2 输出， x_3 为 G_3 输出。前向通路： $P_1 = G_1G_2G_3$ 。回路： $L_1 = -G_2G_3H_2$ ， $L_2 = -G_1G_2G_3H_1$ 。 $\Delta = 1 + G_2G_3H_2 + G_1G_2G_3H_1$ ， $\Delta_1 = 1$ 。

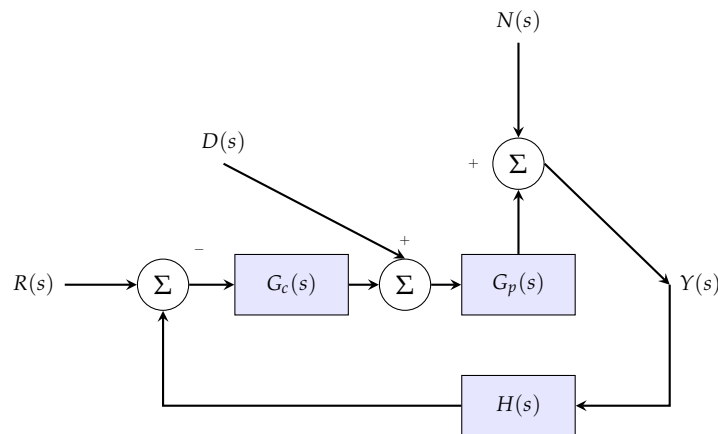
$$T = \frac{G_1G_2G_3}{1 + G_2G_3H_2 + G_1G_2G_3H_1}$$

与框图化简结果一致。

§2.5 典型系统的传递函数

2.5.1 闭环控制系统的基本传递函数

考虑一般反馈控制系统（图2.6），定义以下传递函数：

图 2.6: 考虑扰动 $D(s)$ 和噪声 $N(s)$ 的反馈控制系统

1. 参考输入到输出的闭环传递函数 ($D = 0, N = 0$):

$$\Phi(s) = \frac{Y(s)}{R(s)} = \frac{G_c(s)G_p(s)}{1 + G_c(s)G_p(s)H(s)}$$

2. 误差传递函数 ($D = 0, N = 0$):

$$\Phi_e(s) = \frac{E(s)}{R(s)} = \frac{1}{1 + G_c(s)G_p(s)H(s)}$$

3. 扰动 $D(s)$ 到输出 $Y(s)$ 的传递函数 ($R = 0, N = 0$):

$$\Phi_d(s) = \frac{Y(s)}{D(s)} = \frac{G_p(s)}{1 + G_c(s)G_p(s)H(s)}$$

4. 测量噪声 $N(s)$ 到输出 $Y(s)$ 的传递函数 ($R = 0, D = 0$):

$$\Phi_n(s) = \frac{Y(s)}{N(s)} = -\frac{G_c(s)G_p(s)H(s)}{1 + G_c(s)G_p(s)H(s)}$$

由叠加原理，系统总输出为：

$$Y(s) = \Phi(s)R(s) + \Phi_d(s)D(s) + \Phi_n(s)N(s)$$

注. 四个传递函数具有相同的特征多项式 $1 + G_c G_p H$ ——这正是一般反馈系统的核心特征：系统的稳定性、暂态行为由同一组闭环极点决定，与所考察的具体输入-输出通道无关。

Python 示例：传递函数建模与框图化简验证

```

1 import sympy as sp
2
3 # 定义符号
4 s = sp.symbols('s')
5 G1, G2, G3, H1, H2 = sp.symbols('G1 G2 G3 H1 H2')
6
7 # 方法1：逐步化简
8 # 最内环 G2-G3-H2
9 G23_inner = G2 / (1 + G2 * G3 * H2)
10 # 整个系统
11 T1 = sp.simplify(G1 * G23_inner * G3 / (1 + G1 * G23_inner * G3 * H1))
12 print("逐步化简结果:")
13 sp.pprint(T1)
14
15 # 方法2：Mason公式
16 Delta = 1 + G2*G3*H2 + G1*G2*G3*H1
17 P1 = G1*G2*G3
18 Delta1 = 1
19 T2 = sp.simplify(P1 * Delta1 / Delta)
20 print("\nMason公式结果:")
21 sp.pprint(T2)
22
23 # 验证等价性
24 print("\n两者相等:", sp.simplify(T1 - T2) == 0)

```

注. 本章系统介绍了控制系统数学模型的建立方法。从微分方程到传递函数，从框图化简到 **Mason** 公式，核心思想始终是将复杂的物理系统抽象为简洁的数学描述。典型环节的传递函数是构建任何复杂系统的基本模块，框图化简和 **Mason** 公式则是处理多回路系统的工具——前者直观但步骤繁琐，后者简洁但需仔细识别回路和通路。在后续章节中，本章建立的数学模型将成为时域分析、根轨迹和频域分析的出发点。

时域分析

时域分析是控制系统最基本的分析方法。本章研究系统对典型输入信号的响应特性，建立时域性能指标体系，并介绍 Routh-Hurwitz 代数稳定性判据。

§3.1
典型输入信号

为了横向比较不同系统的性能，需要对输入信号进行标准化。控制理论中常用的典型输入信号包括：

五种典型输入信号的定义：阶跃信号： $r(t) = A \cdot u(t)$, $R(s) = \frac{A}{s}$ (3.1)

斜坡信号： $r(t) = At \cdot u(t)$, $R(s) = \frac{A}{s^2}$ (3.2)

抛物线信号： $r(t) = \frac{A}{2}t^2 \cdot u(t)$, $R(s) = \frac{A}{s^3}$ (3.3)

脉冲信号： $r(t) = A \cdot \delta(t)$, $R(s) = A$ (3.4)

正弦信号： $r(t) = A \sin(\omega t) \cdot u(t)$, $R(s) = \frac{A\omega}{s^2 + \omega^2}$ (3.5)

注. 阶跃信号是最常用也是最重要的测试输入——因为它能充分激励系统的暂态响应，且阶跃型指令在工程中广泛存在（开关启动、设定值突变等）。大多数时域性能指标均基于单位阶跃响应定义。

§3.2
一阶系统的时域分析

3.2.1 一阶系统的数学模型

典型一阶系统的微分方程为：

$$T\dot{y}(t) + y(t) = Ku(t)$$

对应的传递函数为：

$$G(s) = \frac{Y(s)}{U(s)} = \frac{K}{Ts + 1}$$

其中 T 为时间常数（单位：秒）， K 为稳态增益。

3.2.2 一阶系统的单位阶跃响应

输入 $u(t) = u(t)$ （单位阶跃）， $U(s) = 1/s$ ：

$$Y(s) = G(s)U(s) = \frac{K}{Ts + 1} \cdot \frac{1}{s} = K \left(\frac{1}{s} - \frac{T}{Ts + 1} \right)$$

反变换得：

$$y(t) = K(1 - e^{-t/T}), \quad t \geq 0$$

一阶系统阶跃响应的关键特性：

- $t = T$ 时, $y(T) = K(1 - e^{-1}) \approx 0.632K$ 。
- $t = 2T$ 时, $y(2T) \approx 0.865K$ 。
- $t = 3T$ 时, $y(3T) \approx 0.950K$ 。
- $t = 4T$ 时, $y(4T) \approx 0.982K$ 。
- $t = 5T$ 时, $y(5T) \approx 0.993K$ 。

因此，对于一阶系统，通常认为 $t \approx (3 \sim 5)T$ 时暂态过程基本结束。时间常数 T 越小，响应越快。一阶系统的调节时间（按 2% 误差带）为 $t_s \approx 4T$ ，上升时间 $t_r \approx 2.2T$ （10%–90%）。

3.2.3 一阶系统的单位斜坡响应

$$U(s) = 1/s^2:$$

$$Y(s) = \frac{K}{Ts+1} \cdot \frac{1}{s^2} = K \left(\frac{1}{s^2} - \frac{T}{s} + \frac{T^2}{Ts+1} \right)$$

$$y(t) = K(t - T + Te^{-t/T}), \quad t \geq 0$$

稳态时, $y_{ss}(t) = K(t - T)$ ，与输入 Kt 相比存在稳态误差 $e_{ss} = KT$ 。

§ 3.3

二阶系统的时域分析

3.3.1 标准二阶系统的传递函数

典型二阶系统的闭环传递函数标准形式为：

$$\Phi(s) = \frac{Y(s)}{R(s)} = \frac{\omega_n^2}{s^2 + 2\zeta\omega_n s + \omega_n^2}$$

其中 ζ 为阻尼比（damping ratio）， ω_n 为无阻尼自然频率。

系统的特征方程为：

$$s^2 + 2\zeta\omega_n s + \omega_n^2 = 0$$

特征根（闭环极点）：

$$s_{1,2} = -\zeta\omega_n \pm \omega_n \sqrt{\zeta^2 - 1}$$

根据 ζ 的取值范围，二阶系统分为四种工作状态：

- $\zeta = 0$ （无阻尼）： $s_{1,2} = \pm j\omega_n$ ，纯虚极点，等幅振荡。
- $0 < \zeta < 1$ （欠阻尼）： $s_{1,2} = -\zeta\omega_n \pm j\omega_d$ ，共轭复极点， $\omega_d = \omega_n \sqrt{1 - \zeta^2}$ 为阻尼振荡频率。
- $\zeta = 1$ （临界阻尼）： $s_{1,2} = -\omega_n$ ，重实极点。
- $\zeta > 1$ （过阻尼）： $s_{1,2} = -\zeta\omega_n \pm \omega_n \sqrt{\zeta^2 - 1}$ ，两个相异负实极点。

3.3.2 欠阻尼二阶系统的单位阶跃响应

当 $0 < \zeta < 1$ 时，输出响应为：

$$Y(s) = \frac{\omega_n^2}{s^2 + 2\zeta\omega_n s + \omega_n^2} \cdot \frac{1}{s}$$

利用部分分式展开：

$$Y(s) = \frac{1}{s} - \frac{s + \zeta\omega_n}{(s + \zeta\omega_n)^2 + \omega_d^2} - \frac{\zeta\omega_n}{(s + \zeta\omega_n)^2 + \omega_d^2}$$

反变换得典型欠阻尼阶跃响应：

$$y(t) = 1 - \frac{e^{-\zeta\omega_n t}}{\sqrt{1 - \zeta^2}} \sin(\omega_d t + \phi), \quad t \geq 0$$

其中 $\omega_d = \omega_n \sqrt{1 - \zeta^2}$, $\phi = \arctan(\sqrt{1 - \zeta^2}/\zeta) = \arccos(\zeta)$ 。

响应由两部分构成：稳态分量 1，暂态分量为按指数包络 $e^{-\zeta\omega_n t}$ 衰减的正弦振荡。

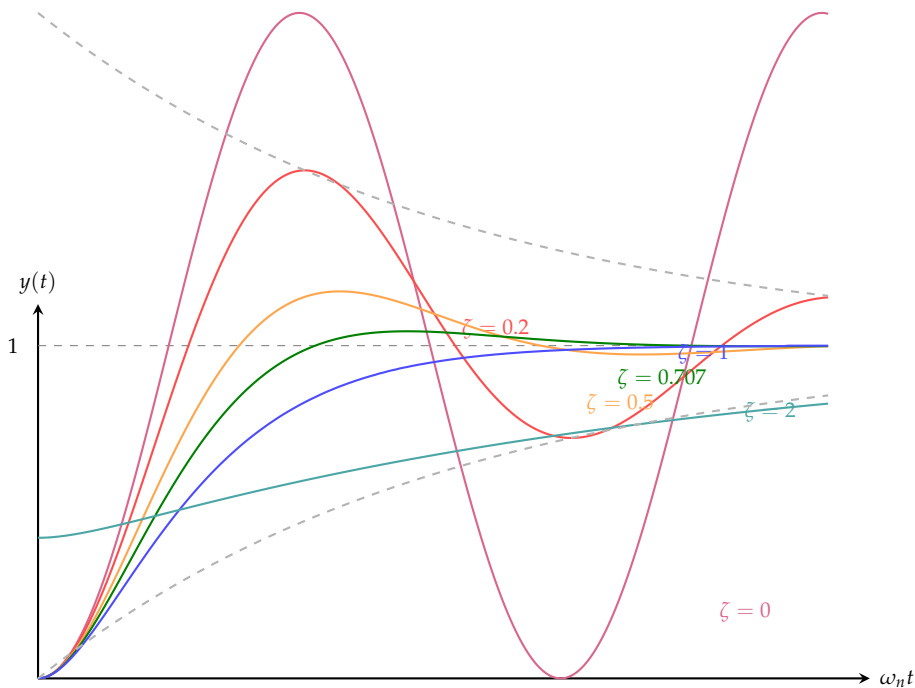


图 3.1: 不同阻尼比 ζ 下标准二阶系统的单位阶跃响应 (ω_n 归一化)

3.3.3 临界阻尼与过阻尼响应

临界阻尼 ($\zeta = 1$): $s_1 = s_2 = -\omega_n$ 。

$$Y(s) = \frac{\omega_n^2}{(s + \omega_n)^2} \cdot \frac{1}{s} = \frac{1}{s} - \frac{1}{s + \omega_n} - \frac{\omega_n}{(s + \omega_n)^2}$$

$$y(t) = 1 - e^{-\omega_n t}(1 + \omega_n t), \quad t \geq 0$$

过阻尼 ($\zeta > 1$): 两个相异负实极点 $s_1 = -(\zeta - \sqrt{\zeta^2 - 1})\omega_n$, $s_2 = -(\zeta + \sqrt{\zeta^2 - 1})\omega_n$ 。

$$y(t) = 1 - \frac{1}{2\sqrt{\zeta^2 - 1}} \left(\frac{e^{s_1 t}}{\zeta - \sqrt{\zeta^2 - 1}} - \frac{e^{s_2 t}}{\zeta + \sqrt{\zeta^2 - 1}} \right)$$

过阻尼响应无振荡、无超调，但响应速度随 ζ 增大而变慢。

注. 对实际控制系统， ζ 的选择需要在响应速度和超调量之间折中。工程中常取 $\zeta \approx 0.707$ (即“最佳阻尼比”)，此时超调量约 4.3%，调节时间最短。随动系统通常选 $\zeta = 0.5 \sim 0.8$ 。

§3.4 时域性能指标

时域性能指标基于单位阶跃响应定义，描述系统的暂态品质和稳态精度。

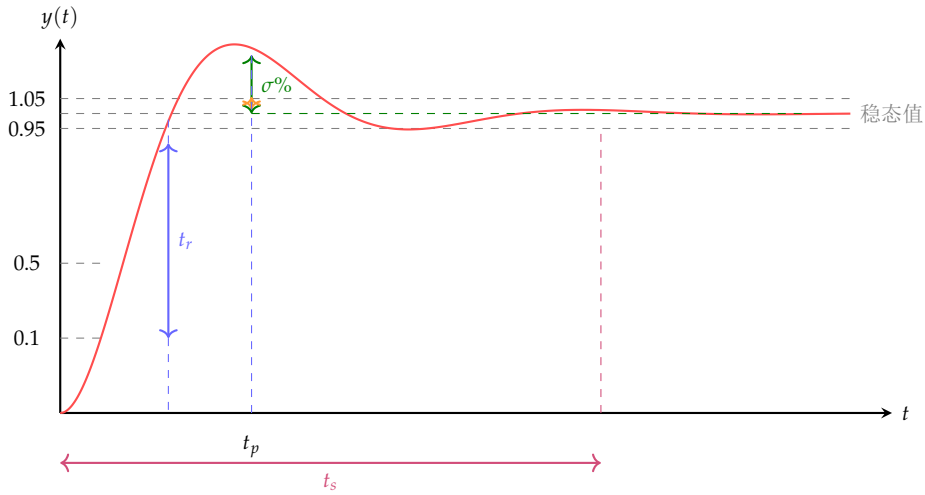


图 3.2: 阶跃响应时域性能指标示意

3.4.1 上升时间 t_r

响应从稳态值的 10%（或 0%）上升至 90%（或 100%）所需时间。对于欠阻尼二阶系统：

$$t_r \approx \frac{1 + 0.7\zeta}{\omega_n}$$

3.4.2 峰值时间 t_p

响应达到第一个峰值所需时间。令 $\frac{dy}{dt} = 0$ 得：

$$t_p = \frac{\pi}{\omega_d} = \frac{\pi}{\omega_n \sqrt{1 - \zeta^2}}$$

3.4.3 超调量 $\sigma\%$

定义为超出稳态值的最大幅值与稳态值之比（百分比）：

$$\sigma\% = \frac{y_{\max} - y(\infty)}{y(\infty)} \times 100\%$$

将 t_p 代入响应表达式：

$$\sigma\% = e^{-\pi\zeta/\sqrt{1-\zeta^2}} \times 100\%$$

超调量仅取决于阻尼比 ζ 。例如 $\zeta = 0.5$ 时 $\sigma\% \approx 16.3\%$ ， $\zeta = 0.707$ 时 $\sigma\% \approx 4.3\%$ ， $\zeta = 1$ 时 $\sigma\% = 0$ 。

3.4.4 调节时间 t_s

响应进入并保持在稳态值 $\pm\Delta$ （通常 $\Delta = 2\%$ 或 5% ）误差带内所需的最短时间。对于欠阻尼系统，由衰减包络 $1 \pm e^{-\zeta\omega_n t_s} / \sqrt{1 - \zeta^2} = 1 \pm \Delta$ ，取 $\Delta = 2\%$ ：

$$t_s \approx \frac{4}{\zeta\omega_n} \quad (2\% \text{ 准则}), \quad t_s \approx \frac{3}{\zeta\omega_n} \quad (5\% \text{ 准则})$$

注. 四个性能指标之间相互制约。增大 ω_n 可同时减小 t_r 和 t_p ，但不影响 $\sigma\%$ 。增大 ζ 可减小 $\sigma\%$ 和振荡次数，但会增大 t_r 。设计者必须在这些指标之间进行合理的折中。

Python 示例：二阶系统时域性能指标计算

```

1 import numpy as np
2 from scipy import signal
3
4 def second_order_performance(zeta, wn, delta=0.02):
5     """计算二阶系统的时域性能指标"""
6     wd = wn * np.sqrt(1 - zeta**2)
7     sigma = np.exp(-np.pi * zeta / np.sqrt(1 - zeta**2))
8
9     tr = (1 + 0.7*zeta) / wn          # 近似上升时间
10    tp = np.pi / wd                  # 峰值时间
11    ts = 4.0 / (zeta * wn) if delta == 0.02 else 3.0 / (zeta * wn) # 调节时间
12
13    print(f"阻尼比 zeta = {zeta:.3f}")
14    print(f"自然频率 omega_n = {wn:.2f} rad/s")
15    print(f"上升时间 t_r = {tr:.3f} s")
16    print(f"峰值时间 t_p = {tp:.3f} s")
17    print(f"超调量 sigma = {sigma*100:.2f}%")
18    print(f"调节时间 t_s ({delta*100:.0f}%) = {ts:.3f} s")
19
20    # 仿真验证
21    num = [wn**2]
22    den = [1, 2*zeta*wn, wn**2]
23    sys = signal.TransferFunction(num, den)
24    t, y = signal.step(sys)
25    y_max_idx = np.argmax(y)
26    print(f"仿真超调量 = {(y[y_max_idx]-1.0)*100:.2f}%")
27    print(f"仿真峰值时间 = {t[y_max_idx]:.3f} s")
28    print("-" * 40)
29
30    # 示例
31    second_order_performance(0.4, 2.0)
32    second_order_performance(0.707, 2.0)
33    second_order_performance(0.9, 2.0)

```

§3.5 稳态误差分析

3.5.1 终值定理与稳态误差

稳态误差定义为 $e_{ss} = \lim_{t \rightarrow \infty} e(t) = \lim_{t \rightarrow \infty} [r(t) - y(t)]$ 。当 $e(t)$ 的 Laplace 变换 $E(s)$ 满足终值定理条件时：

$$e_{ss} = \lim_{t \rightarrow \infty} e(t) = \lim_{s \rightarrow 0} sE(s)$$

对于单位反馈系统 ($H(s) = 1$)，误差传递函数为 $\Phi_e(s) = 1/(1 + G(s))$ ，故：

$$e_{ss} = \lim_{s \rightarrow 0} s \cdot \frac{1}{1 + G(s)} \cdot R(s)$$

3.5.2 系统型别与静态误差系数

设开环传递函数 $G(s)$ 的标准形式为：

$$G(s) = \frac{K(\tau_1 s + 1)(\tau_2 s + 1) \cdots (\tau_q s + 1)}{s^N (T_1 s + 1)(T_2 s + 1) \cdots (T_p s + 1)}$$

其中 N 为积分环节个数（ $s = 0$ 处极点的重数），称为系统**型别**。

定义三个静态误差系数：

- 位置误差系数： $K_p = \lim_{s \rightarrow 0} G(s)$
- 速度误差系数： $K_v = \lim_{s \rightarrow 0} sG(s)$
- 加速度误差系数： $K_a = \lim_{s \rightarrow 0} s^2 G(s)$

$$\text{不同型别系统在不同输入下的稳态误差：阶跃输入}(R(s) = 1/s) : e_{ss} = \frac{1}{1 + K_p} \quad (3.6)$$

$$\text{斜坡输入}(R(s) = 1/s^2) : e_{ss} = \frac{1}{K_v} \quad (3.7)$$

$$\text{抛物线输入}(R(s) = 1/s^3) : e_{ss} = \frac{1}{K_a} \quad (3.8)$$

表 3.1: 系统型别与稳态误差

型别	K_p	K_v	K_a	$e_{ss}(\text{阶跃})$	$e_{ss}(\text{斜坡})$	$e_{ss}(\text{抛物线})$
0 型	K	0	0	$1/(1 + K)$	∞	∞
I 型	∞	K	0	0	$1/K$	∞
II 型	∞	∞	K	0	0	$1/K$

从表中可见，提高系统型别可以消除对低阶输入的稳态误差，但型别越高，稳定性设计越困难——这反映了控制理论中“精度”与“稳定性”之间的本质矛盾。

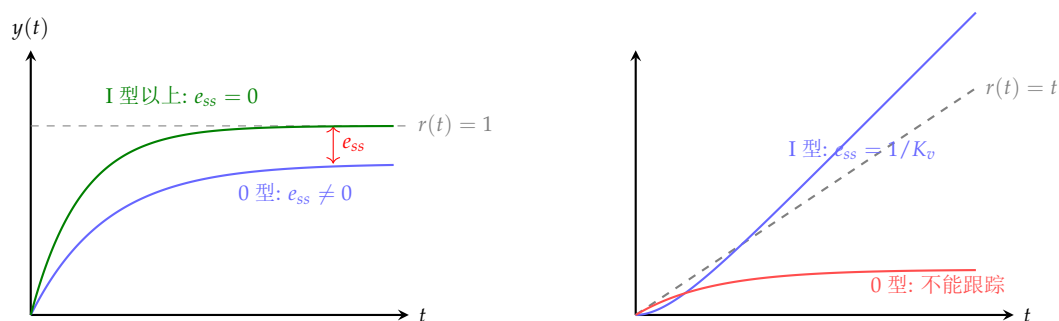


图 3.3: 不同型别系统的稳态误差示意（左：阶跃响应；右：斜坡响应）

§ 3.6 稳定性代数判据

3.6.1 稳定性概念与充要条件

BIBO 稳定性

线性时不变系统是 BIBO（有界输入-有界输出）稳定的，当且仅当系统的所有闭环极点均位于 s 平面的左半平面（即具有负实部）。

稳定性充要条件

连续时间 LTI 系统稳定的充要条件是其闭环特征方程

$$\Delta(s) = a_n s^n + a_{n-1} s^{n-1} + \cdots + a_1 s + a_0 = 0$$

的所有根均具有负实部（即位于左半 s 平面）。

直接求解高阶特征方程的根是困难的。Routh 和 Hurwitz 分别于 1877 年和 1895 年提出了仅需代数运算即可判断稳定性的方法——无需实际求解特征根。

3.6.2 Routh-Hurwitz 判据

Routh-Hurwitz 稳定性判据

特征多项式 $\Delta(s) = a_n s^n + a_{n-1} s^{n-1} + \cdots + a_0$ ($a_n > 0$) 的所有根均具有负实部的必要条件是：

1. 特征多项式各项系数均存在且同号（即 $a_i > 0$ ，对所有 $i = 0, 1, \dots, n$ ）。

充分条件（Routh 判据）：构造 Routh 表，若 Routh 表第一列所有元素均大于零，则系统稳定。若第一列元素符号改变 m 次，则特征方程有 m 个根位于右半平面。

3.6.3 Routh 表的构造

对于特征多项式 $\Delta(s) = a_n s^n + a_{n-1} s^{n-1} + a_{n-2} s^{n-2} + \cdots + a_1 s + a_0$ ，构造 Routh 表如下：

s^n :	a_n	a_{n-2}	a_{n-4}	$\dots$
s^{n-1} :	a_{n-1}	a_{n-3}	a_{n-5}	$\dots$
s^{n-2} :	b_1	b_2	b_3	$\dots$
s^{n-3} :	c_1	c_2	c_3	$\dots$
$\vdots$				
s^0 :	p_1			

$$b_1 = \frac{a_{n-1}a_{n-2} - a_n a_{n-3}}{a_{n-1}}, \quad b_2 = \frac{a_{n-1}a_{n-4} - a_n a_{n-5}}{a_{n-1}}, \quad \dots$$

$$c_1 = \frac{b_1 a_{n-3} - a_{n-1} b_2}{b_1}, \quad c_2 = \frac{b_1 a_{n-5} - a_{n-1} b_3}{b_1}, \quad \dots$$

图 3.4: Routh 表的构造规则示意

表的构造规则：第 k 行的元素利用第 $(k-1)$ 行和第 $(k-2)$ 行的前两列元素按“十字交叉”法则计算：

$$\text{新元素} = \frac{(\text{上一行第一列}) \times (\text{上二行下一列}) - (\text{上二行第一列}) \times (\text{上一行下一列})}{\text{上一行第一列}}$$

3.6.4 Routh 判据的特殊情况

情况一：Routh 表某行第一列元素为零，该行其余元素不全为零。

处理方法：用一个小的正数 ε 代替零，继续计算 Routh 表，然后令 $\varepsilon \rightarrow 0^+$ 检查第一列符号变化。

情况二：Routh 表某行整行全为零。

这表明特征方程存在大小相等、符号相反的根（即关于原点对称的根对），如 $\pm\sigma$ 、 $\pm j\omega$ 或 $\pm\sigma \pm j\omega$ 。

处理方法：

1. 利用上一行（全零行的上一行）的元素构造辅助多项式 $A(s)$ 。
2. 对 $A(s)$ 求导得 $dA(s)/ds$ ，用其系数代替全零行。
3. 辅助方程 $A(s) = 0$ 的根即为对称根，可通过求解 $A(s) = 0$ 获得。

【例题 3.1】 判断特征方程 $\Delta(s) = s^4 + s^3 + 3s^2 + 2s + 2 = 0$ 对应系统的稳定性。

构造 Routh 表：

$$\begin{array}{l|lll} s^4 & 1 & 3 & 2 \\ s^3 & 1 & 2 & 0 \\ s^2 & \frac{1 \cdot 3 - 1 \cdot 2}{1} = 1 & \frac{1 \cdot 2 - 1 \cdot 0}{1} = 2 & \\ s^1 & \frac{1 \cdot 2 - 1 \cdot 2}{1} = 0 & \dots & \end{array}$$

s^1 行第一列为零！用 ε 代替： s^1 行为 ε 、0； s^0 行为 $\frac{\varepsilon \cdot 0 - 1 \cdot 0}{\varepsilon} = 0$ （或直接计算为 2）。仔细计算： s^1 行元素为 $(\varepsilon \cdot 2 - 1 \cdot 0)/\varepsilon = 2$ 。第一列：1, 1, 1, ε , 2。当 $\varepsilon \rightarrow 0^+$ 时符号不变，但需要进一步检查 s^1 行对应的辅助项。

实际上计算后得辅助多项式 $A(s) = s^2 + 2$ ，求导得 $2s$ ， s^1 行系数为 $[2, 0]$ ， s^0 行为 2。第一列全为正——系统稳定。但辅助方程 $s^2 + 2 = 0$ 给出 $s = \pm j\sqrt{2}$ ，表明系统有一对纯虚极点——临界稳定。

Python 示例：Routh 判据实现

```

1 import numpy as np
2
3 def routh_table(coeffs):
4     """构造Routh表，输入特征多项式系数（降幂）"""
5     n = len(coeffs) - 1 # 阶次
6     rows = n + 1
7     cols = (n + 2) // 2
8     table = np.zeros((rows, cols))
9
10    # 填充前两行
11    for j in range(cols):
12        if 2*j < len(coeffs):
13            table[0, j] = coeffs[2*j]
14        if 2*j + 1 < len(coeffs):
15            table[1, j] = coeffs[2*j + 1]
16
17    # 计算后续行
18    for i in range(2, rows):
19        for j in range(cols - 1):
20            if abs(table[i-1, 0]) < 1e-12:
21                # 首列为零，用eps替代
22                table[i-1, 0] = 1e-6
23            table[i, j] = (table[i-1, 0] * table[i-2, j+1] -
24                           table[i-2, 0] * table[i-1, j+1]) / table[i-1, 0]
25
26    print("Routh表第一列:", table[:, 0])
27    sign_changes = sum(1 for i in range(1, len(table[:, 0]))
28                       if table[i-1, 0] * table[i, 0] < 0)
29    if sign_changes > 0:
30        print(f"不稳定！右半平面极点个数 = {sign_changes}")
31    else:
32        print("所有极点均位于左半平面（或虚轴），系统稳定。")
33    return table
34
35    # 测试:  $s^4 + 2s^3 + 3s^2 + 4s + 5$ 
36    coeffs = [1, 2, 3, 4, 5]
37    routh_table(coeffs)
38
39    # 测试:  $s^3 + s^2 + 2s + 24$ 
40    coeffs2 = [1, 1, 2, 24]
41    routh_table(coeffs2)

```

注 Routh-Hurwitz 判据是经典控制理论中最优雅的成果之一——它仅需乘除运算即可判定特征方程的稳定性，无需进行多项式求根。然而它只能回答“是否稳定”，无法给出稳定程度（相对稳定性）。对于相对稳定性的评估，将在第四章（根轨迹法）和频域分析中进一步讨论。

本章小结

本章从时域角度全面分析了控制系统的动态与稳态行为。一阶和二阶系统是理解高阶系统响应的基础——高阶系统的主极点可以通过降阶近似为低阶标准型进行分析。时域性能指标 ($t_r, t_p, \sigma\%, t_s$) 是刻画暂态品质的定量标准，其解析表达式揭示了系统参数 (ζ 和 ω_n) 与性能之间的函数关系。稳态误差分析则通过终值定理和静态误差系数建立系统型别与跟踪精度的联系。Routh-Hurwitz 判据作为稳定性代数判据，是系统综合设计中的第一道检验。

根轨迹法

根轨迹法是 W. R. Evans 于 1948 年提出的一种图解法，用于分析闭环系统极点随增益参数变化的轨迹。它不需复杂的代数运算即可直观揭示系统动态特性随参数的变化规律。

§4.1 根轨迹的基本概念

4.1.1 问题的提出

给定反馈系统，前向通道传递函数为 $KG(s)$ ，反馈通道为 $H(s)$ ，其中 $K \geq 0$ 为可调增益。闭环特征方程为：

$$1 + KG(s)H(s) = 0$$

等价于：

$$KG(s)H(s) = -1$$

当 K 从 0 变化到 $+\infty$ 时，闭环特征方程的根（即闭环极点）在复平面上描绘出的轨迹称为**根轨迹**。

根轨迹（Root Locus）

根轨迹是闭环系统特征方程的根（闭环极点）在复平面上随某一参数（通常为开环增益 K ）从 0 变化至 $+\infty$ 时形成的轨迹曲线。根轨迹始于开环极点（ $K = 0$ ），终于开环零点（ $K \rightarrow \infty$ ）。

根轨迹法的核心价值在于：开环的零极点分布已知，即可绘制闭环极点轨迹，进而分析闭环系统的稳定性、动态品质和稳态误差，而无需直接求解高阶特征方程。

4.1.2 开环传递函数与闭环极点的关系

将开环传递函数写为零极点形式：

$$G(s)H(s) = \frac{N(s)}{D(s)} = K^* \frac{\prod_{i=1}^m (s - z_i)}{\prod_{j=1}^n (s - p_j)}$$

其中 K^* 为根轨迹增益， z_i 为开环零点， p_j 为开环极点。根轨迹方程为：

$$1 + K^* \frac{\prod_{i=1}^m (s - z_i)}{\prod_{j=1}^n (s - p_j)} = 0$$

即：

$$\prod_{j=1}^n (s - p_j) + K^* \prod_{i=1}^m (s - z_i) = 0$$

当 $K^* = 0$ 时，方程退化为 $\prod_{j=1}^n (s - p_j) = 0$ ，闭环极点为开环极点 p_j 。当 $K^* \rightarrow \infty$ 时，方程退化为 $\prod_{i=1}^m (s - z_i) = 0$ ，闭环极点趋于开环零点 z_i 。若 $n > m$ （物理系统通常如此），则有 $(n - m)$ 条根轨迹分支趋于无穷远。

§4.2 幅值条件与相角条件

根轨迹方程 $KG(s)H(s) = -1$ 可分解为两个等价条件：

$$\text{根轨迹的幅值条件与相角条件：幅值条件： } |KG(s)H(s)| = 1, \quad \text{即 } K = \frac{1}{|G(s)H(s)|} \quad (4.1)$$

$$\text{相角条件： } \angle G(s)H(s) = \pm(2k+1)\pi, \quad k = 0, 1, 2, \dots \quad (4.2)$$

相角条件的推导. $KG(s)H(s) = -1 = 1 \cdot e^{j(2k+1)\pi}$, 故 $\angle[KG(s)H(s)] = \angle G(s)H(s) = (2k+1)\pi$ (增益 K 为正实数, 辐角为 0)。 $\square$

相角条件是判定复平面上某点 s 是否在根轨迹上的判据——它不依赖于增益 K 。幅值条件则用于确定根轨迹上的 s 点对应的 K 值。

用零极点形式表达：

- 相角条件： $\sum_{i=1}^m \angle(s - z_i) - \sum_{j=1}^n \angle(s - p_j) = (2k+1)\pi$
- 幅值条件： $K^* = \frac{\prod_{j=1}^n |s - p_j|}{\prod_{i=1}^m |s - z_i|}$

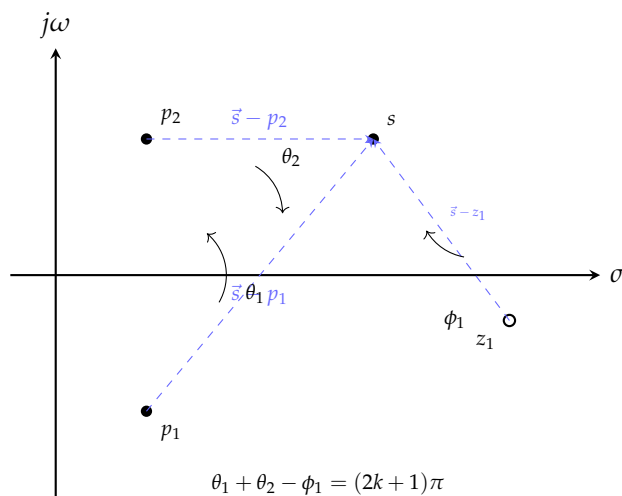


图 4.1: 相角条件的几何解释：从各极点/零点指向试探点 s 的向量辐角之和满足 $(2k+1)\pi$

§4.3 根轨迹的绘制规则

基于相角条件和幅值条件, W. R. Evans 于 1948 年总结了 10 条根轨迹绘制规则。掌握这些规则即可快速、准确地勾画根轨迹的大致形状和走向。

根轨迹的 10 条绘制规则 (Evans 法则)

1. **起点与终点**: 根轨迹有 n 条分支。 $K^* = 0$ 时各分支从开环极点 p_j 出发; $K^* \rightarrow \infty$ 时 m 条分支终止于开环零点 z_i , 剩余 $(n - m)$ 条分支沿渐近线趋于无穷远。

2. **分支数、连续性与对称性**: 根轨迹有 $n = \max(n, m)$ 条分支; 各分支是 K^* 的连续曲线; 且关于实轴对称 (因为特征方程为实系数方程)。

3. **实轴上的根轨迹**: 实轴上某区域属于根轨迹的充要条件是——该区域右侧的开环实极点和实零点的总个数为奇数。

证明: 实轴上任意试探点 s , 所有共轭复数和实轴上左侧的零极点对其提供的相角代数和为 0; 只有右侧的实极点和实零点中每个提供 $+\pi$ 或 $-\pi$ (取决于方向)。因此右侧奇数个点时总相角为奇数倍 π , 满足相角条件。

4. **渐近线**: $(n - m)$ 条趋于无穷的分支沿 $(n - m)$ 条渐近线趋近。渐近线与实轴的交点 σ_a 和夹角 ϕ_a 为:

$$\sigma_a = \frac{\sum_{j=1}^n p_j - \sum_{i=1}^m z_i}{n - m}, \quad \phi_a = \frac{(2k + 1)\pi}{n - m}, \quad k = 0, 1, \dots, n - m - 1$$

5. **分离点 (会合点)**: 两条或以上根轨迹分支在实轴上相遇后又分开的点。分离点 s_b 满足:

$$\sum_{j=1}^n \frac{1}{s_b - p_j} = \sum_{i=1}^m \frac{1}{s_b - z_i}$$

等价于 $\frac{d}{ds} \left[\frac{D(s)}{N(s)} \right] = 0$ 或 $\frac{dK^*}{ds} = 0$ 。

6. **出射角 (从复极点出发的角度) 与入射角 (进入复零点的角度)**: 从复极点 p_ℓ 的出射角:

$$\theta_{p_\ell} = 180^\circ - \sum_{j \neq \ell} \angle(p_\ell - p_j) + \sum_{i=1}^m \angle(p_\ell - z_i)$$

进入复零点 z_ℓ 的入射角:

$$\phi_{z_\ell} = 180^\circ + \sum_{j=1}^n \angle(z_\ell - p_j) - \sum_{i \neq \ell} \angle(z_\ell - z_i)$$

7. **虚轴交点**: 根轨迹与虚轴的交点 $s = j\omega$ 可通过两种方法求得: (a) 令特征方程 $1 + G(j\omega)H(j\omega) = 0$, 即 $\Re[1 + GH] = 0$ 且 $\Im[1 + GH] = 0$, 联立求解 ω 和 K^* ; (b) 用 Routh 判据确定临界增益。

8. **根之和**: 当 $n - m \geq 2$ 时, 闭环极点之和等于开环极点之和 (与 K^* 无关):

$$\sum_{j=1}^n \lambda_j = \sum_{j=1}^n p_j = \text{常数}$$

这一性质意味着此时闭环极点组的重心在复平面上的位置不随增益变化——某条分支向左移动必然伴随着另一分支向右移动。

9. **根之积**: 闭环极点之积与开环增益有关:

$$\prod_{j=1}^n |\lambda_j| = \left| \prod_{j=1}^n p_j + K^* \prod_{i=1}^m z_i \right|$$

10. **增益 K^* 的确定**: 根轨迹上任意点 s_x 对应的增益值由幅值条件给出:

$$K^* = \frac{\prod_{j=1}^n |s_x - p_j|}{\prod_{i=1}^m |s_x - z_i|}$$

注. 这 10 条规则中，规则 1（起点终点）、规则 3（实轴分布）、规则 4（渐近线）、规则 5（分离点）、规则 6（出射入射角）和规则 7（虚轴交点）是绘制准确根轨迹的关键。规则 8（根之和）和规则 9（根之积）主要用于验证和简化计算。规则 2（对称性）总是成立。

4.3.1 绘制规则的应用示例——二阶系统

考虑开环传递函数：

$$G(s)H(s) = \frac{K}{s(s+a)}, \quad a > 0$$

步骤 1： 开环极点 $p_1 = 0$, $p_2 = -a$ ；无有限零点； $n = 2, m = 0$ 。

步骤 2： $n - m = 2$ 条渐近线。

$$\sigma_a = \frac{0 + (-a) - 0}{2} = -\frac{a}{2}, \quad \phi_a = \frac{(2k+1)\pi}{2} = \frac{\pi}{2}, \frac{3\pi}{2}$$

即渐近线为过点 $(-a/2, 0)$ 的两条垂直线。

步骤 3： 实轴分布： $[-a, 0]$ 段在根轨迹上（该段右侧有 2 个极点——偶数？等等，仔细算：在 $[-a, 0]$ 段内，右侧有 $p_1 = 0$ ——1 个，奇数个，所以在轨迹上）。正确：区间 $(-\infty, -a]$ 右侧 2 个极点（偶数），不在轨迹上；区间 $[-a, 0]$ 右侧 1 个极点（奇数），在轨迹上；区间 $[0, \infty)$ 右侧 0 个，不在轨迹上。

步骤 4： 分离点：由规则 5，

$$\frac{1}{s_b} + \frac{1}{s_b + a} = 0 \implies 2s_b + a = 0 \implies s_b = -\frac{a}{2}$$

分离角为 $\pm 90^\circ$ （两分支对称离开实轴）。

步骤 5： 根轨迹：始于 0 和 $-a$ ，沿实轴相向运动，在 $-a/2$ 汇合后以 $\pm 90^\circ$ 角度垂直离开实轴，沿渐近线趋于无穷。该轨迹是垂直于实轴通过 $(-a/2, 0)$ 的直线。

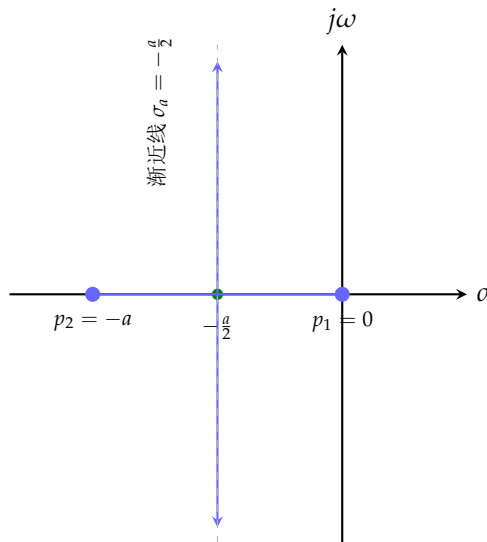


图 4.2: 二阶系统 $G(s) = K/[s(s+a)]$ 的根轨迹

此系统具有重要的物理意义：无论 K 取何正值，闭环极点始终具有负实部，系统总是稳定的。当 K 增大时，阻尼比 ζ 减小，响应振荡加剧，但系统始终稳定——这是二阶系统的固有特性，也是“二阶系统无条件稳定”这一结论在根轨迹中的直观体现。

4.3.2 三阶系统的根轨迹

考虑 $G(s)H(s) = \frac{K}{s(s+1)(s+2)}$ 。

开环极点 $p_1 = 0, p_2 = -1, p_3 = -2; n = 3, m = 0$ 。

渐近线: $n - m = 3$ 。

$$\sigma_a = \frac{0 - 1 - 2}{3} = -1, \quad \phi_a = \frac{\pi}{3}, \pi, \frac{5\pi}{3} \quad (60^\circ, 180^\circ, 300^\circ)$$

实轴分布: $[-1, 0]$ 段和 $(-\infty, -2]$ 段在轨迹上。

分离点:

$$\frac{1}{s} + \frac{1}{s+1} + \frac{1}{s+2} = 0 \implies 3s^2 + 6s + 2 = 0$$

解得 $s_{b1} \approx -0.423$ (在 $[-1, 0]$ 内), $s_{b2} \approx -1.577$ 。 -1.577 不在实轴根轨迹段内 ($[-2, -1]$ 段不在轨迹上), 故仅 $s_{b1} \approx -0.423$ 为有效分离点。

虚轴交点: 特征方程 $s^3 + 3s^2 + 2s + K = 0$ 。构造 Routh 表:

$$\begin{array}{l|ll} s^3 & 1 & 2 \\ s^2 & 3 & K \\ s^1 & \frac{6-K}{3} & 0 \\ s^0 & K & \end{array}$$

令 s^1 行第一列 $(6-K)/3 = 0$, 得临界增益 $K_c = 6$ 。辅助方程 $3s^2 + K = 0$, 代入 $K = 6$: $3s^2 + 6 = 0$, $s = \pm j\sqrt{2}$ 。

结论: 当 $0 < K < 6$ 时系统稳定; 当 $K = 6$ 时临界稳定 (纯虚极点 $\pm j\sqrt{2}$); 当 $K > 6$ 时系统不稳定 (两条分支进入右半平面)。

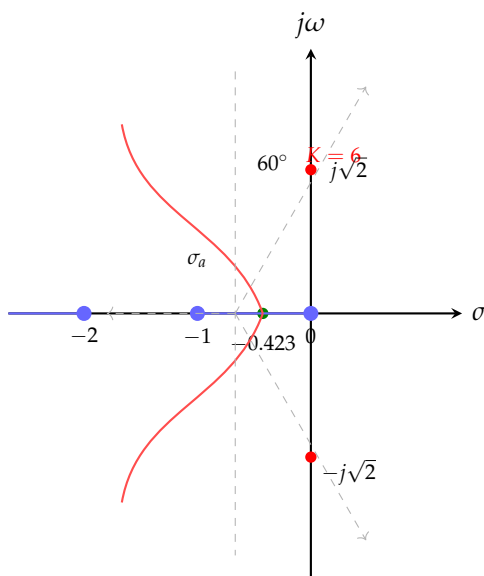


图 4.3: 三阶系统 $G(s) = K/[s(s+1)(s+2)]$ 的根轨迹

Python 示例: 用 Python 绘制根轨迹

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy import signal
4
5 def plot_root_locus(num, den, K_range=None):
6     """绘制根轨迹（简易版：对K采样求闭环极点）"""
7     if K_range is None:
8         K_range = np.logspace(-2, 2, 500)
9
10    poles_all = []
11    for K in K_range:
12        # 闭环特征多项式: den(s) + K * num(s)
13        # 注意: num和den必须同长, 不足补0
14        n = max(len(num), len(den))
15        num_pad = np.hstack([np.zeros(n - len(num)), num])
16        den_pad = np.hstack([np.zeros(n - len(den)), den])
17        closed_den = den_pad + K * num_pad
18        poles = np.roots(closed_den)
19        poles_all.append(poles)
20
21    poles_all = np.array(poles_all)
22
23    plt.figure(figsize=(10, 8))
24    # 绘制各极点随K变化的轨迹
25    for j in range(poles_all.shape[1]):
26        plt.plot(np.real(poles_all[:, j]),
27                 np.imag(poles_all[:, j]), 'b.', markersize=1)
28
29    # 标注开环极点和零点
30    open_poles = np.roots(den)
31    open_zeros = np.roots(num)
32    plt.plot(np.real(open_poles), np.imag(open_poles),
33             'rx', markersize=10, markeredgewidth=2, label='开环极点')
34    if len(open_zeros) > 0:
35        plt.plot(np.real(open_zeros), np.imag(open_zeros),
36                 'go', markersize=10, label='开环零点')
37
38    plt.axhline(y=0, color='k', linewidth=0.5)
39    plt.axvline(x=0, color='k', linewidth=0.5)
40    plt.xlabel('实轴  $\sigma$ ')
41    plt.ylabel('虚轴  $j\omega$ ')
42    plt.title('根轨迹图')
43    plt.legend()
44    plt.grid(True, alpha=0.3)
45    plt.axis('equal')
46    plt.show()
47
48 # 示例:  $G(s)H(s) = K / [s(s+1)(s+2)]$ 
49 num = [1]
50 den = [1, 3, 2, 0]

```

```
51 plot_root_locus(num, den, K_range=np.logspace(-2, 1.2, 800))
```

§4.4 参数根轨迹

4.4.1 非增益参数变化的根轨迹

有时需要分析除开环增益 K 以外的其他参数变化对系统性能的影响。以 $G(s)H(s) = \frac{10}{s(s+T)}$ 为例，分析时间常数 T 从 0 到 ∞ 变化时的根轨迹。

此时特征方程为 $s^2 + Ts + 10 = 0$ 。将含 T 的项移到分子：

$$1 + T \cdot \frac{s}{s^2 + 10} = 0$$

等效开环传递函数为 $G_{eq}(s) = \frac{s}{s^2 + 10}$ ， T 为“等效增益”。开环极点 $p_{1,2} = \pm j\sqrt{10}$ ，开环零点 $z_1 = 0$ 。

$n = 2, m = 1$ ，渐近线： $\sigma_a = \frac{0-0}{1} = 0$ （实际上在此例中确切形式需要具体分析... 更准确地：原特征方程为 $s^2 + Ts + 10 = 0$ ，即 $s = \frac{-T \pm \sqrt{T^2 - 40}}{2}$ ）。当 $T < 2\sqrt{10} \approx 6.32$ 时系统欠阻尼； $T = 6.32$ 时临界阻尼； $T > 6.32$ 时过阻尼。

注. 参数根轨迹表明：标准根轨迹的 10 条规则同样适用于非增益参数 α 的根轨迹，只需将特征方程改写为 $1 + \alpha \cdot G_{eq}(s) = 0$ 的形式即可，其中 α 为待分析参数。

§4.5 基于根轨迹的系统性能分析

4.5.1 主导极点概念

主导极点

闭环系统中，那些距离虚轴最近且附近无闭环零点的极点称为主导极点。高阶系统的动态响应主要由其主导极点决定——远离虚轴的极点对应的暂态分量衰减极为迅速，对系统响应的贡献可忽略。

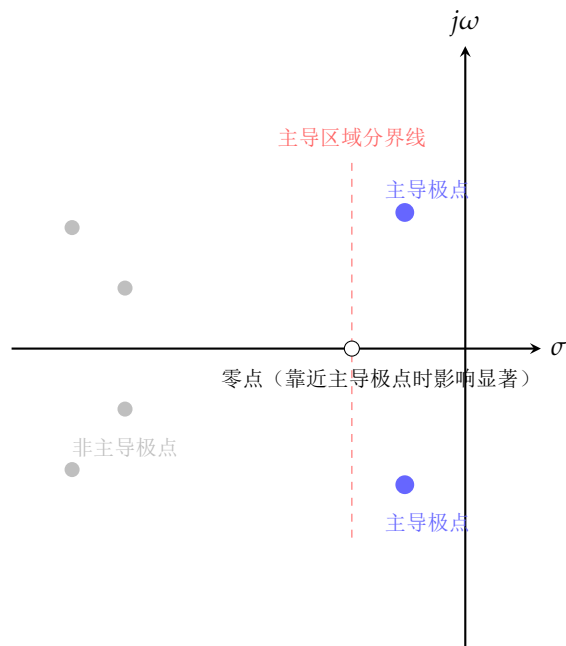


图 4.4: 主导极点概念示意

若主导极点为 $s_{1,2} = -\zeta\omega_n \pm j\omega_n\sqrt{1-\zeta^2}$ ，则系统近似为二阶系统，其时域性能可由前面章节的公式直接估算。主导极点判别准则：距虚轴的距离至少为其他极点的 $1/5$ （即非主导极点实部至少为主导极点实部的 5 倍）。

4.5.2 附加极点对系统性能的影响

附加极点使系统响应变慢。在原开环系统中加入一个实极点 $-p$ ($p > 0$)：根轨迹向右弯曲，导致主导极点的阻尼比减小（振荡加剧）、实部绝对值减小（响应变慢）。从物理角度看，附加极点引入了额外的“惯性”，使系统响应迟钝。

附加闭环极点使稳定性裕度减小。这一点在根轨迹上表现为主导极点向虚轴方向偏移，临界增益降低。

4.5.3 附加零点对系统性能的影响

附加零点使系统响应加快。加入一个实零点 $-z$ ：根轨迹向左弯曲，主导极点向远离虚轴方向运动，阻尼比增大。但当零点位置过于靠右时可能使系统不稳定。

注 附加零极点对系统动态品质的影响可总结为：左极点使响应变慢，左零点使响应加快。这是控制系统中一种深刻的对偶关系，反映了传递函数零极点配置与系统动态之间的内在联系。

§4.6

基于根轨迹的补偿设计

根轨迹法不仅可用于分析，更可用于设计与综合。基本思路是：若系统的根轨迹不满足性能指标，则在系统中引入附加零极点（即串联补偿器），改变根轨迹的形状，使其通过期望的闭环极点位置。

4.6.1 超前补偿（相位超前网络）

超前补偿器的传递函数为：

$$G_c(s) = K_c \frac{s + z_c}{s + p_c}, \quad |z_c| < |p_c|$$

零点靠近虚轴（或位于期望区域），极点远离虚轴。其效果是“吸引”根轨迹向左移动，增加系统阻尼和带宽。

4.6.2 滞后补偿（相位滞后网络）

滞后补偿器的传递函数为：

$$G_c(s) = K_c \frac{s + z_c}{s + p_c}, \quad |p_c| < |z_c|$$

极点和零点靠得很近且紧邻原点，用于提高稳态精度（增大低频增益）而不显著改变暂态响应。

4.6.3 设计流程

基于根轨迹的补偿器设计步骤

1. 根据时域性能指标（ $\sigma\%$ 、 t_s ）确定期望的闭环主导极点位置 $s_{1,2} = -\zeta\omega_n \pm j\omega_n\sqrt{1-\zeta^2}$ 。
2. 绘制原系统根轨迹，检查是否经过期望极点区域。若是，则仅需调节增益 K ；若否，则需要加入补偿器。
3. 选择补偿器类型（超前/滞后/超前-滞后），确定零极点位置。
4. 由相角条件和幅值条件计算补偿器参数和系统增益。
5. 验证补偿后系统的时域响应，必要时迭代调整。

Python 示例：补偿器设计与根轨迹验证

```

1 import numpy as np
2 import control as ct
3
4 # 被控对象: Gp(s) = 1 / [s(s+1)]
5 num_p = [1]
6 den_p = [1, 1, 0]
7 Gp = ct.tf(num_p, den_p)
8
9 # 期望性能: zeta >= 0.5, ts(2%) <= 4s
10 # -> zeta*wn >= 1 -> wn >= 2
11 # 期望主导极点: s_d = -zeta*wn +/- j*wn*sqrt(1-zeta^2)
12 zeta_d = 0.5
13 wn_d = 2.0
14 wd = wn_d * np.sqrt(1 - zeta_d**2)
15 s_d = complex(-zeta_d * wn_d, wd)
16 print(f"期望闭环主导极点: s_d = {s_d:.4f}")
17
18 # 求原系统在期望极点处的相位缺额
19 val = ct.evalfr(Gp, s_d)
20 phase_deficiency = np.pi - np.angle(val)

```



```
21 print(f"相位缺额: {np.degrees(phase_deficiency):.2f} deg")
22
23 # 设计超前补偿器 (此处仅示意, 实际需完整设计算法)
24 # 补偿器形式:  $G_c(s) = K_c * (s+z_c)/(s+p_c)$ 
25 # ... (详细设计代码见后续频域章节)
26
27 # 验证补偿后系统
28 #  $G_{ol} = G_c * G_p$ 
29 # rlocus( $G_{ol}$ ) 检查是否通过期望极点
```

本章小结

根轨迹法是经典控制理论中最重要的图解分析方法之一。它将闭环极点在复平面上的运动轨迹可视化, 为系统分析与综合提供了直观、有力的工具。本章从根轨迹的基本概念出发, 详细阐述了相角条件和幅值条件这两个基础等式, 系统整理了 Evans 的 10 条绘制规则, 并通过二阶与三阶系统的典型实例演示了规则的应用。主导极点概念为高阶系统的降阶近似分析提供了理论依据。最后, 根轨迹补偿设计作为分析与综合的连接桥梁, 展示了根轨迹法在实际控制工程中的价值。

频域分析法是控制理论中最重要的分析方法之一。它通过研究系统在不同频率正弦输入下的稳态响应，揭示系统的动态特性。频域法不仅能用于稳定性分析，还可直接用于系统设计和校正，是经典控制理论的核心工具。

§5.1 频率特性的基本概念

5.1.1 频率特性的定义

设线性定常系统的传递函数为 $G(s)$ ，当输入为正弦信号 $r(t) = A_r \sin(\omega t)$ 时，系统稳态输出为同频率的正弦信号：

$$y_{ss}(t) = A_r |G(j\omega)| \sin(\omega t + \angle G(j\omega))$$

频率特性

传递函数 $G(s)$ 中令 $s = j\omega$ ，得到的复变函数 $G(j\omega)$ 称为系统的频率特性。

$$G(j\omega) = |G(j\omega)| e^{j\angle G(j\omega)} = A(\omega) e^{j\varphi(\omega)}$$

其中 $A(\omega) = |G(j\omega)|$ 为幅频特性， $\varphi(\omega) = \angle G(j\omega)$ 为相频特性。

频率特性的物理意义

频率特性 $G(j\omega)$ 的模 $|G(j\omega)|$ 等于稳态输出与输入的幅值之比，辐角 $\angle G(j\omega)$ 等于稳态输出与输入的相位差。

证明. 设系统输入 $r(t) = A_r \sin(\omega t)$ ，其 Laplace 变换为

$$R(s) = \frac{A_r \omega}{s^2 + \omega^2}$$

输出 $Y(s) = G(s)R(s)$ ，展开为部分分式后取 Laplace 反变换。当 $t \rightarrow \infty$ 时，暂态分量趋于零，稳态分量为

$$y_{ss}(t) = A_r |G(j\omega)| \sin(\omega t + \angle G(j\omega))$$

证毕。 □

5.1.2 频率特性的几何表示

频率特性可以在复平面上用向量表示。对每一频率 ω ， $G(j\omega)$ 对应复平面上的一个点。当 ω 从 0 变化到 ∞ 时，这些点的轨迹构成幅相特性曲线（Nyquist 图）。

频率特性的三种表示

- 幅相频率特性 (Nyquist 图): 以 ω 为参变量, 在复平面上绘制 $\Re[G(j\omega)]$ 和 $\Im[G(j\omega)]$ 的关系。
- 对数频率特性 (Bode 图): 包含对数幅频特性 $L(\omega) = 20 \lg |G(j\omega)|$ (单位: dB) 和对数相频特性 $\varphi(\omega)$ (单位: 度或弧度), 横坐标均为 ω 的对数分度。
- 对数幅相特性 (Nichols 图): 以 $\varphi(\omega)$ 为横坐标, $L(\omega)$ 为纵坐标, ω 为参变量。

【例题 5.1】考虑一阶惯性环节 $G(s) = \frac{1}{Ts+1}$, 其频率特性为

$$G(j\omega) = \frac{1}{1+j\omega T} = \frac{1}{\sqrt{1+\omega^2 T^2}} e^{-j \arctan(\omega T)}$$

幅频: $A(\omega) = 1/\sqrt{1+\omega^2 T^2}$, 相频: $\varphi(\omega) = -\arctan(\omega T)$ 。当 $\omega = 0$ 时, $A = 1$, $\varphi = 0^\circ$; 当 $\omega = 1/T$ (截止频率) 时, $A = 1/\sqrt{2} \approx 0.707$, $\varphi = -45^\circ$; 当 $\omega \rightarrow \infty$ 时, $A \rightarrow 0$, $\varphi \rightarrow -90^\circ$ 。

§5.2

典型环节的频率特性

自动控制系统通常由若干典型环节构成。掌握各典型环节的频率特性是频域分析的基础。

5.2.1 比例环节

传递函数 $G(s) = K$, 频率特性 $G(j\omega) = K$ 。

幅频: $A(\omega) = K$, 对数幅频: $L(\omega) = 20 \lg K$ 。相频: $\varphi(\omega) = 0^\circ$ 。

Nyquist 图为实轴上的一点 $(K, 0)$ 。Bode 图中幅频为平行于横轴的直线, 相频为零度线。

5.2.2 积分环节

传递函数 $G(s) = 1/s$, 频率特性 $G(j\omega) = 1/(j\omega) = -j/\omega$ 。

幅频: $A(\omega) = 1/\omega$, $L(\omega) = -20 \lg \omega$ (斜率为 -20 dB/dec 的直线)。相频: $\varphi(\omega) = -90^\circ$ 。

Nyquist 图为整个负虚轴。

5.2.3 微分环节

传递函数 $G(s) = s$, 频率特性 $G(j\omega) = j\omega$ 。

$L(\omega) = 20 \lg \omega$ ($+20 \text{ dB/dec}$), $\varphi(\omega) = +90^\circ$ 。

Nyquist 图为整个正虚轴。

5.2.4 惯性环节

传递函数 $G(s) = 1/(Ts+1)$, 频率特性

$$G(j\omega) = \frac{1}{1+j\omega T} = \frac{1}{1+\omega^2 T^2} - j \frac{\omega T}{1+\omega^2 T^2}$$

$$L(\omega) = -20 \lg \sqrt{1+\omega^2 T^2}, \quad \varphi(\omega) = -\arctan(\omega T) \quad (5.1)$$

绘制 Bode 图的渐近线:

- 低频段 ($\omega \ll 1/T$): $L(\omega) \approx 0 \text{ dB}$, $\varphi(\omega) \approx 0^\circ$
- 高频段 ($\omega \gg 1/T$): $L(\omega) \approx -20 \lg(\omega T)$, 斜率 -20 dB/dec , $\varphi(\omega) \approx -90^\circ$
- 交接频率 $\omega = 1/T$ 处, 精确值为 $L = -3 \text{ dB}$, $\varphi = -45^\circ$

Nyquist 图为以 $(1/2, 0)$ 为圆心、半径为 $1/2$ 的半圆。

5.2.5 振荡环节

传递函数 $G(s) = \frac{\omega_n^2}{s^2 + 2\zeta\omega_n s + \omega_n^2}$, 频率特性

$$G(j\omega) = \frac{\omega_n^2}{(j\omega)^2 + 2\zeta\omega_n(j\omega) + \omega_n^2} = \frac{1}{(1 - \frac{\omega^2}{\omega_n^2}) + j2\zeta\frac{\omega}{\omega_n}}$$

$$A(\omega) = \frac{1}{\sqrt{(1 - \frac{\omega^2}{\omega_n^2})^2 + (2\zeta\frac{\omega}{\omega_n})^2}}, \quad \varphi(\omega) = -\arctan \frac{2\zeta\frac{\omega}{\omega_n}}{1 - \frac{\omega^2}{\omega_n^2}} \quad (5.2)$$

当阻尼比 $\zeta < \sqrt{2}/2 \approx 0.707$ 时, $A(\omega)$ 出现峰值:

$$M_r = \frac{1}{2\zeta\sqrt{1 - \zeta^2}}, \quad \omega_r = \omega_n\sqrt{1 - 2\zeta^2}$$

Bode 图渐近线:

- 低频段 ($\omega \ll \omega_n$): $L(\omega) \approx 0 \text{ dB}$
- 高频段 ($\omega \gg \omega_n$): $L(\omega) \approx -40 \lg(\omega/\omega_n)$, 斜率 -40 dB/dec
- 交接频率 ω_n 附近可能出现谐振峰

5.2.6 滞后环节

传递函数 $G(s) = e^{-\tau s}$, 频率特性 $G(j\omega) = e^{-j\omega\tau}$ 。

$A(\omega) = 1$, $L(\omega) = 0 \text{ dB}$, $\varphi(\omega) = -\omega\tau$ (随频率线性下降)。

滞后环节不改变幅频特性, 仅引入相位滞后, 对系统稳定性有不利影响。

5.2.7 一阶微分环节

传递函数 $G(s) = Ts + 1$, 频率特性 $G(j\omega) = 1 + j\omega T$ 。

$L(\omega) = 20 \lg \sqrt{1 + \omega^2 T^2}$, 高频斜率 $+20 \text{ dB/dec}$ 。 $\varphi(\omega) = \arctan(\omega T)$, 高频趋近 $+90^\circ$ 。

频域特性叠加原理

设开环系统由若干典型环节串联而成:

$$G(s) = G_1(s)G_2(s) \cdots G_n(s)$$

则开环对数幅频和相频特性分别为各环节对数幅频和相频特性的代数和:

$$L(\omega) = \sum_{i=1}^n L_i(\omega), \quad \varphi(\omega) = \sum_{i=1}^n \varphi_i(\omega)$$

此叠加性质是 Bode 图在系统分析和设计中被广泛应用的重要原因。

§ 5.3 Bode 图

5.3.1 对数频率特性的坐标

Bode 图由两幅图组成：

- 对数幅频特性图：纵坐标 $L(\omega) = 20 \lg |G(j\omega)|$ （单位：dB），横坐标按 $\lg \omega$ 分度，标注 ω 值。
- 对数相频特性图：纵坐标为 $\varphi(\omega)$ （单位：度或弧度），横坐标同样为 $\lg \omega$ 。

频率轴采用对数分度的优点：扩展了低频段，压缩了高频段，能在同一图上表示很宽的频率范围。十倍频程（decade）表示频率增加 10 倍的区间，二倍频程（octave）表示频率增加 1 倍的区间。

Bode 图渐近线绘制步骤

1. 将开环传递函数化为典型环节乘积的标准形式，确定各交接频率。
2. 确定低频段渐近线：在 $\omega < \omega_{\min}$ （最小交接频率）处，根据积分环节个数 ν ，低频渐近线斜率为 -20ν dB/dec，过点 $(1, 20 \lg K)$ 。
3. 在各交接频率处改变斜率：惯性环节加 -20 dB/dec，一阶微分加 $+20$ dB/dec，振荡环节加 -40 dB/dec。
4. 在各交接频率处对相频进行修正。
5. 必要时在交接频率附近对渐近线进行修正（惯性/微分环节修正 ± 3 dB，振荡环节按 ζ 查表修正）。

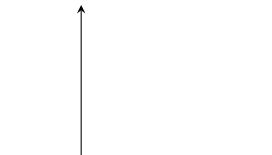
5.3.2 系统辨识中的 Bode 图应用

Bode 图可以用于系统辨识，即通过实验测量的频率响应数据来确定系统的传递函数。基本步骤为：

1. 对系统施加不同频率的正弦信号，测量稳态输出的幅值比和相位差。
2. 在对数坐标纸上绘制实验 Bode 图。
3. 用斜率为 $\pm 20n$ dB/dec (n 为整数) 的渐近线逼近实验幅频曲线。
4. 根据渐近线的交接频率和斜率变化确定各典型环节参数。
5. 写出系统的近似传递函数。

注. 由最小相位系统的幅频特性可以唯一确定其相频特性和传递函数（Bode 定理），因此实际系统辨识时往往只测量幅频特性即可。

$L(\omega)$ (dB)



§5.4 Nyquist 图与稳定性判据

5.4.1 Nyquist 图的基本概念

Nyquist 图（幅相频率特性图）是在复平面上以 ω 从 0 到 ∞ 为参变量绘制的 $G(j\omega)$ 轨迹，也称极坐标图。

对于最小相位系统，Nyquist 图的起始点 ($\omega = 0$) 和终止点 ($\omega \rightarrow \infty$) 分别反映了系统在低频和高频段的特性。

围线映射

设 $F(s)$ 是复变量 s 的有理函数， Γ_s 是 s 平面上一条不经过 $F(s)$ 极点的闭合围线。当 s 沿 Γ_s 顺时针行走一周时， $F(s)$ 在 F 平面上也形成一条闭合围线 Γ_F 。 Γ_F 包围原点的圈数等于 Γ_s 内 $F(s)$ 的零点减去极点数。

5.4.2 Cauchy 定理与 Nyquist 稳定判据

设单位反馈系统的开环传递函数为 $G(s)$ ，闭环特征方程为 $1 + G(s) = 0$ 。令

$$F(s) = 1 + G(s)$$

则 $F(s)$ 的零点即为闭环极点， $F(s)$ 的极点即为开环极点。

取 Nyquist 围线（包围整个右半 s 平面的大半圆）， $F(s)$ 的围线包围原点的圈数为：

$$N_F = Z - P$$

其中 Z 为右半平面闭环极点数， P 为右半平面开环极点数。

由于 $F(s) = 1 + G(s)$ 的围线包围原点等价于 $G(s)$ 的围线包围 $(-1, j0)$ 点，故：

Nyquist 稳定判据（基本形式）

设开环系统有 P 个右半平面极点，开环频率特性 $G(j\omega)$ 的 Nyquist 图逆时针包围 $(-1, j0)$ 点的圈数记为 N （顺时针为负），则闭环系统稳定的充要条件是

$$Z = P - N = 0$$

Nyquist 稳定判据（简化形式）

若开环系统稳定 ($P = 0$)，则闭环系统稳定的充要条件是 $G(j\omega)$ 的 Nyquist 图不包围 $(-1, j0)$ 点。

Nyquist 稳定判据（穿越形式）

设开环系统稳定 ($P = 0$)，Nyquist 图与负实轴的交点均位于 $(-1, j0)$ 点的左侧。设正穿越（从上向下）次数为 N^+ ，负穿越（从下向上）次数为 N^- ，则闭环稳定条件为

$$N^+ - N^- = 0$$

当开环不稳定时，条件为 $N^+ - N^- = P/2$ 。

5.4.3 Nyquist 图的绘制

绘制 Nyquist 图需注意：

- 当 $\omega \rightarrow 0$ 时，对于含 ν 个积分环节的系统，Nyquist 图从无穷远处出发。在 s 平面上以无穷小半径绕过原点处的极点，对应 $G(j\omega)$ 图上以 $\nu \times 90^\circ$ 顺时针的大圆弧补充。
- 利用对称性： $G(-j\omega) = \overline{G(j\omega)}$ ，可只绘制 $\omega \in [0, \infty)$ 然后镜像得到 $\omega \in (-\infty, 0]$ 。

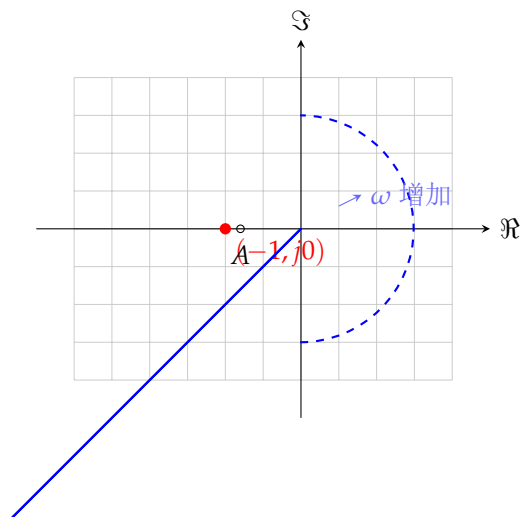


图 5.2: Nyquist 图与 $(-1, j0)$ 关键点

【例题 5.2】 设单位反馈系统开环传递函数为

$$G(s) = \frac{K}{s(s+1)(s+2)}$$

开环极点均为负实数， $P = 0$ 。绘制 $G(j\omega)$ 的 Nyquist 图，当 K 足够大时曲线穿越负实轴于 $(-1, j0)$ 左侧，包围该点一次 ($N = -1$)，闭环不稳定。减小 K 可使 Nyquist 图收缩，不包围 $(-1, j0)$ ，系统稳定。

§ 5.5 稳定裕度

5.5.1 幅值裕度

幅值裕度

设 ω_g 为相位穿越频率，满足 $\angle G(j\omega_g) = -180^\circ$ 。定义幅值裕度

$$G_m = \frac{1}{|G(j\omega_g)|}$$

用分贝表示为

$$G_m(\text{dB}) = -20 \lg |G(j\omega_g)|$$

幅值裕度的物理意义：系统达到临界稳定前，开环增益还能增大的倍数。 $G_m > 1$ （即分贝值为正）时，系统具有正的幅值裕度。

5.5.2 相位裕度

相位裕度

设 ω_c 为幅值穿越频率（也称截止频率），满足 $|G(j\omega_c)| = 1$ （即 0 dB）。定义相位裕度

$$\gamma = 180^\circ + \angle G(j\omega_c)$$

相位裕度的物理意义：系统达到临界稳定前，开环相位还能增加的滞后量。 $\gamma > 0$ 时系统具有正的相位裕度。

5.5.3 从 Bode 图和 Nyquist 图求取稳定裕度

Bode 图上：

- ω_c 在 $L(\omega) = 0$ dB 处确定，向上读取 $\varphi(\omega_c)$ ，相位裕度 $\gamma = 180^\circ + \varphi(\omega_c)$
- ω_g 在 $\varphi(\omega) = -180^\circ$ 处确定，向上读取 $L(\omega_g)$ ，幅值裕度 $G_m = -L(\omega_g)$ dB

Nyquist 图上：

- $G(j\omega)$ 与单位圆的交点对应的频率为 ω_c ，该点与负实轴夹角即为 γ
- $G(j\omega)$ 与负实轴交点对应的频率为 ω_g ，该点到原点的距离的倒数即为 G_m

工程推荐的稳定裕度

为获得满意的控制性能，工程上通常要求：

- 相位裕度 $\gamma = 30^\circ \sim 60^\circ$
- 幅值裕度 $G_m \geq 6$ dB（即 $G_m \geq 2$ ）

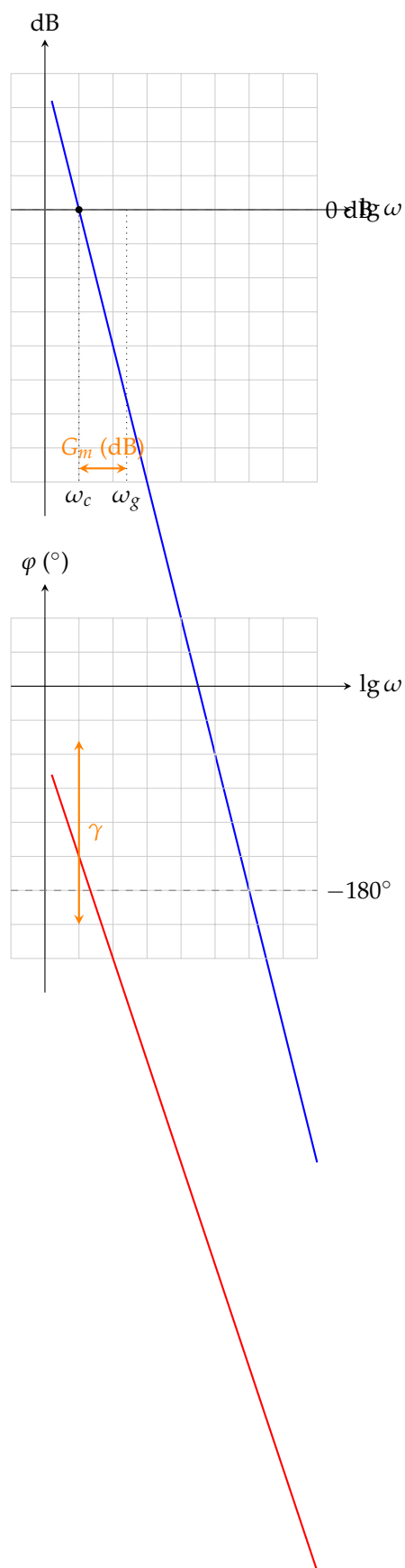


图 5.3: 稳定裕度在 Bode 图上的表示

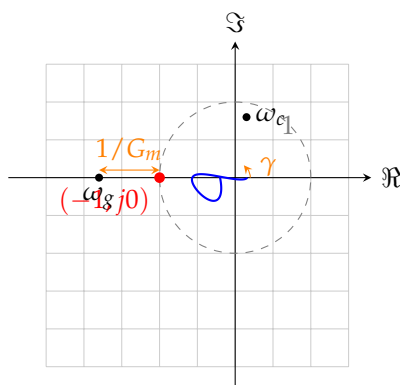


图 5.4: 稳定裕度在 Nyquist 图上的表示

§ 5.6 闭环频率特性

5.6.1 闭环频域指标

对于单位反馈系统，闭环传递函数为

$$\Phi(s) = \frac{G(s)}{1 + G(s)}$$

闭环频率特性 $\Phi(j\omega)$ 的特征量为：

闭环频域指标

1. 谐振峰值 M_p ：闭环幅频特性的最大值：

$$M_p = \max_{\omega} |\Phi(j\omega)|$$

2. 谐振频率 ω_r ：达到 M_p 时所对应的频率。
3. 带宽频率 ω_b ：闭环幅频特性降到 $0.707|\Phi(j0)|$ （即 -3 dB）时的频率。
4. 剪切频率 ω_c ： $|\Phi(j\omega_c)| = 1$ 时的频率。

带宽 ω_b 是表征系统响应速度的重要指标：带宽越大，系统响应越快，但抗高频噪声能力越弱。

5.6.2 Nichols 图

Nichols 图是以开环相频 $\angle G(j\omega)$ 为横坐标、开环对数幅频 $20\lg |G(j\omega)|$ 为纵坐标、 ω 为参变量的图。在 Nichols 图上附加等 **M** 圆（等闭环幅值线）和等 **N** 圆（等闭环相位线）后，可以直接从开环特性读出闭环特性。

$$\left| \frac{G(j\omega)}{1 + G(j\omega)} \right| = M \Rightarrow \left| \frac{G}{1 + G} \right|^2 = \frac{x^2 + y^2}{(1 + x)^2 + y^2} = M^2 \quad (5.3)$$

其中 $G(j\omega) = x + jy$ 。整理得到等 **M** 圆方程：

$$\left(x + \frac{M^2}{M^2 - 1} \right)^2 + y^2 = \left(\frac{M}{M^2 - 1} \right)^2$$

注. Nichols 图是频域设计的重要工具。设计者可以在 Nichols 图上直观地看到开环频率特性形状与闭环性能指标 (M_p 、 ω_b) 之间的关系，从而指导控制器的设计。

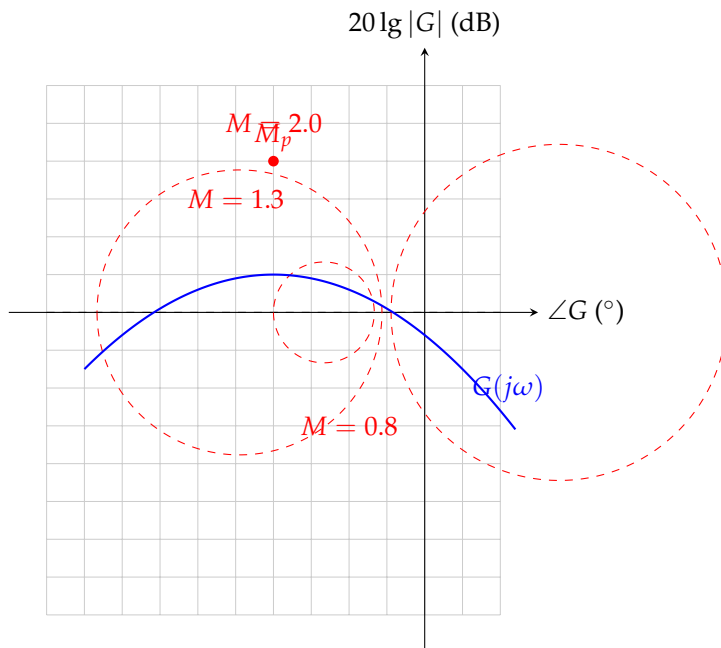


图 5.5: Nichols 图与等 M 圆

§ 5.7

频域特性与时域指标的关系

对于典型的二阶系统，频域指标与时域指标之间存在简洁的解析关系。

5.7.1 二阶系统的频域-时域关系

设二阶系统 $\Phi(s) = \frac{\omega_n^2}{s^2 + 2\zeta\omega_n s + \omega_n^2}$, $0 < \zeta < 1$ 。

$$M_p = \frac{1}{2\zeta\sqrt{1-\zeta^2}}, \quad \omega_r = \omega_n\sqrt{1-2\zeta^2} \quad (\zeta \leq 0.707) \quad (5.4)$$

$$\omega_b = \omega_n\sqrt{1-2\zeta^2 + \sqrt{2-4\zeta^2 + 4\zeta^4}} \quad (5.5)$$

$$\omega_c \approx \omega_n\sqrt{\sqrt{1+4\zeta^4} - 2\zeta^2}$$

时域指标（超调量 $\sigma\%$ ，调节时间 t_s ）与频域指标的关系：

频域-时域关系公式

对于二阶系统，频域指标与时域指标满足近似关系：

$$\sigma\% \approx 0.16 + 0.4(M_p - 1), \quad 1 \leq M_p \leq 1.8 \quad (5.6)$$

$$t_s \approx \frac{k\pi}{\omega_c}, \quad k = 2 \sim 3 \quad (5.7)$$

更精确的关系为：

$$t_s \omega_c \approx \frac{4}{\zeta} \sqrt{\sqrt{1 + 4\zeta^4} - 2\zeta^2}$$

注. 对于高阶系统，可以用主导极点概念近似为二阶系统进行分析。通常要求主导极点与其他极点实部之比大于 5，且不存在靠近虚轴的零点。

5.7.2 由频域指标设计系统

基于频域指标的设计方法可归纳为：

1. 根据稳态精度要求确定开环增益 K 。
2. 根据期望的 ω_c 和 γ 设计中频段形状。
3. 高频段尽可能陡峭以抑制噪声。
4. 低频段斜率反映无差度，斜率为 -20ν dB/dec。

【例题 5.3】 某单位反馈系统要求：速度误差系数 $K_v = 20$ ，相位裕度 $\gamma \geq 50^\circ$ ，截止频率 $\omega_c \geq 2$ rad/s。试确定满足要求的开环传递函数形式。

解：由 $K_v = 20$ 可知系统至少含一个积分环节，且 $K = 20$ 。设

$$G(s) = \frac{20}{s} \cdot \frac{1}{Ts + 1}$$

在 $\omega = 2$ 处， $|G(j2)| = 20/(2\sqrt{1+4T^2})$ ，令其等于 1 解得 T 。由 $\gamma = 90^\circ - \arctan(2T) \geq 50^\circ$ 可得 $T \leq 0.42$ 。取 $T = 0.4$ ，则 $G(s) = 20/[s(0.4s + 1)]$ 满足要求。

§5.8

Python 频域分析仿真

以下 Python 代码演示了如何利用 control 库进行频域分析，包括绘制 Bode 图、Nyquist 图和计算稳定裕度。

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 try:
4     import control as ct
5 except ImportError:
6     print("请安装 control 库: pip install control")
7     exit(1)
8
9 # 定义开环传递函数 G(s) = 20 / [s(s+1)(s+2)]
10 num = [20]
```

```

11 den = [1, 3, 2, 0] #  $s(s+1)(s+2) = s^3 + 3s^2 + 2s$ 
12 G = ct.TransferFunction(num, den)
13
14 # 计算稳定裕度
15 gm, pm, wg, wp = ct.margin(G)
16 print(f"幅值裕度 Gm = {gm:.3f} ({20*np.log10(gm):.2f} dB)")
17 print(f"相位裕度 Pm = {pm:.2f} 度")
18 print(f"相位穿越频率 wg = {wg:.3f} rad/s")
19 print(f"幅值穿越频率 wc = {wp:.3f} rad/s")
20
21 # 绘制 Bode 图
22 plt.figure(figsize=(10, 8))
23 ct.bode_plot(G, dB=True, Hz=False, grid=True)
24 plt.suptitle("Bode 图", fontsize=14)
25 plt.tight_layout()
26 plt.show()
27
28 # 绘制 Nyquist 图
29 plt.figure(figsize=(8, 8))
30 ct.nyquist_plot(G, omega=np.logspace(-2, 2, 500))
31 plt.title("Nyquist 图")
32 plt.grid(True)
33 plt.show()
34
35 # 闭环频率特性
36 T = ct.feedback(G, 1)
37 plt.figure(figsize=(8, 4))
38 mag, phase, omega = ct.bode(T, dB=True, Hz=False, plot=False)
39 plt.subplot(1, 2, 1)
40 plt.semilogx(omega, 20*np.log10(mag))
41 plt.grid(True)
42 plt.xlabel("频率 (rad/s)")
43 plt.ylabel("幅值 (dB)")
44 plt.title("闭环幅频特性")
45
46 # 求取闭环指标
47 mag_linear = mag
48 Mp = np.max(mag_linear)
49 omega_r = omega[np.argmax(mag_linear)]
50 # 带宽: 幅值降到 0.707 倍零频值
51 dc_gain = mag_linear[0]
52 idx_bw = np.where(mag_linear < 0.707 * dc_gain)[0]
53 omega_b = omega[idx_bw[0]] if len(idx_bw) > 0 else omega[-1]
54 print(f"谐振峰值 Mp = {Mp:.3f}")
55 print(f"谐振频率 wr = {omega_r:.3f} rad/s")
56 print(f"带宽频率 wb = {omega_b:.3f} rad/s")
57
58 plt.show()

```

§5.9 本章小结

本章系统介绍了频域分析的基本理论和方法。频率特性是联系系统传递函数与动态性能的桥梁。Bode 图因其叠加性和渐近线绘制方法而成为工程上最常用的工具；Nyquist 稳定判据从复变函数论出发，给出了严格的闭环稳定性条件。稳定裕度概念将稳定性从“是/否”的二值判断扩展到“好/坏”的定量度量，为系统校正提供了依据。闭环频域指标 M_p 、 ω_r 、 ω_b 与时域指标之间存在定量关系，使得频域设计结果可以直接预测时域响应品质。

在下一章中，我们将利用频域分析工具进行系统校正设计，通过引入补偿器改善系统的稳态和动态性能。

系统校正与 PID 控制

系统校正 (Compensation) 是指在原有控制系统的基础上, 通过引入附加装置 (校正器或补偿器) 来改善系统的性能, 使其满足给定的性能指标要求。PID 控制作为最经典、最广泛应用的校正方法, 在工业过程控制中占据主导地位。

§6.1
校正的基本概念

6.1.1 校正的分类

按校正装置在系统中的连接方式, 可分为:

校正方式的分类

- 串联校正: 校正器 $G_c(s)$ 与固有部分 $G_0(s)$ 串联。结构简单, 是最常用的校正方式。
- 反馈校正: 校正器 $H_c(s)$ 包围部分固有环节构成局部反馈回路。可抑制参数变化和干扰的影响。
- 前馈校正: 校正器 $G_f(s)$ 根据参考输入或干扰直接产生补偿信号, 不改变系统稳定性。
- 复合校正: 同时使用上述两种以上的校正方式。

6.1.2 频率法校正的目标

从频域角度, 对系统开环对数频率特性的期望形状可概括为“低、中、高”三段:

开环对数幅频特性的三段期望

- 低频段 ($\omega < \omega_1$): 反映稳态精度。应有较大的增益和足够的斜率 (-20ν dB/dec), 以保证稳态误差小。
- 中频段 ($\omega_1 \sim \omega_2$, 含截止频率 ω_c): 决定稳定性和动态品质。应以 -20 dB/dec 斜率穿越 0 dB 线, 且中频段应有足够宽度 ($h = \omega_2/\omega_1$ 称为中频宽)。
- 高频段 ($\omega > \omega_2$): 影响抗高频噪声能力。斜率应尽可能陡 (如 -40 或 -60 dB/dec)。

中频宽与相位裕度的关系

对于典型的中频段形状 $(-40 \rightarrow -20 \rightarrow -40)$ ，设中频宽 $h = \omega_2/\omega_1$ ，相位裕度近似为

$$\gamma \approx \arcsin \frac{h-1}{h+1}$$

h 越大， γ 越大，系统相对稳定性越好。

§ 6.2
超前校正

6.2.1 超前校正的传递函数

超前校正器

超前校正器 (Lead Compensator) 的传递函数为

$$G_c(s) = K_c \frac{\alpha Ts + 1}{Ts + 1}, \quad \alpha > 1$$

其中 α 为超前因子， T 为时间常数， K_c 为增益。

其零极点分布为：零点 $z = -1/(\alpha T)$ ，极点 $p = -1/T$ 。由于 $\alpha > 1$ ，零点比极点更靠近虚轴， $|z| < |p|$ 。

6.2.2 超前校正的频域特性

频率特性：

$$G_c(j\omega) = K_c \frac{1 + j\alpha\omega T}{1 + j\omega T}$$

对数幅频：低频段 $L = 20 \lg K_c$ ，经过 $\omega = 1/(\alpha T)$ （零点交接频率）后斜率变为 $+20 \text{ dB/dec}$ ，再经过 $\omega = 1/T$ （极点交接频率）后恢复水平线。最大相角出现在 ω_m 处。

$$\varphi_c(\omega) = \arctan(\alpha\omega T) - \arctan(\omega T) \quad (6.1)$$

令 $d\varphi_c/d\omega = 0$ 得：

$$\omega_m = \frac{1}{T\sqrt{\alpha}}, \quad \varphi_{\max} = \arcsin \frac{\alpha - 1}{\alpha + 1} \quad (6.2)$$

在 ω_m 处的对数幅值为：

$$L_c(\omega_m) = 20 \lg K_c + 10 \lg \alpha$$

注. 超前校正提供正相角，可提高系统的相位裕度；同时在中频段引入 $+20 \text{ dB/dec}$ 的幅频斜率变化，有助于提升截止频率 ω_c ，加快系统响应速度。

6.2.3 超前校正的设计步骤

超前校正的频率法设计步骤

1. 根据稳态误差要求确定开环增益 K 。
2. 绘制未校正系统的 Bode 图，求取当前的相位裕度 γ_0 和截止频率 ω_{c0} 。
3. 确定需要补充的最大超前相角 $\varphi_m = \gamma^* - \gamma_0 + \Delta$ ，其中 γ^* 为期望相位裕度， $\Delta = 5^\circ \sim 15^\circ$ 为补偿超前校正引入后 ω_c 右移造成的相位损失。

4. 由 φ_m 计算 α ：

$$\alpha = \frac{1 + \sin \varphi_m}{1 - \sin \varphi_m}$$

5. 确定校正后的 ω_c ：在未校正系统的 Bode 图上，找到满足 $L_0(\omega) = -10 \lg \alpha$ 的频率即为 ω_c 。

6. 确定时间常数 T ：

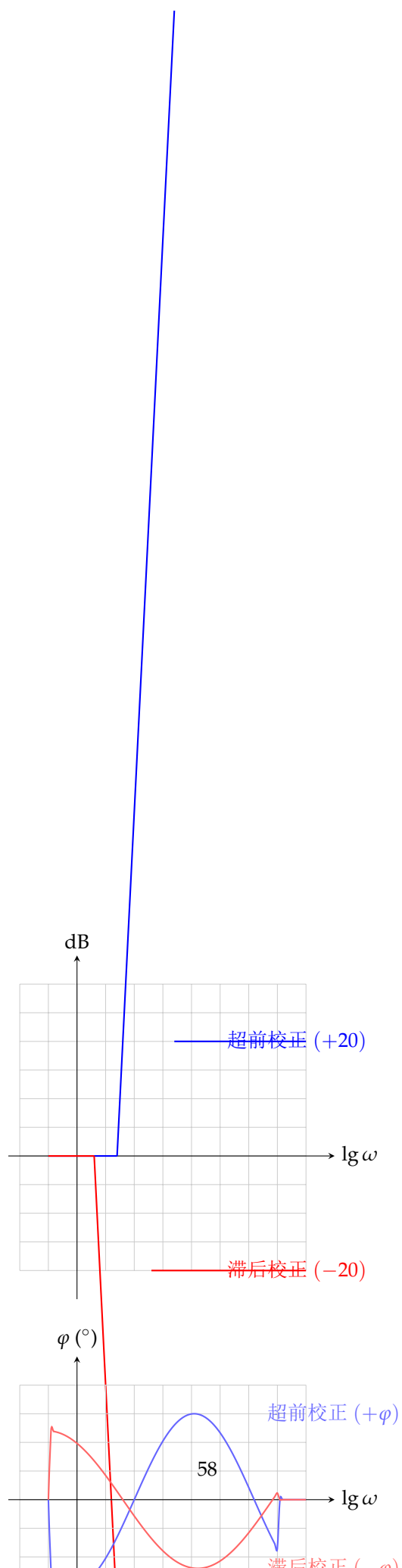
$$T = \frac{1}{\omega_c \sqrt{\alpha}}$$

7. 写出校正器传递函数 $G_c(s) = K_c(\alpha Ts + 1)/(Ts + 1)$ ，其中 $K_c = \alpha$ 或另行调整。

8. 校验校正后系统的各项性能指标。

【例题 6.1】 单位反馈系统固有部分 $G_0(s) = \frac{4}{s(s+2)}$ 。要求：速度误差系数 $K_v \geq 20$ ，相位裕度 $\gamma \geq 50^\circ$ ，截止频率 $\omega_c \geq 4 \text{ rad/s}$ 。设计超前校正器。

解：(1) $K_v = \lim_{s \rightarrow 0} sG(s) = 2K_c$ ，令 $K_c = 10$ 满足 $K_v = 20$ 。(2) 未校正系统 $G(s) = \frac{20}{s(0.5s+1)}$ ，得 $\omega_{c0} \approx 6.3 \text{ rad/s}$ ， $\gamma_0 \approx 18^\circ$ 。(3) 需补充相角 $\varphi_m = 50^\circ - 18^\circ + 10^\circ = 42^\circ$ 。(4) $\alpha \approx 5.0$ 。(5) 找到 $L_0(\omega) = -10 \lg 5 \approx -7 \text{ dB}$ 对应的频率为 $\omega_c \approx 9 \text{ rad/s}$ 。(6) $T = 1/(9\sqrt{5}) \approx 0.05$ 。(7) $G_c(s) = \frac{0.25s+1}{0.05s+1}$ 。校验：校正后 $\gamma \approx 52^\circ$ ，满足要求。



§ 6.3 滞后校正

6.3.1 滞后校正的传递函数

滞后校正器

滞后校正器 (Lag Compensator) 的传递函数为

$$G_c(s) = K_c \frac{Ts + 1}{\beta Ts + 1}, \quad \beta > 1$$

其中 β 为滞后因子。极点 $p = -1/(\beta T)$ 比零点 $z = -1/T$ 更靠近虚轴。

频率特性：

$$G_c(j\omega) = K_c \frac{1 + j\omega T}{1 + j\beta\omega T}$$

$$\varphi_c(\omega) = \arctan(\omega T) - \arctan(\beta\omega T) < 0 \quad (6.3)$$

最大滞后相角出现在 $\omega_m = 1/(T\sqrt{\beta})$ ，值为

$$\varphi_{\max} = \arcsin \frac{1 - \beta}{1 + \beta} \quad (\text{负值})$$

注. 滞后校正不利用其负相角，而是利用其高频衰减特性 ($20\lg\beta$ dB 的幅值衰减)，将截止频率降低到相位裕度较大的区域，从而提高稳定裕度。设计时应将滞后校正的交接频率 $1/T$ 和 $1/(\beta T)$ 安排在远离 ω_c 的低频段，使其负相角不影响中频段的相位裕度。

6.3.2 滞后校正的设计步骤

滞后校正的频率法设计步骤

1. 根据稳态误差确定开环增益 K 。
2. 绘制未校正系统的 Bode 图。
3. 在未校正系统的相频曲线上，找到满足 $\varphi_0(\omega) = -180^\circ + \gamma^* + \Delta$ (其中 $\Delta = 5^\circ \sim 12^\circ$ 为滞后校正引入的相位损失) 的频率，作为校正后的 ω_c 。
4. 确定参数 β : $\beta = |G_0(j\omega_c)|$ (即未校正系统在 ω_c 处的幅值)。
5. 将滞后校正的零点交接频率 $1/T$ 取为 $\omega_c/5 \sim \omega_c/10$ ，以减小负相角影响。
6. 计算 $T = (5 \sim 10)/\omega_c$ 。
7. 写出校正器 $G_c(s) = K_c \frac{Ts+1}{\beta Ts+1}$ 。
8. 校验性能指标。

【例题 6.2】 系统 $G_0(s) = \frac{K}{s(s+1)(0.5s+1)}$ ，要求 $K_v \geq 5$ ， $\gamma \geq 40^\circ$ ， $\omega_c \geq 0.5$ rad/s。

解：取 $K = 5$ 。未校正系统 $\omega_{c0} \approx 2.1$ rad/s， $\gamma_0 \approx -20^\circ$ (不稳定)。采用滞后校正。期望 $\gamma^* = 40^\circ$ ，考虑 $\Delta = 6^\circ$ ，寻找 $\varphi = -180^\circ + 46^\circ = -134^\circ$ 对应的频率 $\omega \approx 0.5$ rad/s，此处 $|G_0(j\omega)| \approx 10$ ，即 $\beta = 10$ 。

取 $1/T = \omega_c/10 = 0.05 \text{ rad/s}$, $T = 20$ 。得

$$G_c(s) = \frac{20s + 1}{200s + 1}$$

校验：校正后 $\gamma \approx 42^\circ$ ，满足要求。

§ 6.4 滞后-超前校正

滞后-超前校正 (Lag-Lead Compensator) 结合了超前校正 (改善动态性能) 和滞后校正 (改善稳态精度) 的优点。

滞后-超前校正器

传递函数的一般形式为

$$G_c(s) = K_c \frac{(\tau_1 s + 1)(\tau_2 s + 1)}{(\alpha \tau_1 s + 1)(\frac{\tau_2}{\beta} s + 1)}, \quad \alpha > 1, \beta > 1$$

其中超前部分的参数为 α 、 τ_1 ，滞后部分的参数为 β 、 τ_2 。

通常先设计超前部分以满足动态性能 (γ 、 ω_c)，再设计滞后部分以满足稳态精度 (K_v 、 K_a 等)。

典型实现电路 (有源 RC 网络)：

- 采用运算放大器实现，超前网络通过输入电容和反馈电阻构成
- 滞后-超前网络通过两组 RC 元件组合实现

【例题 6.3】 某系统的设计规范同时要求大的静态增益和充分的相位裕度。可先采用超前校正达到要求的 ω_c 和 γ ，然后串联滞后校正提高低频增益而不影响中频段特性。

§ 6.5 PID 控制

6.5.1 PID 控制的基本原理

PID 控制器由比例 (P)、积分 (I) 和微分 (D) 三个环节并联组成。

PID 控制律

连续时间 PID 控制律的时域表达式为

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt}$$

传递函数形式：

$$G_c(s) = K_p + \frac{K_i}{s} + K_d s = K_p \left(1 + \frac{1}{T_i s} + T_d s \right)$$

其中 $T_i = K_p/K_i$ 为积分时间常数， $T_d = K_d/K_p$ 为微分时间常数。

6.5.2 各环节的作用

PID 各环节的控制作用

- **比例作用 (P):** 产生与误差成比例的控制量。增大 K_p 可加快响应速度、减小稳态误差，但过大会导致振荡甚至不稳定。
- **积分作用 (I):** 对误差进行累积，用于消除稳态误差。 T_i 越小积分作用越强，但会降低稳定性。
- **微分作用 (D):** 对误差变化率作出响应，具有“预见性”，可改善动态性能、减小超调。 T_d 越大微分作用越强，但会放大高频噪声。

6.5.3 位置式与增量式 PID

在数字控制系统中，PID 控制律需要离散化。

位置式 PID 算法

对连续 PID 控制律进行矩形数值积分近似：

$$u(k) = K_p e(k) + K_i T_s \sum_{j=0}^k e(j) + \frac{K_d}{T_s} [e(k) - e(k-1)]$$

增量式 PID 算法

计算控制量的增量：

$$\Delta u(k) = u(k) - u(k-1) \quad (6.4)$$

$$= K_p [e(k) - e(k-1)] + K_i T_s e(k) \quad (6.5)$$

$$+ \frac{K_d}{T_s} [e(k) - 2e(k-1) + e(k-2)] \quad (6.6)$$

增量式 PID 的优点：计算量小，输出为增量，便于实现无扰动切换；不产生积分饱和。

注. 积分饱和 (Integral Windup) 是实际 PID 控制中的常见问题。当执行器饱和时，积分器持续累积误差，导致系统超调增大、调节时间延长。常用的解决方法包括积分分离法、遇限削弱积分法和反计算抗饱和法。

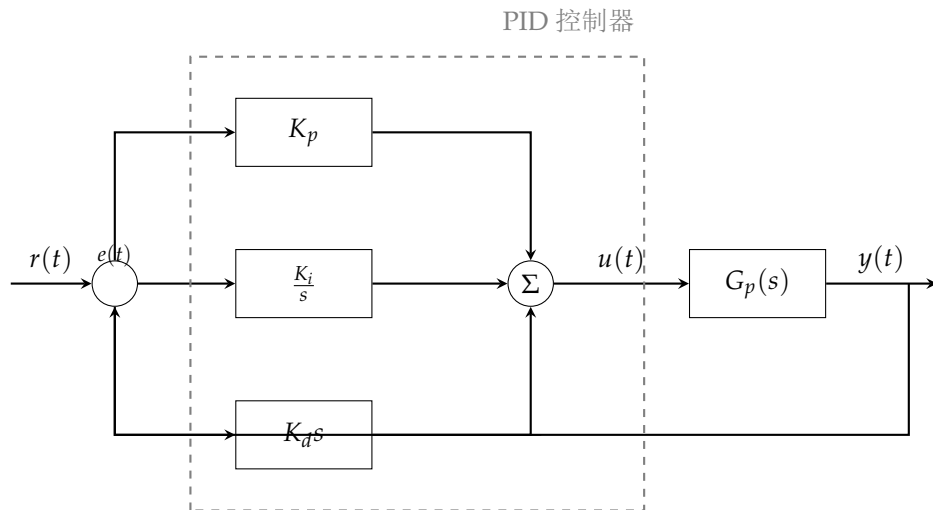


图 6.2: PID 控制器结构框图（P、I、D 三通道并联）

6.5.4 Ziegler-Nichols 整定方法

Ziegler-Nichols 方法是工程上最经典的 PID 参数整定方法，分为两种：

Ziegler-Nichols 第一法（阶跃响应法）

对开环系统施加阶跃输入，记录其响应曲线。在响应曲线的拐点处作切线，得到等效滞后时间 L 和等效时间常数 T 。根据下表确定 PID 参数：

- P 控制： $K_p = T/L$
- PI 控制： $K_p = 0.9 T/L$, $T_i = L/0.3$
- PID 控制： $K_p = 1.2 T/L$, $T_i = 2L$, $T_d = 0.5L$

Ziegler-Nichols 第二法（临界比例度法/临界振荡法）

1. 将控制器设为纯比例控制（ $T_i = \infty$, $T_d = 0$ ）。
2. 逐渐增大 K_p ，直至系统输出出现等幅振荡（临界稳定状态）。
3. 记录此时的临界增益 K_u 和临界振荡周期 T_u 。
4. 根据下表确定 PID 参数：
 - P 控制： $K_p = 0.5K_u$
 - PI 控制： $K_p = 0.45K_u$, $T_i = 0.85 T_u$
 - PID 控制： $K_p = 0.6K_u$, $T_i = 0.5 T_u$, $T_d = 0.125 T_u$

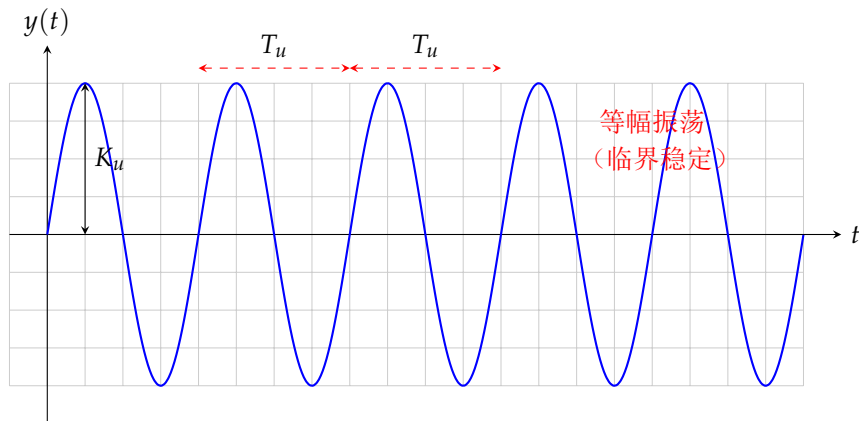


图 6.3: Ziegler-Nichols 临界振荡法示意

6.5.5 PID 参数对系统性能的影响

表 6.1: PID 参数增大时对系统性能的影响

参数	上升时间	超调量	调节时间	稳态误差
K_p	减小	增大	影响小	减小
K_i	减小	增大	增大	消除
K_d	影响小	减小	减小	影响小

§ 6.6 前馈-反馈复合校正

6.6.1 不变性原理

不变性原理（扰动补偿）

设扰动 $d(t)$ 通过传递函数 $G_d(s)$ 影响系统输出。若在扰动作用点之前引入前馈补偿器 $G_f(s)$ ，使得

$$G_f(s)G(s) + G_d(s) = 0$$

即

$$G_f(s) = -\frac{G_d(s)}{G(s)}$$

则系统输出对扰动 $d(t)$ 完全不变（完全补偿）。

实际应用中，由于 $G_f(s)$ 可能需要微分环节或非因果系统，通常只能实现部分不变性（稳态补偿）。最常见的是按扰动的不变性调节，也称前馈控制。

6.6.2 前馈-反馈复合控制

前馈控制单独使用时对模型精度要求很高。将前馈控制与反馈控制结合，可以：

- 前馈补偿主要扰动，快速减小偏差
- 反馈补偿消除剩余误差及其他未建模扰动

复合校正的结构

设前馈控制器的传递函数为 $G_{ff}(s)$ ，反馈控制器为 $G_c(s)$ ，对象为 $G_p(s)$ 。当

$$G_{ff}(s) = \frac{1}{G_p(s)}$$

时，系统对参考输入的跟踪可实现完全不变性。

§6.7 Smith 预估器

对于含有纯滞后的过程：

$$G_p(s) = G_0(s)e^{-\tau s}$$

其中 τ 为滞后时间。纯滞后的存在使系统的相位严重滞后，常规 PID 控制往往难以获得满意效果。

Smith 预估器

Smith 预估器的基本思想：利用被控对象的数学模型，将滞后环节从闭环特征方程中“移出”。其结构为在常规 PID 控制器基础上，并联一个预估模型：

$$G_m(s) = \hat{G}_0(s)(1 - e^{-\hat{\tau}s})$$

其中 $\hat{G}_0(s)$ 和 $\hat{\tau}$ 分别为 $G_0(s)$ 和 τ 的模型估计值。

当模型完全准确 ($\hat{G}_0 = G_0$, $\hat{\tau} = \tau$) 时，Smith 预估器将闭环特征方程中的滞后项消除，使控制器设计等同于对无滞后对象 $G_0(s)$ 的设计，从而可以选用较高的增益，获得更好的控制性能。

注. Smith 预估器的主要局限性在于对模型精度的敏感性。当模型存在误差时，性能可能显著下降，甚至出现不稳定。因此，Smith 预估器常与自适应控制或鲁棒控制结合使用。

§6.8 Python PID 控制器仿真

以下 Python 代码演示了 PID 控制器的数字仿真，包括 Ziegler-Nichols 参数整定和阶跃响应分析。

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 try:
4     import control as ct
5 except ImportError:
6     print("pip install control")
7     exit(1)
8
9 class PIDController:
10     def __init__(self, Kp, Ki, Kd, Ts, setpoint=0):
11         self.Kp = Kp
12         self.Ki = Ki
13         self.Kd = Kd
14         self.Ts = Ts

```

```

15     self.setpoint = setpoint
16     self.reset()
17
18     def reset(self):
19         self.prev_error = 0.0
20         self.prev_prev_error = 0.0
21         self.integral = 0.0
22         self.output = 0.0
23
24     def update(self, measurement):
25         error = self.setpoint - measurement
26         # 增量式 PID
27         P_term = self.Kp * (error - self.prev_error)
28         I_term = self.Ki * self.Ts * error
29         D_term = (self.Kd / self.Ts) * (error - 2*self.prev_error
30                                         + self.prev_prev_error)
31
32         delta_u = P_term + I_term + D_term
33         self.output += delta_u
34         # 抗积分饱和 (简单限幅)
35         self.output = np.clip(self.output, -10, 10)
36         self.prev_prev_error = self.prev_error
37         self.prev_error = error
38         return self.output
39
40 def simulate_pid():
41     # 被控对象:  $G(s) = 1 / (s^2 + 2s + 1)$ 
42     num = [1]
43     den = [1, 2, 1]
44     G = ct.TransferFunction(num, den)
45
46     # Ziegler-Nichols 第二法参数 (示例)
47     Ku = 2.0      # 临界增益
48     Tu = 3.14     # 临界振荡周期
49
50     # PID 参数
51     Kp = 0.6 * Ku
52     Ki = Kp / (0.5 * Tu)
53     Kd = Kp * 0.125 * Tu
54
55     print(f"Z-N 整定: Kp={Kp:.3f}, Ki={Ki:.3f}, Kd={Kd:.3f}")
56
57     # 仿真
58     Ts = 0.01
59     t_end = 20
60     t = np.arange(0, t_end, Ts)
61
62     pid = PIDController(Kp, Ki, Kd, Ts, setpoint=1.0)
63     y = np.zeros_like(t)
64     u = np.zeros_like(t)
65
66     # 简单离散仿真 (欧拉法)
67     x = np.array([0.0, 0.0]) # [x1, x2] = [y, dy/dt]

```

```

66     for i in range(1, len(t)):
67         u[i] = pid.update(y[i-1])
68         # 状态空间离散: A=[0 1; -1 -2], B=[0; 1]
69         dx = np.array([x[1], -x[0] - 2*x[1] + u[i]])
70         x = x + Ts * dx
71         y[i] = x[0]
72
73     # 绘图
74     fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(10, 6))
75     ax1.plot(t, y, 'b-', linewidth=1.5)
76     ax1.axhline(y=1.0, color='gray', linestyle='--')
77     ax1.set_ylabel('输出 $y(t)$')
78     ax1.set_title('PID 控制阶跃响应')
79     ax1.grid(True)
80     ax2.plot(t, u, 'r-', linewidth=1.0)
81     ax2.set_ylabel('控制量 $u(t)$')
82     ax2.set_xlabel('时间 (s)')
83     ax2.grid(True)
84     plt.tight_layout()
85     plt.show()
86
87     # 计算性能指标
88     overshoot = (np.max(y) - 1.0) / 1.0 * 100
89     steady_error = np.abs(y[-1] - 1.0)
90     print(f"超调量 = {overshoot:.2f}%")
91     print(f"稳态误差 = {steady_error:.4f}")
92
93 if __name__ == "__main__":
94     simulate_pid()

```

§6.9 本章小结

本章系统介绍了控制系统的校正方法。串联校正是最常用的校正方式，超前校正提供正相角以改善动态性能，滞后校正利用高频衰减提高稳态精度，滞后-超前校正结合了两者优先点。PID 控制作为工业标准，通过 P、I、D 三个环节的配合实现灵活的控制策略，Ziegler-Nichols 方法提供了简洁实用的参数整定途径。对于存在纯滞后的对象，Smith 预估器能够有效补偿滞后效应。前馈-反馈复合校正则利用不变性原理进一步提高系统性能。

在后续章节中，我们将进入现代控制理论部分，从状态空间的视角重新审视控制系统的分析与设计问题。

状态空间基础

状态空间法是现代控制理论的基石。与经典控制理论仅关注输入-输出关系不同，状态空间法通过引入系统内部的状态变量，可以完整描述系统的内部结构和动态行为，适用于多输入多输出（MIMO）系统、时变系统和非线性系统。

§7.1

状态空间的基本概念

7.1.1 状态与状态变量

状态变量

系统的状态变量是一组能够完全描述系统动态行为的最小变量集合。设 n 个状态变量为 $x_1(t), x_2(t), \dots, x_n(t)$ ，则只要知道这些状态变量在 t_0 时刻的值以及 $t \geq t_0$ 时刻的输入，就能唯一确定系统在 $t \geq t_0$ 任何时刻的行为。

状态向量与状态空间

将 n 个状态变量组成 n 维列向量

$$\mathbf{x}(t) = [x_1(t), x_2(t), \dots, x_n(t)]^T \in \mathbb{R}^n$$

称为状态向量。以 $x_1, x_2, \dots, x_n$ 为坐标轴张成的 n 维空间称为状态空间。

- 状态变量的选取不是唯一的，但个数 n （即系统阶数）是确定的。
- 物理上可测量的量（如位置、速度）常被选为状态变量。
- 最小性意味着状态变量之间是线性无关的。

7.1.2 状态方程与输出方程

线性定常连续系统的状态空间描述由两个矩阵方程组成：

状态空间模型

$$\dot{\mathbf{x}}(t) = \mathbf{A}\mathbf{x}(t) + \mathbf{B}\mathbf{u}(t) \quad (7.1)$$

$$\mathbf{y}(t) = \mathbf{C}\mathbf{x}(t) + \mathbf{D}\mathbf{u}(t) \quad (7.2)$$

其中：

- $\mathbf{x}(t) \in \mathbb{R}^n$ 为 n 维状态向量
- $\mathbf{u}(t) \in \mathbb{R}^m$ 为 m 维输入（控制）向量
- $\mathbf{y}(t) \in \mathbb{R}^p$ 为 p 维输出向量
- $\mathbf{A} \in \mathbb{R}^{n \times n}$ 为系统矩阵（状态矩阵）
- $\mathbf{B} \in \mathbb{R}^{n \times m}$ 为输入矩阵（控制矩阵）
- $\mathbf{C} \in \mathbb{R}^{p \times n}$ 为输出矩阵
- $\mathbf{D} \in \mathbb{R}^{p \times m}$ 为直接传递矩阵（前馈矩阵）

方程 $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u}$ 称为状态方程（一阶向量微分方程）， $\mathbf{y} = \mathbf{C}\mathbf{x} + \mathbf{D}\mathbf{u}$ 称为输出方程（代数方程）。

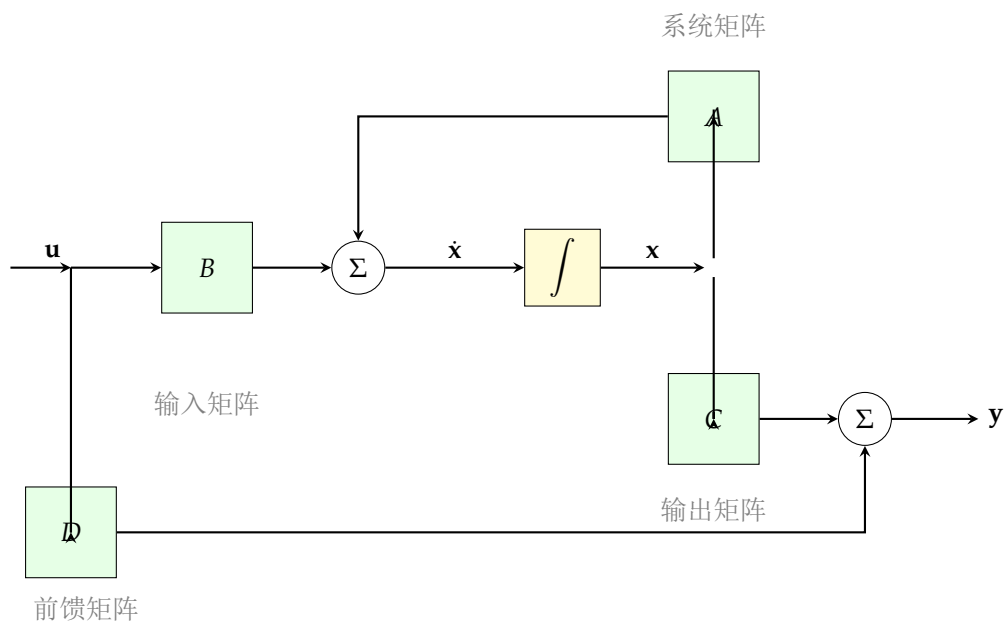


图 7.1: 线性定常系统的状态空间框图（MIMO 结构）

§7.2 从传递函数到状态空间

同一个传递函数可以对应无穷多种状态空间实现。以下是几种常用的**标准型**实现。

7.2.1 能控标准型

设单输入单输出（SISO）系统的传递函数为

$$G(s) = \frac{Y(s)}{U(s)} = \frac{b_0 s^n + b_1 s^{n-1} + \cdots + b_n}{s^n + a_1 s^{n-1} + \cdots + a_n}$$

当系统严格真（ $b_0 = 0$ ，即分母次数高于分子）时，能控标准型为：

$$A_c = \begin{bmatrix} 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 1 \\ -a_n & -a_{n-1} & -a_{n-2} & \cdots & -a_1 \end{bmatrix}, \quad B_c = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \\ 1 \end{bmatrix} \quad (7.3)$$

$$C_c = [b_n - a_n b_0, b_{n-1} - a_{n-1} b_0, \dots, b_1 - a_1 b_0] + b_0 [0, \dots, 0]$$

当 $b_0 = 0$ 时简化为 $C_c = [b_n, b_{n-1}, \dots, b_1]$ 。

能控标准型的特点

能控标准型中， A_c 的最后一行是分母多项式系数的负值（按升幂排列）， B_c 的最后一行元素为 1，其余为 0。该实现一定是能控的。

7.2.2 能观标准型

能观标准型可以通过对偶关系从能控标准型得到：

$$A_o = A_c^\top, \quad B_o = C_c^\top, \quad C_o = B_c^\top \quad (7.4)$$

7.2.3 对角标准型与 Jordan 标准型

当传递函数的分母多项式可分解为互异特征根时，可化为对角标准型。设

$$G(s) = \sum_{i=1}^n \frac{r_i}{s - \lambda_i}$$

其中 λ_i 为极点， r_i 为对应的留数。

$$A = \begin{bmatrix} \lambda_1 & 0 & \cdots & 0 \\ 0 & \lambda_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \lambda_n \end{bmatrix}, \quad B = \begin{bmatrix} 1 \\ 1 \\ \vdots \\ 1 \end{bmatrix}, \quad C = [r_1, r_2, \dots, r_n] \quad (7.5)$$

对角标准型的优点：各状态变量之间完全解耦（ A 为对角阵），便于分析和设计。

若存在重根，则需用 Jordan 标准型。对 m 重特征值 λ ，对应的 Jordan 块为：

$$J = \begin{bmatrix} \lambda & 1 & 0 & \cdots & 0 \\ 0 & \lambda & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \ddots & \vdots \\ 0 & 0 & \cdots & \lambda & 1 \\ 0 & 0 & \cdots & 0 & \lambda \end{bmatrix}_{m \times m}$$

【例题 7.1】将传递函数 $G(s) = \frac{2s+3}{s^2+3s+2}$ 化为能控标准型和对角标准型。

(1) 分母： $s^2 + 3s + 2 = (s+1)(s+2)$ ， $a_1 = 3$ ， $a_2 = 2$ 。分子： $b_1 = 2$ ， $b_2 = 3$ ($b_0 = 0$)。

能控标准型：

$$A_c = \begin{bmatrix} 0 & 1 \\ -2 & -3 \end{bmatrix}, B_c = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, C_c = [3, 2]$$

(2) 对角标准型：部分分式展开

$$G(s) = \frac{2s+3}{(s+1)(s+2)} = \frac{1}{s+1} + \frac{1}{s+2}$$

$$A = \begin{bmatrix} -1 & 0 \\ 0 & -2 \end{bmatrix}, B = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, C = [1, 1]$$

§7.3 从状态空间到传递函数

7.3.1 传递函数矩阵的推导

对状态方程两边取 Laplace 变换（零初始条件）：

$$s\mathbf{X}(s) = A\mathbf{X}(s) + B\mathbf{U}(s)$$

$$\mathbf{X}(s) = (sI - A)^{-1}B\mathbf{U}(s)$$

代入输出方程：

$$\mathbf{Y}(s) = C(sI - A)^{-1}B\mathbf{U}(s) + D\mathbf{U}(s)$$

传递函数矩阵

对于状态空间模型 (A, B, C, D) ，系统的传递函数矩阵为

$$G(s) = C(sI - A)^{-1}B + D$$

其维度为 $p \times m$ （输出数 $\times$ 输入数）。

$$(sI - A)^{-1} = \frac{\text{adj}(sI - A)}{\det(sI - A)} \quad (7.6)$$

$\det(sI - A)$ 即为系统的特征多项式。

【例题 7.2】求系统 $A = \begin{bmatrix} 0 & 1 \\ -2 & -3 \end{bmatrix}$ ， $B = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$ ， $C = [3, 2]$ ， $D = 0$ 的传递函数。

$$sI - A = \begin{bmatrix} s & -1 \\ 2 & s+3 \end{bmatrix}, \quad \det(sI - A) = s^2 + 3s + 2$$

$$(sI - A)^{-1} = \frac{1}{s^2 + 3s + 2} \begin{bmatrix} s+3 & 1 \\ -2 & s \end{bmatrix}$$

$$G(s) = C(sI - A)^{-1}B = \frac{1}{s^2 + 3s + 2} [3, 2] \begin{bmatrix} s+3 & 1 \\ -2 & s \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \frac{2s+3}{s^2 + 3s + 2}$$

与第 7.2 节的例一致，验证了状态空间实现与传递函数之间的等价性。

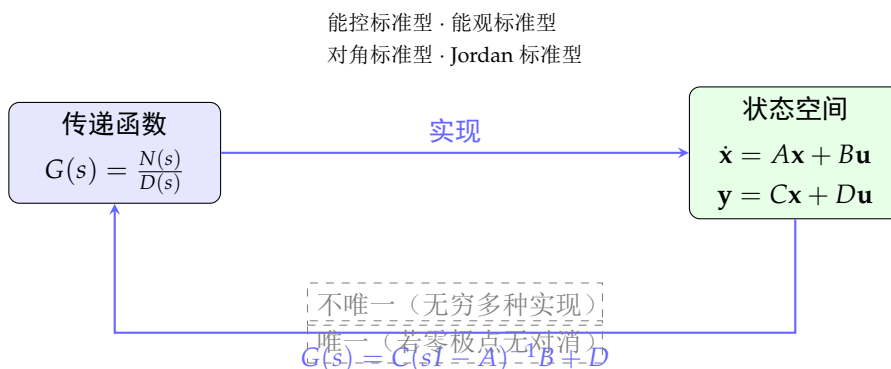


图 7.2: 传递函数与状态空间之间的转换关系

§7.4 状态方程的解

7.4.1 状态转移矩阵

齐次状态方程 $\dot{\mathbf{x}} = A\mathbf{x}$ (无输入情况) 的解为

$$\mathbf{x}(t) = e^{At} \mathbf{x}(0)$$

其中矩阵指数 e^{At} 称为状态转移矩阵，记为 $\Phi(t)$ 。

矩阵指数

$$\Phi(t) = e^{At} \triangleq I + At + \frac{1}{2!} A^2 t^2 + \cdots = \sum_{k=0}^{\infty} \frac{A^k t^k}{k!}$$

该级数对任意方阵 A 和 $t \geq 0$ 均收敛。

非齐次状态方程 $\dot{\mathbf{x}} = A\mathbf{x} + B\mathbf{u}$ 的解为

$$\mathbf{x}(t) = e^{At} \mathbf{x}(0) + \int_0^t e^{A(t-\tau)} B \mathbf{u}(\tau) d\tau$$

该形式称为状态转移公式，第一项为零输入响应，第二项为零状态响应。

7.4.2 Laplace 变换法

对状态方程取 Laplace 变换：

$$s\mathbf{X}(s) - \mathbf{x}(0) = A\mathbf{X}(s) + B\mathbf{U}(s)$$

$$\mathbf{X}(s) = (sI - A)^{-1}\mathbf{x}(0) + (sI - A)^{-1}B\mathbf{U}(s)$$

取 Laplace 反变换：

$$\mathbf{x}(t) = \mathcal{L}^{-1}\{(sI - A)^{-1}\}\mathbf{x}(0) + \mathcal{L}^{-1}\{(sI - A)^{-1}B\mathbf{U}(s)\}$$

由此得到状态转移矩阵的 Laplace 变换表示：

$$\Phi(t) = e^{At} = \mathcal{L}^{-1}\{(sI - A)^{-1}\}$$

7.4.3 Cayley-Hamilton 法

Cayley-Hamilton 定理

设 $A \in \mathbb{R}^{n \times n}$ 的特征多项式为

$$\Delta(\lambda) = \det(\lambda I - A) = \lambda^n + a_1\lambda^{n-1} + \cdots + a_n$$

则矩阵 A 满足自身的特征方程：

$$A^n + a_1A^{n-1} + \cdots + a_nI = 0$$

利用 Cayley-Hamilton 定理， A^n 及更高次幂均可表示为 $I, A, \dots, A^{n-1}$ 的线性组合，从而：

$$e^{At} = \alpha_0(t)I + \alpha_1(t)A + \cdots + \alpha_{n-1}(t)A^{n-1}$$

其中 $\alpha_i(t)$ 由特征值确定。

Cayley-Hamilton 法计算 $\Phi(t)$

1. 求 A 的特征值 $\lambda_1, \lambda_2, \dots, \lambda_n$ 。
2. 若特征值互异，解 Vandermonde 方程组：

$$e^{\lambda_i t} = \alpha_0 + \alpha_1\lambda_i + \cdots + \alpha_{n-1}\lambda_i^{n-1}, \quad i = 1, \dots, n$$

3. 若有重特征值，对重根附加求导条件。
4. 代入 $\Phi(t) = \sum_{k=0}^{n-1} \alpha_k(t)A^k$ 。

【例题 7.3】 计算 $A = \begin{bmatrix} 0 & 1 \\ 0 & -2 \end{bmatrix}$ 的状态转移矩阵。

特征值： $\lambda_1 = 0, \lambda_2 = -2$ (互异)。

$$\begin{cases} e^{0 \cdot t} = 1 = \alpha_0 \\ e^{-2t} = \alpha_0 - 2\alpha_1 \end{cases}$$

解得 $\alpha_0 = 1, \alpha_1 = (1 - e^{-2t})/2$ 。

$$\Phi(t) = \alpha_0 I + \alpha_1 A = \begin{bmatrix} 1 & \frac{1-e^{-2t}}{2} \\ 0 & e^{-2t} \end{bmatrix}$$

§7.5 状态转移矩阵的性质

状态转移矩阵的基本性质

状态转移矩阵 $\Phi(t) = e^{At}$ 满足以下性质：

1. 初始条件： $\Phi(0) = I$ (n 阶单位矩阵)

2. 群性质（半群性质）：

$$\Phi(t_1 + t_2) = \Phi(t_1)\Phi(t_2) = \Phi(t_2)\Phi(t_1), \quad \forall t_1, t_2$$

3. 可逆性：

$$\Phi^{-1}(t) = \Phi(-t)$$

4. 求导性质：

$$\frac{d}{dt}\Phi(t) = A\Phi(t) = \Phi(t)A$$

5. 状态转移：对任意初始时刻 t_0 ,

$$\mathbf{x}(t) = \Phi(t - t_0)\mathbf{x}(t_0)$$

6. 乘积性质：

$$\Phi(t_2 - t_0) = \Phi(t_2 - t_1)\Phi(t_1 - t_0)$$

7. 斜对称性：

$$\Phi^\top(-t) = [\Phi^{-1}(t)]^\top$$

证明. (证明性质 2 一群性质) 由矩阵指数的定义：

$$\Phi(t_1 + t_2) = e^{A(t_1+t_2)} = e^{At_1} \cdot e^{At_2}$$

因为 At_1 与 At_2 可交换（二者均为 A 的标量倍），所以矩阵指数的乘法性质成立。同理可证可逆性和求导性质。 $\square$

注. 群性质 $\Phi(t_1 + t_2) = \Phi(t_1)\Phi(t_2)$ 表明状态转移矩阵构成一个单参数变换群，这是线性定常系统的重要特征。对于线性时变系统，状态转移矩阵 $\Phi(t, t_0)$ 不具有此性质（依赖于两个参数），但仍满足 $\Phi(t_2, t_0) = \Phi(t_2, t_1)\Phi(t_1, t_0)$ 。

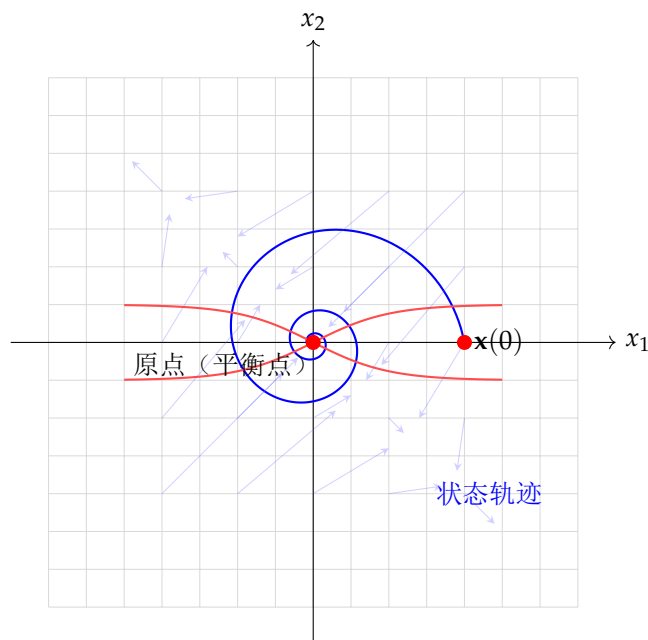


图 7.3: 二维状态空间中的状态轨迹（相平面图）

§7.6

线性变换与特征结构

7.6.1 坐标变换

对状态向量作非奇异线性变换 $\bar{\mathbf{x}} = P\mathbf{x}$ (P 为 $n \times n$ 非奇异阵), 则状态空间模型变为:

$$\bar{A} = PAP^{-1}, \quad \bar{B} = PB, \quad \bar{C} = CP^{-1}, \quad \bar{D} = D \quad (7.7)$$

线性变换下的不变性

在非奇异线性变换 $\bar{\mathbf{x}} = P\mathbf{x}$ 下, 以下量保持不变:

1. 特征值: $\det(\lambda I - \bar{A}) = \det(\lambda I - A)$
2. 传递函数矩阵: $G(s) = \bar{C}(sI - \bar{A})^{-1}\bar{B} + \bar{D} = C(sI - A)^{-1}B + D$
3. 能控性和能观性
4. 特征多项式

7.6.2 特征值与特征向量

特征值与特征向量

对于矩阵 $A \in \mathbb{R}^{n \times n}$ ，若存在标量 λ 和非零向量 $\mathbf{v}$ 满足

$$A\mathbf{v} = \lambda\mathbf{v}$$

则称 λ 为 A 的**特征值**， $\mathbf{v}$ 为对应的**特征向量**。

特征值由特征方程 $\det(\lambda I - A) = 0$ 确定。特征值决定了系统的动态特性：

- 实部为负的特征值对应衰减（稳定）模态
- 实部为正的特征值对应发散（不稳定）模态
- 虚部不为零的特征值对应振荡模态
- 特征值的模决定衰减/发散的速度

7.6.3 对角化与模态分解

若 A 有 n 个线性无关的特征向量 $\mathbf{v}_1, \dots, \mathbf{v}_n$ ，则取变换矩阵

$$P^{-1} = [\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_n]$$

可使 $\bar{A} = PAP^{-1}$ 化为对角阵 $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_n)$ 。

在对角化形式下，各状态变量（称为**模态坐标**）相互解耦，齐次解为

$$\mathbf{x}(t) = \sum_{i=1}^n c_i \mathbf{v}_i e^{\lambda_i t}$$

其中系数 c_i 由初始条件确定。每个模态独立地按 $e^{\lambda_i t}$ 的规律演化。

§7.7 离散时间状态空间

7.7.1 离散状态空间模型

线性定常离散时间系统的状态空间描述为：

离散状态空间模型

$$\mathbf{x}_{k+1} = G\mathbf{x}_k + H\mathbf{u}_k \quad (7.8)$$

$$\mathbf{y}_k = C\mathbf{x}_k + D\mathbf{u}_k \quad (7.9)$$

其中 $G \in \mathbb{R}^{n \times n}$ ， $H \in \mathbb{R}^{n \times m}$ ， k 为采样时刻。

离散系统的解：

$$\mathbf{x}_k = G^k \mathbf{x}_0 + \sum_{j=0}^{k-1} G^{k-1-j} H \mathbf{u}_j$$

离散传递函数矩阵：

$$G(z) = C(zI - G)^{-1}H + D$$

7.7.2 连续系统的离散化

将连续系统 $\dot{\mathbf{x}} = A\mathbf{x} + B\mathbf{u}$ 以采样周期 T_s 离散化。

精确离散化（零阶保持器假设）：

$$G = e^{AT_s}, \quad H = \left(\int_0^{T_s} e^{A\tau} d\tau \right) B \quad (7.10)$$

近似离散化（欧拉法）：

$$G \approx I + AT_s, \quad H \approx BT_s \quad (7.11)$$

双线性变换（Tustin 法）：

$$G \approx \left(I - \frac{T_s}{2} A \right)^{-1} \left(I + \frac{T_s}{2} A \right), \quad H \approx \left(I - \frac{T_s}{2} A \right)^{-1} B \sqrt{T_s} \quad (7.12)$$

离散化的稳定性保留条件

当连续系统采用零阶保持器精确离散化时，连续系统的稳定性被精确保留到离散系统：连续特征值 λ_i 映射为离散特征值 $e^{\lambda_i T_s}$ 。若 $\Re(\lambda_i) < 0$ ，则 $|e^{\lambda_i T_s}| < 1$ 。

7.7.3 离散系统的稳定性

离散系统的稳定条件：所有特征值的模均小于 1（即特征值位于单位圆内）。

$$|\lambda_i(G)| < 1, \quad i = 1, 2, \dots, n$$

【例题 7.4】连续系统 $A = \begin{bmatrix} 0 & 1 \\ -2 & -3 \end{bmatrix}$, $T_s = 0.1$ ，离散化。

精确离散化（使用 Python 计算）：

$$G = e^{AT_s} \approx \begin{bmatrix} 0.9909 & 0.0861 \\ -0.1722 & 0.7326 \end{bmatrix}$$

两个特征值均位于单位圆内，离散系统稳定。

§7.8 Python 状态方程数值求解

以下代码演示了使用 SciPy 的 `odeint` 求解状态方程，以及连续系统离散化和状态转移矩阵的计算。

```
1 import numpy as np
2 from scipy.integrate import odeint
3 from scipy.linalg import expm, eig
4 import matplotlib.pyplot as plt
5
6 # 定义连续系统
```

```

7  A = np.array([[0, 1],
8              [-2, -3]])
9  B = np.array([[0],
10             [1]])
11 C = np.array([[3, 2]])
12 D = np.array([[0]])
13
14 # 1. 状态方程数值求解
15 def state_equation(x, t, u_func):
16     u = u_func(t)
17     return A @ x + B.flatten() * u
18
19 def u_step(t):
20     return 1.0 if t >= 0 else 0.0
21
22 t = np.linspace(0, 10, 1000)
23 x0 = np.array([0.0, 0.0])
24 sol = odeint(state_equation, x0, t, args=(u_step,))
25
26 # 绘图
27 fig, axes = plt.subplots(2, 1, figsize=(10, 6))
28 axes[0].plot(t, sol[:, 0], 'b-', label='$x_1(t)$')
29 axes[0].plot(t, sol[:, 1], 'r--', label='$x_2(t)$')
30 axes[0].set_ylabel('状态变量')
31 axes[0].legend()
32 axes[0].grid(True)
33 axes[0].set_title('状态方程数值解 (odeint)')
34
35 y = sol @ C.T
36 axes[1].plot(t, y, 'g-', linewidth=1.5)
37 axes[1].set_ylabel('输出 $y(t)$')
38 axes[1].set_xlabel('时间 (s)')
39 axes[1].grid(True)
40 plt.tight_layout()
41 plt.show()
42
43 # 2. 计算状态转移矩阵
44 t_val = 1.0
45 Phi = expm(A * t_val)
46 print(f"状态转移矩阵 Phi({t_val}) = ")
47 print(Phi)
48
49 # 验证性质
50 Phi0 = expm(A * 0)
51 print(f"\nPhi(0) = I? {np.allclose(Phi0, np.eye(2))}")
52 Phi_neg = expm(A * (-t_val))
53 print(f"Phi(-t) = Phi(t)^(-1)? {np.allclose(Phi_neg, np.linalg.inv(Phi))}")
54
55 # 验证群性质
56 t1, t2 = 0.5, 0.7
57 Phi_sum = expm(A * (t1 + t2))

```

```

58 Phi_prod = expm(A * t1) @ expm(A * t2)
59 print(f"Phi(t1+t2) = Phi(t1)*Phi(t2)? "
60       f"{np.allclose(Phi_sum, Phi_prod)}")
61
62 # 3. 连续系统离散化 (零阶保持器)
63 Ts = 0.1
64 G = expm(A * Ts)
65 # H = integral_0^Ts exp(A*tau) * B dtau
66 n = A.shape[0]
67 M = np.zeros((n+1, n+1))
68 M[:n, :n] = A
69 M[:n, n] = B.flatten()
70 M_exp = expm(M * Ts)
71 G_d = M_exp[:n, :n]
72 H_d = M_exp[:n, n].reshape(-1, 1)
73
74 print(f"\n离散化 (Ts={Ts}):")
75 print(f"G = \n{G_d}")
76 print(f"H = \n{H_d}")
77
78 # 验证传递函数等价
79 import control as ct
80 sys_c = ct.ss(A, B, C, D)
81 sys_d = ct.ss(G_d, H_d, C, D, Ts)
82 print(f"\n连续极点: {np.linalg.eigvals(A)}")
83 print(f"\n离散极点: {np.linalg.eigvals(G_d)}")
84
85 # 4. 特征结构分析
86 eigvals, eigvecs = eig(A)
87 print(f"\n特征值: {eigvals}")
88 print(f"\n特征向量矩阵: \n{eigvecs}")
89
90 # 对角化验证
91 if np.linalg.matrix_rank(eigvecs) == n:
92     Lambda = np.diag(eigvals)
93     A_diag = eigvecs @ Lambda @ np.linalg.inv(eigvecs)
94     print(f"对角化验证: {np.allclose(A_diag, A)}")
95
96 plt.show()

```

§7.9 本章小结

本章建立了现代控制理论的基本框架——状态空间方法。我们从状态变量的概念出发，定义了状态方程和输出方程，引入了系统矩阵 A 、输入矩阵 B 、输出矩阵 C 和前馈矩阵 D 四个核心矩阵。在传递函数与状态空间之间，能控标准型、能观标准型和对角标准型提供了标准的实现途径。状态转移矩阵 $\Phi(t) = e^{At}$ 是分析状态方程解的核心工具，其群性质揭示了线性定常系统的本质结构。线性变换保持系统的输入-输出特性不变，特征值则决定了各模态的动态行为。最后，我们讨论了离散时间状态空间模型及其与连续模

型的联系，为数字控制系统的分析与设计奠定了基础。

下一章将深入探讨状态空间的两个基本结构性质——能控性与能观性，它们是现代控制理论的精髓所在。

能控性与能观性

能控性 (Controllability) 和能观性 (Observability) 是 Rudolf Kalman 于 1960 年提出的现代控制理论的两个核心概念。能控性研究输入能否将系统状态驱动到任意期望值，能观性研究输出能否唯一确定系统的内部状态。二者共同构成了系统结构分析的基础。

§ 8.1
能控性的基本概念

8.1.1 能控性的定义

状态能控性

对于线性定常系统 $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u}$ ，若对任意初始状态 $\mathbf{x}(0) = \mathbf{x}_0$ 和任意终端状态 $\mathbf{x}_f$ ，存在有限时刻 $t_f > 0$ 和定义在 $[0, t_f]$ 上的输入 $\mathbf{u}(t)$ ，使得 $\mathbf{x}(t_f) = \mathbf{x}_f$ ，则称系统是状态完全能控的，简称系统能控。

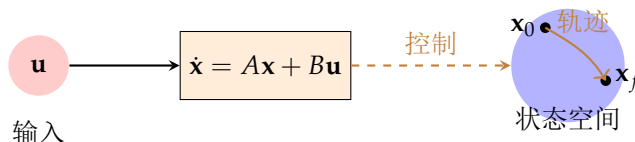
物理意义：输入信号能否将系统的状态从任意初始点“驱动”到任意目标点。

【例题 8.1】考虑系统

$$\dot{\mathbf{x}} = \begin{bmatrix} -1 & 0 \\ 0 & -2 \end{bmatrix} \mathbf{x} + \begin{bmatrix} 1 \\ 0 \end{bmatrix} u$$

状态 x_2 的运动方程为 $\dot{x}_2 = -2x_2$ ，与输入 u 无关。因此无论 $u(t)$ 如何选取，都无法改变 x_2 的演化，系统不完全能控。

不完全能控意味着系统包含“孤立”于输入之外的状态变量（或状态子空间），这些状态是输入无法影响的。



能否找到 $\mathbf{u}(t)$ 将系统
从 $\mathbf{x}_0$ 驱动到 $\mathbf{x}_f$?

图 8.1: 能控性的物理意义示意

§ 8.2 能控性判据

8.2.1 能控性矩阵与秩判据

能控性秩判据 (Kalman 秩判据)

线性定常系统 (A, B) 状态完全能控的充分必要条件是能控性矩阵

$$C = [B, AB, A^2B, \dots, A^{n-1}B]$$

满秩, 即

$$\text{rank}(C) = n$$

证明. 由状态方程的解:

$$\mathbf{x}(t_f) = e^{At_f} \mathbf{x}(0) + \int_0^{t_f} e^{A(t_f-\tau)} B \mathbf{u}(\tau) d\tau$$

整理得

$$\int_0^{t_f} e^{-A\tau} B \mathbf{u}(\tau) d\tau = e^{-At_f} \mathbf{x}(t_f) - \mathbf{x}(0) \triangleq \mathbf{w}$$

由 Cayley-Hamilton 定理, $e^{-A\tau}$ 可表为 $I, A, \dots, A^{n-1}$ 的线性组合:

$$e^{-A\tau} = \sum_{k=0}^{n-1} \alpha_k(\tau) A^k$$

代入得

$$\sum_{k=0}^{n-1} A^k B \int_0^{t_f} \alpha_k(\tau) \mathbf{u}(\tau) d\tau = \mathbf{w}$$

要使对任意 $\mathbf{w}$ 存在解 $\mathbf{u}(\tau)$, C 必须行满秩。□

能控性矩阵 C 的维度为 $n \times (n \cdot m)$ 。对于单输入系统 ($m = 1$), C 是 $n \times n$ 方阵, 能控条件简化为 $\det(C) \neq 0$ 。

8.2.2 PBH 判据

PBH (Popov-Belevitch-Hautus) 能控性判据

系统 (A, B) 能控的充要条件是: 对 A 的每个特征值 λ_i , 有

$$\text{rank}([\lambda_i I - A, B]) = n$$

PBH 特征向量判据

系统 (A, B) 能控的充要条件是: 不存在 A 的左特征向量 $\mathbf{w} \neq 0$ 使得 $\mathbf{w}^\top B = 0$ 。换言之, A 的左特征向量不能与 B 的所有列正交。

PBH 判据的优势是直接利用特征值信息, 可判断出哪些模态不能控。

【例题 8.2】 系统 $A = \begin{bmatrix} -1 & 0 \\ 0 & -2 \end{bmatrix}$, $B = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$ 。

能控性矩阵:

$$C = [B, AB] = \begin{bmatrix} 1 & -1 \\ 0 & 0 \end{bmatrix}, \quad \text{rank}(C) = 1 < 2$$

系统不能控。

PBH 检验 (特征值 $\lambda_2 = -2$):

$$[-2I - A, B] = \begin{bmatrix} -1 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}, \quad \text{rank} = 1 < 2$$

$\lambda_2 = -2$ 对应的左特征向量 $\mathbf{w} = [0, 1]^\top$ 满足 $\mathbf{w}^\top B = 0$, 对应模态不能控。

8.2.3 能控标准型与能控性

在第 7 章中介绍的能控标准型 (A_c, B_c) 一定是能控的。验证如下: 对 n 阶能控标准型,

$$C = [B_c, A_c B_c, \dots, A_c^{n-1} B_c] = \begin{bmatrix} 0 & 0 & \cdots & 0 & 1 \\ 0 & 0 & \cdots & 1 & * \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 1 & \cdots & * & * \\ 1 & * & \cdots & * & * \end{bmatrix}$$

该矩阵为下三角形式的伴随矩阵, 行列式为 ± 1 , 因此满秩。

§ 8.3

能观性的基本概念

8.3.1 能观性的定义

状态能观性

对于线性定常系统

$$\dot{\mathbf{x}} = A\mathbf{x} + B\mathbf{u}, \quad \mathbf{y} = C\mathbf{x} + D\mathbf{u}$$

若对任意初始状态 $\mathbf{x}(0) = \mathbf{x}_0$, 存在有限时刻 $t_f > 0$, 使得 $\mathbf{x}_0$ 可由 $[0, t_f]$ 上的输入 $\mathbf{u}(t)$ 和输出 $\mathbf{y}(t)$ 唯一确定, 则称系统是状态完全能观的, 简称系统能观。

物理意义: 仅通过测量系统的输入和输出, 能否“反推”出系统内部所有状态变量的初始值 (或当前值)。

注. 能观性不依赖于输入 $\mathbf{u}(t)$, 因为输入的影响可以通过计算扣除。因此, 检验能观性时通常令 $\mathbf{u}(t) \equiv 0$, 即仅分析齐次系统。

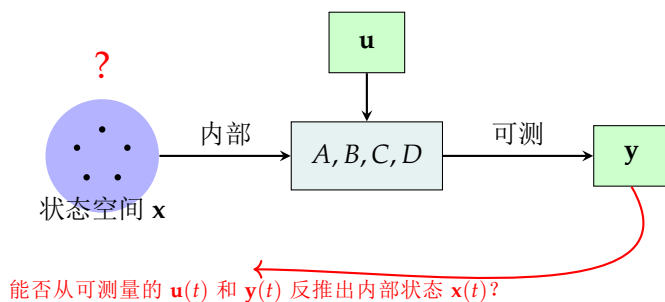


图 8.2: 能观性的物理意义示意

§ 8.4 能观性判据

8.4.1 能观性矩阵与秩判据

能观性秩判据 (Kalman 秩判据)

线性定常系统 (A, C) 状态完全能观的充分必要条件是能观性矩阵

$$\mathcal{O} = \begin{bmatrix} C \\ CA \\ CA^2 \\ \vdots \\ CA^{n-1} \end{bmatrix}$$

满秩, 即

$$\text{rank}(\mathcal{O}) = n$$

能观性矩阵 $\mathcal{O}$ 的维度为 $(n \cdot p) \times n$ 。对于单输出系统 ($p = 1$), $\mathcal{O}$ 是 $n \times n$ 方阵。

证明. 令 $\mathbf{u}(t) \equiv 0$, 则输出为 $\mathbf{y}(t) = Ce^{At}\mathbf{x}(0)$ 。利用 Cayley-Hamilton 定理将 e^{At} 展开, 对输出及其各阶导数在 $t = 0$ 处求值, 得

$$\begin{bmatrix} \mathbf{y}(0) \\ \dot{\mathbf{y}}(0) \\ \vdots \\ \mathbf{y}^{(n-1)}(0) \end{bmatrix} = \mathcal{O} \mathbf{x}(0)$$

要从输出信息唯一确定 $\mathbf{x}(0)$, $\mathcal{O}$ 必须列满秩。 □

8.4.2 PBH 能观性判据

PBH 能观性判据

系统 (A, C) 能观的充要条件是: 对 A 的每个特征值 λ_i , 有

$$\text{rank} \begin{bmatrix} \lambda_i I - A \\ C \end{bmatrix} = n$$

PBH 特征向量判据 (能观性)

系统 (A, C) 能观的充要条件是: 不存在 A 的右特征向量 $\mathbf{v} \neq 0$ 使得 $C\mathbf{v} = 0$ 。换言之, A 的特征向量不能落在 C 的零空间中。

【例题 8.3】 系统 $A = \begin{bmatrix} -1 & 0 \\ 0 & -2 \end{bmatrix}$, $C = [1, 0]$ 。

能观性矩阵:

$$\mathcal{O} = \begin{bmatrix} C \\ CA \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ -1 & 0 \end{bmatrix}, \quad \text{rank}(\mathcal{O}) = 1 < 2$$

系统不能观。 $\lambda = -2$ 对应的特征向量 $\mathbf{v} = [0, 1]^T$ 满足 $C\mathbf{v} = 0$, 该模态在输出中不可见。

注. 能观性判据的物理含义是：系统必须保证每一个模态（特征向量方向上的运动）都能在输出中留下“痕迹”。如果某个特征向量 $\mathbf{v}_i$ 满足 $\mathbf{C}\mathbf{v}_i = 0$ ，则该模态对输出毫无贡献，自然无法从输出反推。

§ 8.5 对偶原理

对偶原理

对于两个系统：

$$\Sigma_1 = (A, B, C), \quad \Sigma_2 = (A^\top, C^\top, B^\top)$$

则 Σ_1 能控的充要条件是 Σ_2 能观； Σ_1 能观的充要条件是 Σ_2 能控。

证明. Σ_1 的能控性矩阵为

$$\mathcal{C}_{\Sigma_1} = [B, AB, \dots, A^{n-1}B]$$

Σ_2 的能观性矩阵为

$$\mathcal{O}_{\Sigma_2} = \begin{bmatrix} B^\top \\ B^\top A^\top \\ \vdots \\ B^\top (A^\top)^{n-1} \end{bmatrix} = \mathcal{C}_{\Sigma_1}^\top$$

因此 $\text{rank}(\mathcal{C}_{\Sigma_1}) = \text{rank}(\mathcal{O}_{\Sigma_2})$ 。同理 $\text{rank}(\mathcal{O}_{\Sigma_1}) = \text{rank}(\mathcal{C}_{\Sigma_2})$ 。 □

注. 对偶原理揭示了能控性和能观性在数学上的完美对称性。这在工程上有重要应用：能控性分析的结果可以直接“对偶”地转化为能观性结论，反之亦然。

8.5.1 对偶原理的应用

对偶原理为控制系统的设计提供了重要工具：

- 状态反馈极点配置问题（对能控系统）与全维状态观测器设计问题（对能观系统）是对偶的
- 最优控制问题（LQR）与最优估计问题（Kalman 滤波）在数学结构上对偶
- 可设计对偶变换，将状态观测器设计转化为一个“对偶系统”的状态反馈设计

§ 8.6 Kalman 分解

8.6.1 结构分解的基本思想

对于不完全能控和/或不完全能观的系统，可以通过适当的坐标变换将其分解为四个具有不同结构特性的子系统。

Kalman 规范分解定理

对于任意线性定常系统 (A, B, C) ，存在非奇异变换 $\bar{\mathbf{x}} = P\mathbf{x}$ ，使变换后的系统具有以下分块结构：

$$\begin{bmatrix} \dot{\bar{\mathbf{x}}}_{co} \\ \dot{\bar{\mathbf{x}}}_{c\bar{o}} \\ \dot{\bar{\mathbf{x}}}_{\bar{c}o} \\ \dot{\bar{\mathbf{x}}}_{\bar{c}\bar{o}} \end{bmatrix} = \begin{bmatrix} A_{11} & 0 & A_{13} & 0 \\ A_{21} & A_{22} & A_{23} & A_{24} \\ 0 & 0 & A_{33} & 0 \\ 0 & 0 & A_{43} & A_{44} \end{bmatrix} \begin{bmatrix} \bar{\mathbf{x}}_{co} \\ \bar{\mathbf{x}}_{c\bar{o}} \\ \bar{\mathbf{x}}_{\bar{c}o} \\ \bar{\mathbf{x}}_{\bar{c}\bar{o}} \end{bmatrix} + \begin{bmatrix} B_1 \\ B_2 \\ 0 \\ 0 \end{bmatrix} \mathbf{u} \quad (8.1)$$

$$\mathbf{y} = \begin{bmatrix} C_1 & 0 & C_3 & 0 \end{bmatrix} \begin{bmatrix} \bar{\mathbf{x}}_{co} \\ \bar{\mathbf{x}}_{c\bar{o}} \\ \bar{\mathbf{x}}_{\bar{c}o} \\ \bar{\mathbf{x}}_{\bar{c}\bar{o}} \end{bmatrix} + D\mathbf{u} \quad (8.2)$$

四个子系统的含义：

Kalman 分解的四个子系统

1. $\bar{\mathbf{x}}_{co}$ ：能控且能观子系统 — 传递函数对应的最小实现部分。
2. $\bar{\mathbf{x}}_{c\bar{o}}$ ：能控不能观子系统 — 输入可驱动，但输出中不可见。
3. $\bar{\mathbf{x}}_{\bar{c}o}$ ：不能控能观子系统 — 输出中可见，但输入无法影响。
4. $\bar{\mathbf{x}}_{\bar{c}\bar{o}}$ ：不能控不能观子系统 — 输入无法影响，输出中不可见。

系统传递函数矩阵仅由子系统 co （能控能观部分）决定：

$$G(s) = C_1(sI - A_{11})^{-1}B_1 + D$$

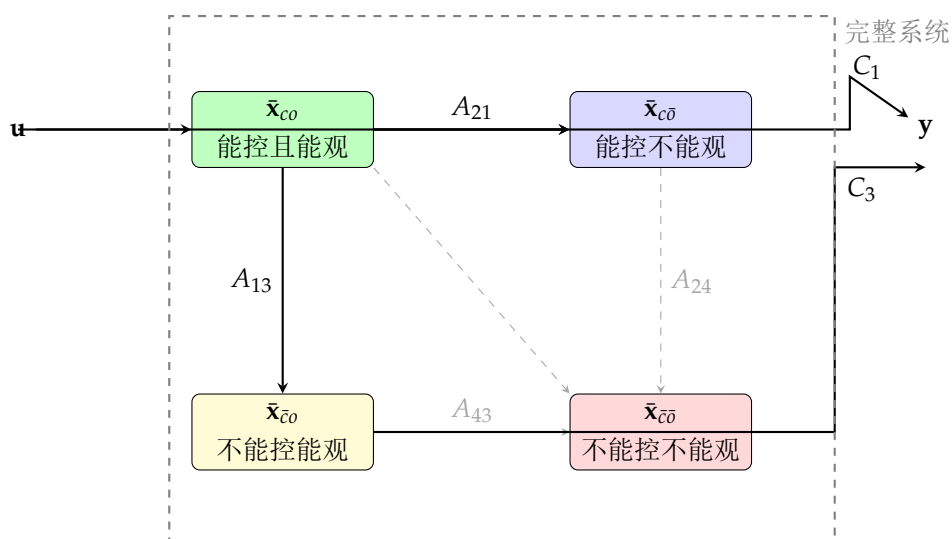


图 8.3: Kalman 分解的四部分结构

8.6.2 Kalman 分解的构造方法

构造 Kalman 分解的步骤：

1. 计算能控性矩阵 C ，设其秩为 $n_c < n$ 。取 C 中 n_c 个线性无关列作为能控子空间的基，并扩充为 $\mathbb{R}^n$ 的基，构成变换矩阵的前 n_c 列。
2. 同理或利用对偶性处理能观子空间。
3. 将能控子空间和能观子空间求交集、并集，构造四个子空间的基向量。
4. 组成变换矩阵 P ，计算 $\bar{A} = PAP^{-1}$ 、 $\bar{B} = PB$ 、 $\bar{C} = CP^{-1}$ 。

【例题 8.4】考虑系统

$$A = \begin{bmatrix} -2 & 0 \\ 0 & -2 \end{bmatrix}, B = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, C = [0, 1], D = 0$$

能控性： $\text{rank}(C) = 1$ (x_2 不能控)。能观性： $\text{rank}(\mathcal{O}) = 1$ (x_1 不能观)。经 Kalman 分解：

$$\bar{A} = \begin{bmatrix} -2 & 0 \\ 0 & -2 \end{bmatrix}, \bar{B} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \bar{C} = [0, 1]$$

能控能观子空间维数为 0，系统传递函数为 $G(s) = 0$ （零极相消）。

§8.7 最小实现

8.7.1 最小实现的定义

最小实现

对于给定的传递函数矩阵 $G(s)$ ，若存在状态空间实现 (A, B, C, D) 使得

$$G(s) = C(sI - A)^{-1}B + D$$

且该实现的阶数 n 是有可能实现中最小的，则称 (A, B, C, D) 为 $G(s)$ 的**最小实现**。

最小实现的充要条件

状态空间实现 (A, B, C, D) 是最小实现的充分必要条件是系统既能控又能观。

证明. 若系统不能控或不能观，根据 Kalman 分解，其传递函数仅由能控能观子部分 (A_{11}, B_1, C_1) 决定，而该子部分的阶数小于原系统的阶数，因此原实现不是最小的。反之，若系统既能控又能观，则传递函数的任何实现阶数都不小于 n 。□

8.7.2 不可约传递函数

当传递函数的分子分母多项式存在公因子（零极点对消）时，任何状态空间实现的阶数都将大于最小实现的阶数。具体来说：

- 若 SISO 传递函数 $G(s) = N(s)/D(s)$ 有零极点对消，则消去公因子后的阶数为最小实现的阶数。
- 在消去公因子后，系统既能控又能观。
- 对消的零极点对对应的模态要么不能控，要么不能观（或两者兼有），它们不出现在传递函数中。

【例题 8.5】 传递函数 $G(s) = \frac{s+1}{(s+1)(s+2)} = \frac{1}{s+2}$ 。分母分子有公因子 $(s+1)$ ，发生零极对消。最小实现为一阶系统， $\tilde{A} = -2$ ， $\tilde{B} = 1$ ， $\tilde{C} = 1$ 。若实现为二阶（保存对消因式），则为不能控或不能观的非最小实现。

注. 最小实现的概念在系统辨识和控制器降阶设计中非常重要。实际应用中，通常希望获得阶数尽可能低的模型，以简化分析和控制器设计。

§8.8 Python 能控性与能观性分析

以下 Python 代码演示了能控性矩阵、能观性矩阵的计算，秩检验，PBH 检验以及 Kalman 分解的数值实现。

```

1 import numpy as np
2 from scipy.linalg import eig, null_space, matrix_rank
3 import control as ct
4
5 def controllability_matrix(A, B):
6     n = A.shape[0]
7     B = np.atleast_2d(B)
8     if B.shape[1] == 1:
9         B = B.reshape(-1, 1)
10    Ctrb = B.copy()
11    for i in range(1, n):
12        Ctrb = np.hstack([Ctrb, np.linalg.matrix_power(A, i) @ B])
13    return Ctrb
14
15 def observability_matrix(A, C):
16     n = A.shape[0]
17     C = np.atleast_2d(C)
18     Obsv = C.copy()
19     for i in range(1, n):
20        Obsv = np.vstack([Obsv, C @ np.linalg.matrix_power(A, i)])
21    return Obsv
22
23 def is_controllable(A, B):
24     n = A.shape[0]
25     Ctrb = controllability_matrix(A, B)
26     r = np.linalg.matrix_rank(Ctrb)
27     print(f"能控性矩阵秩: {r} / {n}")
28     return r == n
29
30 def is_observable(A, C):
31     n = A.shape[0]
32     Obsv = observability_matrix(A, C)
33     r = np.linalg.matrix_rank(Obsv)
34     print(f"能观性矩阵秩: {r} / {n}")
35     return r == n
36
37 def pbh_test_controllability(A, B):

```

```

38     n = A.shape[0]
39     eigvals = np.linalg.eigvals(A)
40     controllable = True
41     for lam in eigvals:
42         M = np.hstack([lam * np.eye(n) - A, B])
43         r = np.linalg.matrix_rank(M)
44         if r < n:
45             print(f"PBH: lambda={lam:.4f} 秩={r}<{n} -> 不能控")
46             controllable = False
47     return controllable
48
49 def pbh_test_observability(A, C):
50     n = A.shape[0]
51     eigvals = np.linalg.eigvals(A)
52     observable = True
53     for lam in eigvals:
54         M = np.vstack([lam * np.eye(n) - A, C])
55         r = np.linalg.matrix_rank(M)
56         if r < n:
57             print(f"PBH: lambda={lam:.4f} 秩={r}<{n} -> 不能观")
58             observable = False
59     return observable
60
61 # --- 示例 1: 能控/能观检验 ---
62 print("=" * 50)
63 print("示例 1: 典型二阶系统")
64 A1 = np.array([[0, 1],
65                [-2, -3]])
66 B1 = np.array([[0],
67                [1]])
68 C1 = np.array([[1, 0]])
69
70 print("能控性检验:")
71 print(f" 能控: {is_controllable(A1, B1)}")
72 print("能观性检验:")
73 print(f" 能观: {is_observable(A1, C1)}")
74
75 # --- 示例 2: 不完全能控系统 ---
76 print("\n" + "=" * 50)
77 print("示例 2: 不完全能控系统")
78 A2 = np.array([[ -1, 0],
79                [ 0, -2]])
80 B2 = np.array([[1],
81                [0]])
82 C2 = np.array([[1, 1]])
83
84 print("能控性矩阵:")
85 Ctrb2 = controllability_matrix(A2, B2)
86 print(Ctrb2)
87 print(f"rank = {np.linalg.matrix_rank(Ctrb2)}")
88 print(f"能控: {is_controllable(A2, B2)}")

```

```

89 print(f"能观: {is_observable(A2, C2)}")
90
91 # --- 示例 3: PBH 检验 ---
92 print("\n" + "=" * 50)
93 print("示例 3: PBH 判据检验")
94 print("PBH 能控性检验 (A2, B2):")
95 pbh_test_controllability(A2, B2)
96 print("PBH 能观性检验 (A2, C2):")
97 pbh_test_observability(A2, C2)
98
99 # --- 示例 4: 最小实现验证 ---
100 print("\n" + "=" * 50)
101 print("示例 4: 最小实现与传递函数")
102 sys = ct.ss(A1, B1, C1, 0)
103 G = ct.ss2tf(sys)
104 print("传递函数:", G)
105
106 # 验证对偶原理
107 A_dual = A1.T
108 B_dual = C1.T
109 C_dual = B1.T
110 print(f"\n原系统 能控={is_controllable(A1,B1)}, 能观={is_observable(A1,C1)}")
111 print(f"对偶系统 能控={is_controllable(A_dual,B_dual)}, "
112       f"能观={is_observable(A_dual,C_dual)}")

```

§8.9 本章小结

能控性和能观性共同构成了现代控制理论的两根支柱。本章系统阐述了这两个概念的定义、物理意义和严格的数学判据。

能控性矩阵 $C = [B, AB, \dots, A^{n-1}B]$ 和能观性矩阵 O 的秩条件是检验系统结构性质的基本工具。PBH 判据则从特征值和特征向量的角度提供了更深层的洞察——某个模态不能控当且仅当存在 A 的左特征向量与 B 的所有列正交；某个模态不能观当且仅当存在 A 的右特征向量落在 C 的零空间中。

对偶原理 $(A, B, C) \leftrightarrow (A^T, C^T, B^T)$ 揭示了能控性与能观性之间的优美对称性，并将两者统一在同一个数学框架下。Kalman 分解将任意系统规范地分解为四个部分，其核心结论是：系统的传递函数仅由能控能观子部分决定。由此自然地引出最小实现的概念——既能控又能观的状态空间实现。

在后续的现代控制理论内容中，能控性是状态反馈极点配置的前提，能观性是状态观测器设计的前提，二者共同为控制器-观测器综合奠定了理论基石。

极点配置与状态反馈

在现代控制理论中，“极点配置”是状态反馈控制器设计的核心方法。经典控制理论通过调节 PID 参数来改变闭环极点的位置，而现代控制理论则利用完整的系统状态信息，通过状态反馈矩阵 K 可以“任意配置”闭环系统的极点位置。本章从状态反馈的基本概念出发，逐步介绍极点配置定理、Ackermann 公式、状态观测器设计以及分离原理，最终建立基于观测器的输出反馈控制框架。状态反馈和极点配置为后续章节的最优控制理论奠定了坚实的方法论基础。

§9.1

状态反馈的基本概念

9.1.1 状态反馈的结构

考虑一个 n 阶连续时间线性时不变系统：

$$\dot{\mathbf{x}} = A\mathbf{x} + B\mathbf{u}, \quad \mathbf{y} = C\mathbf{x} \quad (9.1)$$

其中 $\mathbf{x} \in \mathbb{R}^n$ 为状态向量， $\mathbf{u} \in \mathbb{R}^m$ 为控制输入， $\mathbf{y} \in \mathbb{R}^p$ 为输出向量。状态反馈控制律定义为

$$\mathbf{u} = -K\mathbf{x} + \mathbf{v} \quad (9.2)$$

其中 $K \in \mathbb{R}^{m \times n}$ 是状态反馈增益矩阵， $\mathbf{v}$ 是新的参考输入（也称外生输入）。将反馈律代入原系统，得到闭环系统：

$$\dot{\mathbf{x}} = (A - BK)\mathbf{x} + B\mathbf{v} \quad (9.3)$$

状态反馈对能控性的影响

状态反馈不改变系统的能控性，即 (A, B) 能控当且仅当 $(A - BK, B)$ 能控。注意，状态反馈可能改变系统的能观性。

证明. 对于任意 K ，构造能控性矩阵：

$$\mathcal{C}_{A-BK} = \begin{bmatrix} B & (A - BK)B & \cdots & (A - BK)^{n-1}B \end{bmatrix}$$

可以证明 $\text{rank}(C_{A-BK}) = \text{rank}(C_A)$ ，因为 $(A-BK)^i B$ 可以表示为 $A^i B$ 加上 $A^j B$ ($j < i$) 的线性组合。□

注. 能控性不变是状态反馈最重要的性质之一，它保证了我们可以利用反馈来自由配置闭环系统的动力学行为，而不会“失去”对任何模态的控制能力。

9.1.2 闭环系统的特征值

闭环系统的特征方程为

$$\det(sI - (A - BK)) = 0$$

闭环极点为矩阵 $A - BK$ 的特征值： $\lambda_1^{\text{cl}}, \lambda_2^{\text{cl}}, \dots, \lambda_n^{\text{cl}}$ 。通过选择不同的反馈增益 K ，闭环极点将在复平面上移动。所谓的“极点配置”问题，就是对于一个给定的期望闭环极点集合

$$\Lambda^{\text{des}} = \{\lambda_1^{\text{d}}, \lambda_2^{\text{d}}, \dots, \lambda_n^{\text{d}}\}$$

(其中复数极点必须以共轭对出现)，寻找 K 使得

$$\lambda_i(A - BK) = \lambda_i^{\text{d}}, \quad i = 1, 2, \dots, n$$

期望极点选择原则

在工程实践中，期望极点的选择通常遵循以下原则：

1. 主导极点（最靠近虚轴的极点）应具有期望的阻尼比 ζ 和自然频率 ω_n ，以保证动态响应满足超调量和调节时间等指标；
2. 非主导极点的实部应远小于主导极点实部（一般取 5-10 倍），使其衰减足够快而不会显著影响系统响应；
3. 所有极点必须具有负实部以保证闭环稳定性；
4. 当系统有复数期望极点时，必须成对配置以保证反馈增益矩阵 K 为实矩阵。

§9.2 极点配置定理

9.2.1 能控性与极点配置的等价性

极点配置定理 (Wonham, 1967)

线性时不变系统 $\dot{\mathbf{x}} = A\mathbf{x} + B\mathbf{u}$ 可以通过状态反馈 $\mathbf{u} = -K\mathbf{x}$ 任意配置闭环极点的充分必要条件是系统完全能控。

证明. 充分性：假设系统处于能控标准型。对于单输入系统 ($m = 1$)，设其能控标准型为

$$A_c = \begin{bmatrix} 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 1 \\ -a_0 & -a_1 & -a_2 & \cdots & -a_{n-1} \end{bmatrix}, \quad B_c = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \\ 1 \end{bmatrix}$$

设期望闭环特征多项式为

$$\alpha_d(s) = s^n + \alpha_{n-1}s^{n-1} + \cdots + \alpha_1s + \alpha_0$$

取反馈增益矩阵

$$K = \begin{bmatrix} \alpha_0 - a_0 & \alpha_1 - a_1 & \cdots & \alpha_{n-1} - a_{n-1} \end{bmatrix}$$

则 $A_c - B_c K$ 的最后一行恰好由 $-\alpha_0, -\alpha_1, \dots, -\alpha_{n-1}$ 组成，其特征多项式即为 $\alpha_d(s)$ 。

必要性：若系统不完全能控，则存在能控性分解，其中不能控子系统的极点不受状态反馈影响，因此无法任意配置。 $\square$

9.2.2 多输入系统的极点配置

对于多输入系统 ($m > 1$)，反馈增益矩阵 K 中有 $m \times n$ 个自由参数，而对 n 个极点配置只有 n 个约束条件（对应特征多项式的 n 个系数）。因此多输入系统存在无限多种 K 可以实现相同的极点配置，这种自由度可用于实现额外的设计目标（如鲁棒性优化）。

常用的多输入极点配置方法包括：

1. **转换为单输入问题**：选取合适的 $m \times 1$ 向量 $\mathbf{p}$ ，构造单输入系统 $(A, B\mathbf{p})$ （需保证 $(A, B\mathbf{p})$ 能控），按单输入方法设计反馈 $K_1 \in \mathbb{R}^{1 \times n}$ ，最终取 $K = \mathbf{p}K_1$ ；
2. **特征结构配置**：不仅指定极点位置，还指定对应的特征向量方向，充分利用多输入自由度；
3. **数值优化方法**：通过求解 Riccati 方程或线性矩阵不等式（LMI）获得具有鲁棒性的反馈增益。

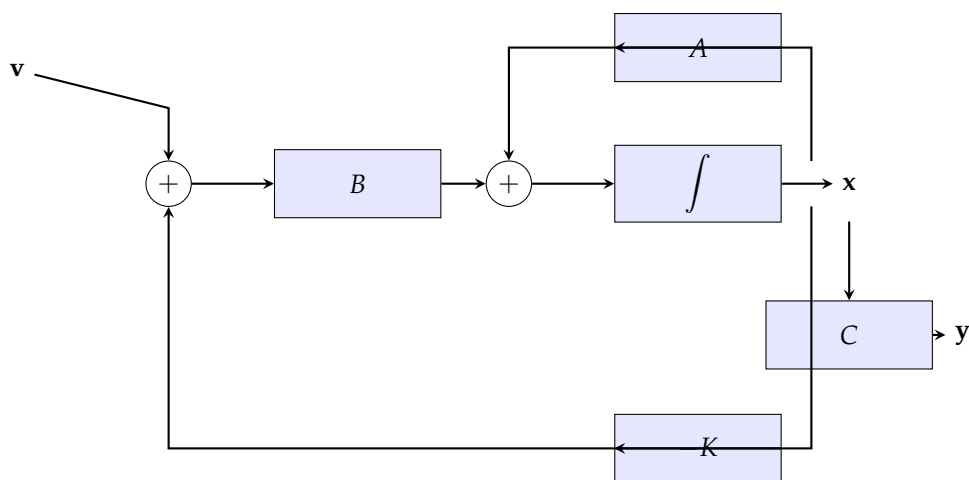


图 9.1: 状态反馈控制系统结构框图。状态 $\mathbf{x}$ 通过反馈增益 $-K$ 叠加到参考输入 $\mathbf{v}$ 上，形成闭环动态。

§9.3 Ackermann 公式

9.3.1 公式推导

Ackermann 公式为单输入系统提供了一种一步计算反馈增益矩阵 K 的解析方法，是极点配置问题最经典的封闭解。

Ackermann 公式

对于完全能控的单输入系统 (A, B) ，若期望的闭环特征多项式为

$$\alpha_d(s) = s^n + \alpha_{n-1}s^{n-1} + \cdots + \alpha_1s + \alpha_0$$

则使得闭环极点恰好为 $\alpha_d(s)$ 的根的状态反馈增益为

$$K = \begin{bmatrix} 0 & 0 & \cdots & 0 & 1 \end{bmatrix} C^{-1} \alpha_d(A)$$

其中 $C = [B, AB, \dots, A^{n-1}B]$ 为能控性矩阵， $\alpha_d(A)$ 是将多项式 $\alpha_d(s)$ 中的 s 替换为矩阵 A 所得的矩阵多项式：

$$\alpha_d(A) = A^n + \alpha_{n-1}A^{n-1} + \cdots + \alpha_1A + \alpha_0I$$

证明. 利用能控标准型的性质。设变换 T 将 (A, B) 变换为能控标准型 (A_c, B_c) ，即

$$A_c = TAT^{-1}, \quad B_c = TB$$

其中 $T = C_c C^{-1}$ ， C_c 是能控标准型的能控性矩阵。

已知在能控标准型下的反馈增益为 $K_c = [\alpha_0 - a_0, \dots, \alpha_{n-1} - a_{n-1}]$ 。由 Cayley-Hamilton 定理，

$$\alpha_d(A_c) = A_c^n + \alpha_{n-1}A_c^{n-1} + \cdots + \alpha_0I$$

考虑 $\alpha_d(A_c)$ 的最后一行，它恰好等于 K_c 。因此

$$K_c = e_n^\top \alpha_d(A_c)$$

其中 $e_n^\top = [0, \dots, 0, 1]$ 。在原坐标系下，

$$K = K_c T = e_n^\top \alpha_d(A_c) T = e_n^\top T \alpha_d(A) T^{-1} T = e_n^\top T \alpha_d(A)$$

而 $e_n^\top T = e_n^\top C_c C^{-1}$ 。在能控标准型中， C_c 是特定的上三角矩阵， $e_n^\top C_c = [0, \dots, 0, 1]$ ，从而 $e_n^\top T = [0, \dots, 0, 1]C^{-1}$ ，即得证。 $\square$

9.3.2 计算步骤与应用

利用 Ackermann 公式计算状态反馈增益的步骤：

Ackermann 极点配置算法

1. 确定期望闭环特征多项式 $\alpha_d(s) = (s - \lambda_1^d)(s - \lambda_2^d) \cdots (s - \lambda_n^d)$ ，提取系数 $\alpha_0, \alpha_1, \dots, \alpha_{n-1}$ ；
2. 构造能控性矩阵 $C = [B, AB, \dots, A^{n-1}B]$ ，验证 $\text{rank}(C) = n$ ；
3. 计算矩阵多项式 $\alpha_d(A) = A^n + \alpha_{n-1}A^{n-1} + \cdots + \alpha_1A + \alpha_0I$ ；
4. 计算反馈增益 $K = e_n^\top C^{-1} \alpha_d(A)$ 。

【例题 9.1】(二阶系统的极点配置) 考虑系统

$$A = \begin{bmatrix} 0 & 1 \\ 2 & 3 \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

期望闭环极点为 $\lambda_{1,2}^d = -3 \pm j4$ 。

期望特征多项式为

$$\alpha_d(s) = (s + 3 - j4)(s + 3 + j4) = s^2 + 6s + 25$$

即 $\alpha_1 = 6$, $\alpha_0 = 25$ 。

能控性矩阵：

$$C = \begin{bmatrix} 0 & 1 \\ 1 & 3 \end{bmatrix}, \quad C^{-1} = \begin{bmatrix} -3 & 1 \\ 1 & 0 \end{bmatrix}$$

矩阵多项式：

$$\alpha_d(A) = A^2 + 6A + 25I = \begin{bmatrix} 2 & 3 \\ 6 & 11 \end{bmatrix} + \begin{bmatrix} 0 & 6 \\ 12 & 18 \end{bmatrix} + \begin{bmatrix} 25 & 0 \\ 0 & 25 \end{bmatrix} = \begin{bmatrix} 27 & 9 \\ 18 & 54 \end{bmatrix}$$

反馈增益：

$$K = \begin{bmatrix} 0 & 1 \end{bmatrix} \begin{bmatrix} -3 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 27 & 9 \\ 18 & 54 \end{bmatrix} = \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} 27 & 9 \\ 18 & 54 \end{bmatrix} = \begin{bmatrix} 27 & 9 \end{bmatrix}$$

验证： $A - BK = \begin{bmatrix} 0 & 1 \\ -25 & -6 \end{bmatrix}$ ，其特征值为 $-3 \pm j4$ ，与期望一致。

注. Ackermann 公式在数值计算中可能不稳定（当系统阶数较高时），因为需要计算矩阵幂 A^n ，可能引入较大的舍入误差。在实际工程中，通常使用更鲁棒的数值方法（如 MATLAB 的 `place` 命令），但在低阶系统和理论分析中，Ackermann 公式仍然是最清晰的解析表达。

§9.4 状态观测器

当系统状态无法直接测量时，需要设计状态观测器来“估计”状态，然后用估计状态代替真实状态进行反馈控制。

9.4.1 开环与闭环观测器

开环观测器（Luenberger, 1964 年的早期版本）直接仿真原系统：

$$\dot{\hat{\mathbf{x}}} = A\hat{\mathbf{x}} + B\mathbf{u}$$

其估计误差 $\tilde{\mathbf{x}} = \mathbf{x} - \hat{\mathbf{x}}$ 满足

$$\dot{\tilde{\mathbf{x}}} = A\tilde{\mathbf{x}}$$

若 A 本身不稳定，则估计误差发散——这就是开环观测器的根本缺陷。

闭环观测器（Luenberger 观测器）引入了输出估计误差的反馈修正项：

$$\dot{\hat{\mathbf{x}}} = A\hat{\mathbf{x}} + B\mathbf{u} + L(\mathbf{y} - C\hat{\mathbf{x}}) \quad (9.4)$$

其中 $L \in \mathbb{R}^{n \times p}$ 是观测器增益矩阵。估计误差动态为

$$\dot{\tilde{\mathbf{x}}} = (A - LC)\tilde{\mathbf{x}}$$

只要通过设计 L 使 $(A - LC)$ 的特征值具有充分负的实部，估计误差将渐近收敛到零。

观测器极点配置（对偶原理）

系统 (A, C) 能观的充分必要条件是：存在观测器增益 L 使得 $A - LC$ 具有任意指定的特征值。这等价于对偶系统 $(A^\top, C^\top)$ 的能控性与极点配置问题。

证明. 注意到

$$\lambda(A - LC) = \lambda((A - LC)^\top) = \lambda(A^\top - C^\top L^\top)$$

将 $(A^\top, C^\top)$ 看作状态空间的“系统”，则 $L^\top$ 就扮演了“反馈增益”的角色。根据极点配置定理，存在 $L^\top$ 使得 $A^\top - C^\top L^\top$ 具有任意特征值的充要条件是 $(A^\top, C^\top)$ 能控，即 (A, C) 能观。因此 L 可以通过对偶系统的 Ackermann 公式或 `place` 命令计算。 $\square$

9.4.2 观测器增益的设计

对偶系统的 Ackermann 公式：对于单输出系统 ($p = 1$)，若期望观测器特征多项式为 $\beta_d(s) = s^n + \beta_{n-1}s^{n-1} + \dots + \beta_0$ ，则

$$L^\top = [0 \ 0 \ \dots \ 1] \mathcal{O}^{-1} \beta_d(A^\top)$$

其中 $\mathcal{O}$ 为能观性矩阵 $[C^\top, A^\top C^\top, \dots, (A^\top)^{n-1} C^\top]^\top$ 。

观测器极点的选择原则：

1. 观测器极点应比控制器极点具有更负的实部（通常快 3-5 倍），以保证估计误差的衰减远快于系统动态；
2. 但若观测器极点过于远离虚轴，观测器增益会很大，导致噪声放大严重，形成“噪声峰值”效应；
3. 工程实践中需在估计速度与噪声敏感性之间进行折中。

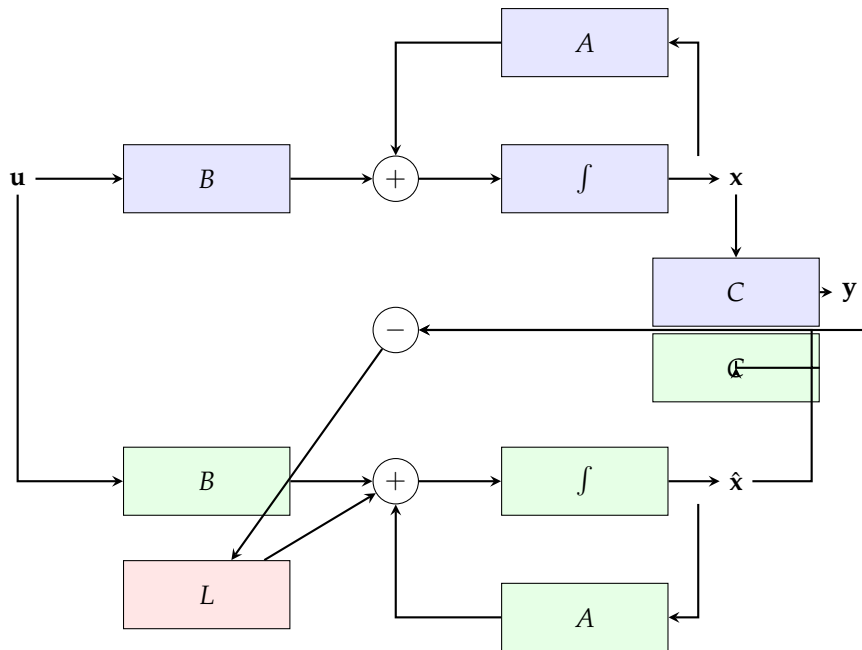


图 9.2: Luenberger 状态观测器结构。观测器复制了被控对象的结构，并通过估计误差 $y - C\hat{x}$ 经增益 L 进行反馈修正。

§9.5 分离原理

9.5.1 控制器与观测器的独立设计

分离原理 (Separation Principle) 是现代控制理论中最优美的结果之一，它表明：状态反馈控制器和状态观测器可以独立设计，组合后整个闭环系统的极点集合恰好是控制器极点与观测器极点的并集。

考虑基于观测器的状态反馈：

$$\mathbf{u} = -K\hat{\mathbf{x}} + \mathbf{v}$$

其中 $\hat{\mathbf{x}}$ 由 Luenberger 观测器提供。闭环增广系统的状态为 $[\mathbf{x}^\top, \tilde{\mathbf{x}}^\top]^\top$ ，其中 $\tilde{\mathbf{x}} = \mathbf{x} - \hat{\mathbf{x}}$ 。状态空间方程为：

$$\frac{d}{dt} \begin{bmatrix} \mathbf{x} \\ \tilde{\mathbf{x}} \end{bmatrix} = \begin{bmatrix} A - BK & BK \\ 0 & A - LC \end{bmatrix} \begin{bmatrix} \mathbf{x} \\ \tilde{\mathbf{x}} \end{bmatrix} + \begin{bmatrix} B \\ 0 \end{bmatrix} \mathbf{v} \quad (9.5)$$

分离原理

增广系统的特征值为

$$\lambda \left(\begin{bmatrix} A - BK & BK \\ 0 & A - LC \end{bmatrix} \right) = \lambda(A - BK) \cup \lambda(A - LC)$$

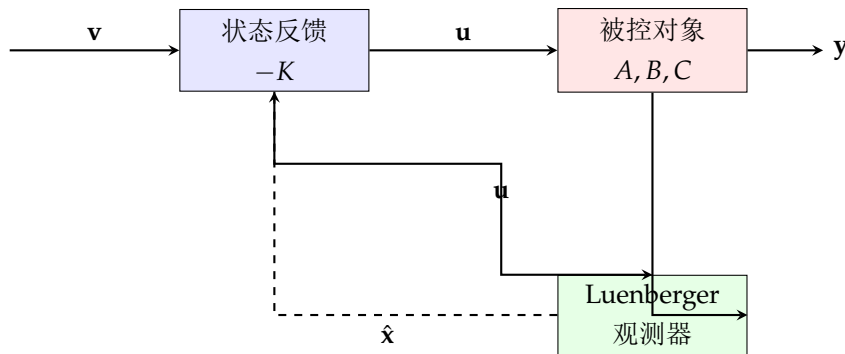
即闭环系统的极点恰好是状态反馈极点（由 K 决定）与观测器极点（由 L 决定）的并集。

证明. 由于增广系统矩阵是上三角块矩阵，其特征多项式为

$$\det \begin{bmatrix} sI - (A - BK) & -BK \\ 0 & sI - (A - LC) \end{bmatrix} = \det(sI - (A - BK)) \cdot \det(sI - (A - LC))$$

□

注. 分离原理的工程意义极为深远：控制器增益 K 的设计可以假定全部状态可测（按理想情况配置极点），观测器增益 L 的设计可以独立进行（按误差衰减速度要求配置），两者互不干扰。这极大地简化了实际控制系统的设计流程。



分离原理： K 和 L 可独立设计

图 9.3: 分离原理示意图。控制器增益 K 和观测器增益 L 可以独立设计： K 假设全状态可测，按理想极点配置； L 按误差衰减速度设计。

9.5.2 观测器增益的极点配置（对偶设计）

观测器增益 L 的设计可通过对偶系统的 Ackermann 公式或直接利用极点配置算法完成。对于单输出系统，设期望的观测器特征多项式为

$$\beta_d(s) = (s - \mu_1)(s - \mu_2) \cdots (s - \mu_n) = s^n + \beta_{n-1}s^{n-1} + \cdots + \beta_0$$

其中 μ_i 是期望的观测器极点（应远大于主导控制器极点的实部）。

利用对偶性，将原系统 (A, C) 的对偶系统 $(A^\top, C^\top)$ 视为“需配置极点的系统”，利用 Ackermann 公式计算：

$$L^\top = e_n^\top \mathcal{O}_d^{-1} \beta_d(A^\top)$$

其中 $\mathcal{O}_d = [C^\top, A^\top C^\top, \dots, (A^\top)^{n-1} C^\top]$ 。

【例题 9.2】(倒立摆的状态观测器) 考虑倒立摆的线性化模型：

$$A = \begin{bmatrix} 0 & 1 \\ \frac{g}{\ell} & 0 \end{bmatrix}, \quad C = \begin{bmatrix} 1 & 0 \end{bmatrix}$$

其中 $g = 9.81$, $\ell = 0.5$ 。设计观测器使估计误差衰减的极点位于 $\mu_{1,2} = -6 \pm j0$ 。

期望特征多项式为 $\beta_d(s) = (s + 6)^2 = s^2 + 12s + 36$ 。利用对偶 Ackermann 公式可得观测器增益 L 。

§9.6 基于观测器的输出反馈

9.6.1 完整结构

基于观测器的输出反馈控制系统由三部分组成：被控对象、状态观测器和状态反馈控制器。其数学描述为：

被控对象：

$$\dot{\mathbf{x}} = A\mathbf{x} + B\mathbf{u}, \quad \mathbf{y} = C\mathbf{x}$$

Luenberger 观测器：

$$\dot{\hat{\mathbf{x}}} = A\hat{\mathbf{x}} + B\mathbf{u} + L(\mathbf{y} - C\hat{\mathbf{x}})$$

控制律：

$$\mathbf{u} = -K\hat{\mathbf{x}} + \mathbf{v}$$

综合以上方程，可得从参考输入 $\mathbf{v}$ 到输出 $\mathbf{y}$ 的闭环传递函数。值得注意的重要事实是：

传递函数的等价性

基于观测器的输出反馈控制系统的传递函数（从 $\mathbf{v}$ 到 $\mathbf{y}$ ）等于直接状态反馈控制系统的传递函数。即观测器的引入不改变输入-输出关系。

证明. 计算闭环传递函数。观测器方程可写为

$$\dot{\hat{\mathbf{x}}} = (A - BK - LC)\hat{\mathbf{x}} + L\mathbf{y} + B\mathbf{v}$$

经过拉普拉斯变换并消去 $\hat{\mathbf{x}}$ ，可以证明 $\mathbf{Y}(s) = C(sI - A + BK)^{-1}B\mathbf{V}(s)$ ，与直接状态反馈一致。 $\square$

这一性质有两层含义：理论上它体现了分离原理的优雅；实践上它意味着观测器动态不影响从参考输入到输出的稳态增益和传递零点。

9.6.2 传递函数的实现——二自由度结构

基于观测器的输出反馈可以重新组织为一种“二自由度”结构。定义控制器传递函数 $K(s)$ ：

$$K(s) = K(sI - A + BK + LC)^{-1}L$$

则控制信号为

$$\mathbf{U}(s) = K(s)\mathbf{Y}(s) + \mathbf{V}(s)$$

这意味着整个控制系统等价于一个动态输出反馈控制器。 $K(s)$ 被称为基于观测器的控制器。

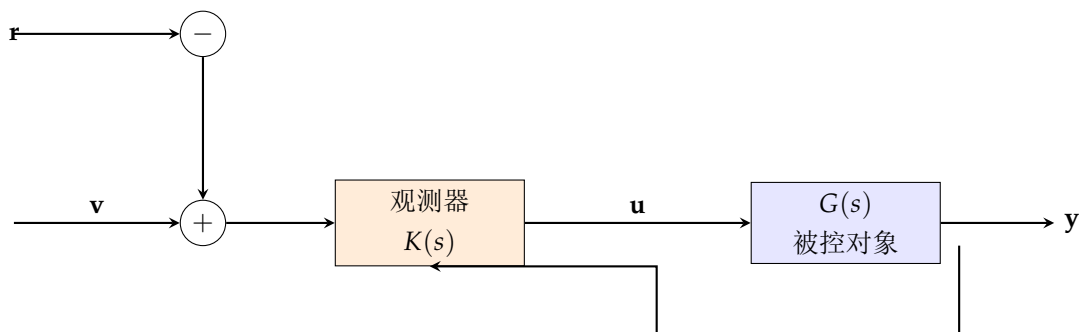


图 9.4: 基于观测器的输出反馈控制系统的等效二自由度结构。 $K(s)$ 是基于观测器的动态控制器，实现了参考输入 $\mathbf{r}$ （前馈）和输出 $\mathbf{y}$ （反馈）的双通道处理。

§9.7

MATLAB 与 Python 实现

9.7.1 MATLAB 实现

MATLAB 提供了 `place` 和 `acker` 两个命令用于极点配置。`place` 基于特征结构配置的数值算法，适用于多输入系统和病态系统；`acker` 是 Ackermann 公式的实现，仅适用于单输入系统。

MATLAB 极点配置示例

```
1 \begin{lstlisting}[language=Matlab]
2 % 系统定义
3 A = [0 1 0; 0 0 1; -2 -3 -5];
4 B = [0; 0; 1];
5 C = [1 0 0];
6 D = 0;
7
8 % 期望极点
9 p_des = [-3+4j, -3-4j, -10];
10
11 % 验证能控性
12 Co = ctrb(A, B);
13 rank_Co = rank(Co); % 应为3
14
15 % 状态反馈增益（多输入系统推荐place）
16 K = place(A, B, p_des);
17
18 % 验证闭环极点
19 eig(A - B*K)
20
21 % 观测器设计（对偶方法）
22 p_obs = [-15, -15, -15]; % 观测器极点（快5倍）
23 L = place(A', C', p_obs)';
24 eig(A - L*C)
25 \end{lstlisting}
```

9.7.2 Python 实现

在 Python 中，可利用 `scipy.signal` 的 `place_poles` 或 `control` 库实现极点配置。

Python 极点配置与观测器设计

```

1 \begin{lstlisting}[language=Python]
2 import numpy as np
3 from scipy import linalg, signal
4
5 # 系统矩阵
6 A = np.array([[0, 1, 0],
7               [0, 0, 1],
8               [-2, -3, -5]])
9 B = np.array([[0], [0], [1]])
10 C = np.array([[1, 0, 0]])
11
12 # 期望极点
13 p_des = np.array([-3+4j, -3-4j, -10])
14
15 # 方法一: scipy.signal.place_poles
16 result = signal.place_poles(A, B, p_des)
17 K_scipy = result.gain_matrix
18 print(f"K (scipy) = {K_scipy}")
19
20 # 方法二: Ackermann公式 (手动实现)
21 def ackermann(A, B, p_des):
22     n = A.shape[0]
23     poly = np.poly(p_des) # 期望特征多项式的系数
24     alpha_dA = np.zeros_like(A)
25     for i, coeff in enumerate(poly):
26         alpha_dA += coeff * np.linalg.matrix_power(A, n - i)
27     C_ctrb = np.hstack([np.linalg.matrix_power(A, i) @ B
28                         for i in range(n)])
29     K = (np.linalg.inv(C_ctrb)[-1, :]) @ alpha_dA
30     return K.reshape(1, -1)
31
32 K_acker = ackermann(A, B, p_des)
33 print(f"\nK (Ackermann) = {K_acker}")
34
35 # 观测器设计
36 p_obs = np.array([-15, -15, -15])
37 L = signal.place_poles(A.T, C.T, p_obs).gain_matrix.T
38 print(f"\nL = \n{L}")
39
40 # 验证闭环极点
41 print(f"\n控制器极点: {np.linalg.eigvals(A - B @ K_scipy)}")
42 print(f"观测器极点: {np.linalg.eigvals(A - L @ C)}")
43 \end{lstlisting}

```

Python 基于观测器的闭环仿真

```

1 \begin{lstlisting}[language=Python]
2 import numpy as np
3 from scipy.integrate import solve_ivp
4 import matplotlib.pyplot as plt
5
6 # 系统参数
7 A = np.array([[0, 1], [9.81/0.5, 0]]) # 倒立摆
8 B = np.array([[0], [1/0.5]])
9 C = np.array([[1, 0]])
10
11 # 控制器设计
12 p_ctrl = np.array([-3+4j, -3-4j])
13 K, _, _ = signal.place_poles(A, B, p_ctrl).gain_matrix, None, None
14
15 # 观测器设计
16 p_obs = np.array([-15, -15])
17 L = signal.place_poles(A.T, C.T, p_obs).gain_matrix.T
18
19 # 增广系统
20 n = A.shape[0]
21 A_aug = np.block([
22     [A, -B @ K],
23     [L @ C, A - B @ K - L @ C]
24 ])
25 B_aug = np.block([[B], [B]])
26
27 # 仿真
28 def sys_dyn(t, z):
29     return A_aug @ z + B_aug.flatten() * np.sin(t) # 正弦参考
30
31 t_span = (0, 10)
32 z0 = np.array([0.1, 0, 0, 0])
33 sol = solve_ivp(sys_dyn, t_span, z0, max_step=0.01)
34
35 x = sol.y[:2]
36 xhat = sol.y[2:]
37
38 fig, axes = plt.subplots(2, 1)
39 axes[0].plot(sol.t, x[0], 'b', label=r'$x_1$ (true)')
40 axes[0].plot(sol.t, xhat[0], 'r--', label=r'$\hat{x}_1$ (est)')
41 axes[0].legend()
42 axes[0].set_ylabel('Position')
43 axes[1].plot(sol.t, x[1], 'b', label=r'$x_2$ (true)')
44 axes[1].plot(sol.t, xhat[1], 'r--', label=r'$\hat{x}_2$ (est)')
45 axes[1].legend()
46 axes[1].set_ylabel('Velocity')
47 axes[1].set_xlabel('Time [s]')
48 plt.show()
49 \end{lstlisting}

```

注. Python 的 `control` 库也提供了 `place` 和 `acker` 的直接调用接口, 需要安装 `python-control` 包。上述代码使用了 `scipy.signal.place_poles` 作为替代方案, 该方法基于 Kautsky-Nichols-Van Dooren 算法, 具有更好的数值稳定性。

§ 9.8

小结

本章从状态反馈的基本概念出发, 系统阐述了极点配置的理论基础与实现方法。核心结论包括:

1. **极点配置定理**: 能控性是任意配置闭环极点的充要条件, 为状态反馈设计提供了理论保障;
2. **Ackermann 公式**: 给出了单输入系统反馈增益的解析表达, 具有重要的理论意义;
3. **对偶原理**: 将观测器设计问题转化为对偶系统的极点配置问题, 统一了控制器和观测器的方法论;
4. **分离原理**: 控制器和观测器可独立设计, 组合后整个系统的极点集为两者的并集, 大大简化了工程实现。

极点配置方法为线性系统控制设计提供了完整的理论框架。但在实际应用中, 如何兼顾性能指标 (如控制能量、鲁棒性等) 仍是重要课题。下一章将介绍线性二次型最优控制, 它在极点配置的基础上进一步引入了性能优化的思想。

线性二次型最优控制

前面章节介绍的极点配置方法解决了“将极点配置到期望位置”的问题,但如何选择“最优的”极点位置?当系统有多个控制输入时,如何权衡控制性能和控制能量?线性二次型最优控制(LQR/LQG)给出了系统的答案:通过引入二次型性能指标,将控制问题转化为数学优化问题,用 Riccati 方程给出最优解。本章全面介绍 LQR 理论、Kalman 滤波、LQG 分离定理及其扩展 EKF,并结合 Python 实现帮助读者掌握最优控制的建模与求解。

§ 10.1

最优控制问题的提法

10.1.1 性能指标（代价函数）

控制系统的“最优性”需要量化的评判标准,这就是性能指标(Performance Index)或称代价函数(Cost Function)。一般形式的最优控制问题是:寻找控制律 $\mathbf{u}^*(\cdot)$ 使以下性能指标最小化:

$$J = \phi(\mathbf{x}(t_f), t_f) + \int_{t_0}^{t_f} L(\mathbf{x}(t), \mathbf{u}(t), t) dt$$

其中 ϕ 是终端代价, L 是运行代价。

对于线性二次型问题,取无限时域二次型性能指标:

$$J = \int_0^{\infty} (\mathbf{x}^T(t) \mathbf{Q} \mathbf{x}(t) + \mathbf{u}^T(t) \mathbf{R} \mathbf{u}(t)) dt \quad (10.1)$$

其中 $\mathbf{Q} \in \mathbb{R}^{n \times n}$ 是状态权重矩阵(半正定, $\mathbf{Q} \succeq 0$), $\mathbf{R} \in \mathbb{R}^{m \times m}$ 是控制权重矩阵(正定, $\mathbf{R} \succ 0$)。

10.1.2 $\mathbf{Q}$ 和 $\mathbf{R}$ 的物理意义

权重矩阵的选取准则

1. 状态权重矩阵 $\mathbf{Q}$: $\mathbf{Q}$ 的元素 q_{ii} 反映了对状态分量 x_i 偏离期望值的惩罚程度。 q_{ii} 越大,对应的状态分量收敛越快,但可能需要更大的控制能量;
2. 控制权重矩阵 $\mathbf{R}$: $\mathbf{R}$ 的元素 r_{jj} 反映了对控制信号 u_j 幅值的惩罚程度。 r_{jj} 越大,控制行为越“保守”(响应慢、节省能量); r_{jj} 越小,控制越“激进”(响应快、耗费能量);
3. 交叉项: 性能指标 $\mathbf{x}^T \mathbf{Q} \mathbf{x} + \mathbf{u}^T \mathbf{R} \mathbf{u}$ 中两项的相对大小决定了性能最优性与控制经济性之间的权衡关系。

注. 在实际调参中，通常先将 Q 和 R 取为对角矩阵，然后根据闭环响应逐步调整。一种常用的初始化策略是 Bryson 法则：取 $q_{ii} = 1/(x_i^{\max})^2$ ， $r_{jj} = 1/(u_j^{\max})^2$ ，其中上标 $\max$ 表示对应量的最大允许值。

10.1.3 最优控制问题的 Hamiltonian

引入协态向量 $\lambda(t) \in \mathbb{R}^n$ ，定义 Hamiltonian 函数

$$\mathcal{H}(\mathbf{x}, \mathbf{u}, \lambda) = \mathbf{x}^\top Q \mathbf{x} + \mathbf{u}^\top R \mathbf{u} + \lambda^\top (A \mathbf{x} + B \mathbf{u})$$

Pontryagin 极小值原理指出，最优控制 $\mathbf{u}^*$ 和最优轨线 $\mathbf{x}^*$ 必须满足：

$$\dot{\mathbf{x}}^* = \nabla_{\lambda} \mathcal{H} = A \mathbf{x}^* + B \mathbf{u}^* \quad (10.2)$$

$$-\dot{\lambda} = \nabla_{\mathbf{x}} \mathcal{H} = 2Q \mathbf{x}^* + A^\top \lambda \quad (10.3)$$

$$0 = \nabla_{\mathbf{u}} \mathcal{H} = 2R \mathbf{u}^* + B^\top \lambda \quad (10.4)$$

从式 (10.4) 可得最优控制的表达式：

$$\mathbf{u}^* = -\frac{1}{2} R^{-1} B^\top \lambda$$

§ 10.2 连续时间 LQR

10.2.1 Riccati 微分方程与代数方程

对于无限时域问题，假设协态向量与状态向量呈线性关系：

$$\lambda = 2P\mathbf{x}$$

其中 $P \in \mathbb{R}^{n \times n}$ 为对称正定矩阵（待求）。代入最优控制表达得

$$\mathbf{u}^* = -R^{-1} B^\top P \mathbf{x} = -K \mathbf{x}, \quad K = R^{-1} B^\top P$$

将协态和状态方程结合，并注意到在稳态时 $\dot{P} = 0$ ，得到连续时间代数 Riccati 方程（CARE）：

$$A^\top P + PA - PBR^{-1}B^\top P + Q = 0 \quad (10.5)$$

连续时间 LQR 的稳定性

若 (A, B) 能稳且 $(A, Q^{1/2})$ 能检测，则 CARE 存在唯一对称正定解 P^* ，并且闭环系统

$$\dot{\mathbf{x}} = (A - BR^{-1}B^\top P^*) \mathbf{x}$$

是渐近稳定的，即 $A - BK$ 的所有特征值具有负实部。

10.2.2 最优增益的推导

从 Riccati 方程求解得到 P 后，最优状态反馈增益矩阵为：

$$K = R^{-1} B^\top P \quad (10.6)$$

最优性能指标的值为

$$J^* = \mathbf{x}_0^\top P \mathbf{x}_0$$

其中 $\mathbf{x}_0$ 是初始状态。这一结果说明 P 矩阵的每一个元素都直接关系到最优性能的取值。

LQR 的频域特性——回路传递恢复

LQR 设计具有优良的鲁棒性：在单输入系统中，状态反馈 LQR 控制器具有至少 60° 的相位裕度和 -6 到 $+\infty$ dB 的增益裕度。但对于基于观测器的 LQR，此鲁棒性可能会丧失。

10.2.3 加权矩阵的选择与极点移动趋势

通过让 $R \rightarrow 0$ （控制代价极低），部分闭环极点趋向于系统的不变零点（如果它们是稳定的）或其镜像（如果它们是不稳定的）；通过让 $R \rightarrow \infty$ （控制代价极高），部分闭环极点趋向于开环的稳定极点。这种渐近行为被称为“廉价格控制”和“贵价格控制”分析，为加权矩阵的选择提供了直觉指导。

【例题 10.1】（一阶系统的 LQR 设计） 考虑 $A = 1, B = 1$ （不稳定一阶系统）， $Q = 1, R = \rho$ 。Riccati 方程为

$$1 \cdot P + P \cdot 1 - P \cdot 1 \cdot \frac{1}{\rho} \cdot 1 \cdot P + 1 = 0$$

整理得 $P^2 - 2\rho P - \rho = 0$ ，解得

$$P = \rho + \sqrt{\rho^2 + \rho}$$

最优增益 $K = P/\rho = 1 + \sqrt{1 + 1/\rho}$ 。闭环极点 $A - BK = -\sqrt{1 + 1/\rho} < 0$ ，稳定。当 $\rho \rightarrow 0$ 时 $K \rightarrow \infty$ （极快的响应）；当 $\rho \rightarrow \infty$ 时 $K \rightarrow 2$ （适中的响应）。

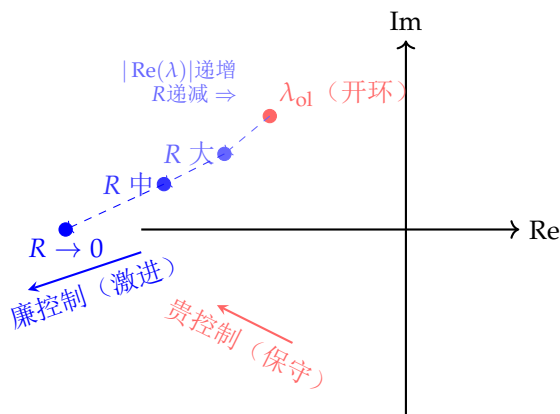


图 10.1: LQR 加权矩阵 R 对闭环极点位置的影响。 $R \rightarrow \infty$ （高控制代价）时极点靠近开环稳定极点， $R \rightarrow 0$ （低控制代价）时极点趋向于更负的实部（部分趋向不稳定零点的镜像）。

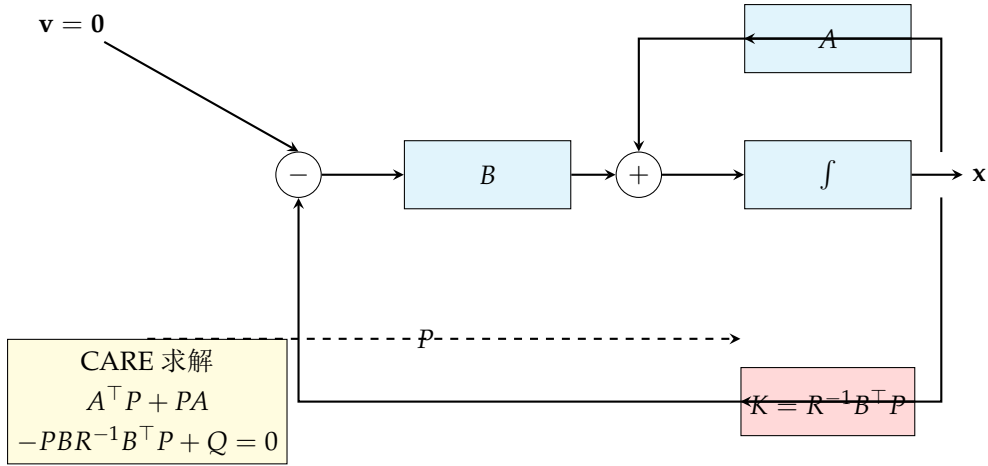


图 10.2: LQR 最优状态反馈结构。Riccati 方程的解 P 赋予最优增益 K , 使得二次型性能指标最小化。

§ 10.3

离散时间 LQR 与 LQG 问题

10.3.1 离散时间 LQR

对于离散线性系统 $\mathbf{x}_{k+1} = A_d \mathbf{x}_k + B_d \mathbf{u}_k$, 二次型性能指标为

$$J = \sum_{k=0}^{\infty} (\mathbf{x}_k^\top Q \mathbf{x}_k + \mathbf{u}_k^\top R \mathbf{u}_k)$$

离散时间代数 Riccati 方程 (DARE) 为

$$P = A_d^\top P A_d - A_d^\top P B_d (R + B_d^\top P B_d)^{-1} B_d^\top P A_d + Q \quad (10.7)$$

最优反馈增益 $K = (R + B_d^\top P B_d)^{-1} B_d^\top P A_d$ 。

10.3.2 LQG 问题的提法

在 LQR 框架中, 假设全部状态可测。当只有噪声污染的输出可测时, 问题转化为 Linear Quadratic Gaussian (LQG) 问题:

$$\dot{\mathbf{x}} = A\mathbf{x} + B\mathbf{u} + \mathbf{w}, \quad \mathbf{y} = C\mathbf{x} + \mathbf{v}$$

其中 $\mathbf{w} \sim \mathcal{N}(0, W)$ 是过程噪声, $\mathbf{v} \sim \mathcal{N}(0, V)$ 是测量噪声, 两者互不相关。性能指标取为随机期望:

$$J = \mathbb{E} \left[\int_0^\infty (\mathbf{x}^\top Q \mathbf{x} + \mathbf{u}^\top R \mathbf{u}) dt \right]$$

LQG 分离定理

LQG 问题的最优解由两部分独立组成:

1. **Kalman 滤波器**: 根据含噪声测量 $\mathbf{y}$ 给出状态的最优估计 $\hat{\mathbf{x}}$;
2. **LQR 控制器**: 将 Kalman 滤波器的状态估计 $\hat{\mathbf{x}}$ 代入 LQR 控制律 $\mathbf{u} = -K\hat{\mathbf{x}}$ 。

两部分独立设计: 滤波器的增益取决于噪声协方差 W 和 V , 控制器的增益取决于性能加权矩阵 Q 和 R 。

§ 10.4 Kalman 滤波器

10.4.1 问题描述与推导思路

Kalman 滤波器是线性高斯系统的最小方差状态估计器。考虑离散时间系统：

$$\mathbf{x}_{k+1} = A_d \mathbf{x}_k + B_d \mathbf{u}_k + \mathbf{w}_k, \quad \mathbf{y}_k = C \mathbf{x}_k + \mathbf{v}_k$$

其中 $\mathbf{w}_k \sim \mathcal{N}(0, W)$, $\mathbf{v}_k \sim \mathcal{N}(0, V)$, 且 $\mathbb{E}[\mathbf{w}_k \mathbf{v}_j^\top] = 0$ 。

10.4.2 离散 Kalman 滤波五大公式

Kalman 滤波按“预测-更新”循环递推，每个时间步由以下五组公式构成：

离散时间 Kalman 滤波器

1. 状态预测：

$$\hat{\mathbf{x}}_{k|k-1} = A_d \hat{\mathbf{x}}_{k-1|k-1} + B_d \mathbf{u}_{k-1}$$

2. 误差协方差预测：

$$P_{k|k-1} = A_d P_{k-1|k-1} A_d^\top + W$$

3. Kalman 增益：

$$K_k = P_{k|k-1} C^\top (C P_{k|k-1} C^\top + V)^{-1}$$

4. 状态更新：

$$\hat{\mathbf{x}}_{k|k} = \hat{\mathbf{x}}_{k|k-1} + K_k (\mathbf{y}_k - C \hat{\mathbf{x}}_{k|k-1})$$

5. 误差协方差更新：

$$P_{k|k} = (I - K_k C) P_{k|k-1}$$

注. 符号约定： $\hat{\mathbf{x}}_{k|k-1}$ 表示基于到 $k-1$ 时刻的测量值对 k 时刻状态的先验估计， $\hat{\mathbf{x}}_{k|k}$ 表示获得第 k 个测量值后的后验估计。 $P_{k|k-1}$ 和 $P_{k|k}$ 分别是相应的误差协方差矩阵。Kalman 增益 K_k 在最小化后验协方差矩阵的迹的意义下是最优的。

Kalman 增益的推导概要. 定义后验估计误差 $\tilde{\mathbf{x}}_{k|k} = \mathbf{x}_k - \hat{\mathbf{x}}_{k|k}$ ，其后验协方差为

$$P_{k|k} = \mathbb{E}[\tilde{\mathbf{x}}_{k|k} \tilde{\mathbf{x}}_{k|k}^\top]$$

代入更新方程，得到

$$P_{k|k} = (I - K_k C) P_{k|k-1} (I - K_k C)^\top + K_k V K_k^\top$$

对 K_k 求偏导并令 $\partial \text{tr}(P_{k|k}) / \partial K_k = 0$ ，解出最优增益表达式。 □

10.4.3 Kalman 滤波的稳定性

对于线性时不变系统，若 (A_d, C) 能检测且 $(A_d, W^{1/2})$ 能稳，则 Kalman 滤波的估计误差协方差收敛到一个稳态值 P_∞ ，它是离散时间代数 Riccati 方程

$$P = A_d P A_d^\top - A_d P C^\top (C P C^\top + V)^{-1} C P A_d^\top + W$$

的对称正定解。滤波器的极点由 $A_d - K_\infty C$ 决定。

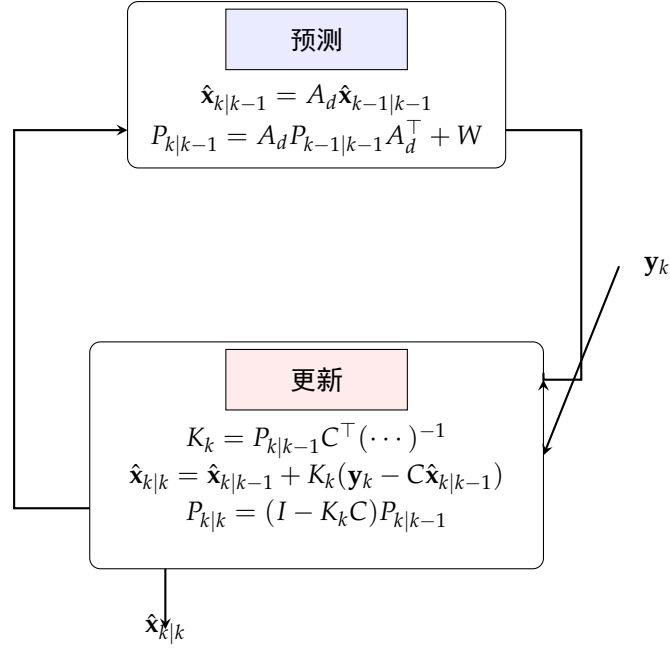


图 10.3: Kalman 滤波器的预测-更新递推流程。每个时间步先执行状态和协方差的预测，再利用测量值进行修正更新。

10.4.4 连续时间 Kalman-Bucy 滤波器

对于连续时间系统，等价的结果是 Kalman-Bucy 滤波器：

$$\dot{\hat{\mathbf{x}}} = A\hat{\mathbf{x}} + B\mathbf{u} + L(\mathbf{y} - C\hat{\mathbf{x}})$$

其中 $L = PC^T V^{-1}$ ， P 满足连续时间滤波 Riccati 方程

$$AP + PA^T - PC^T V^{-1} CP + W = 0 \quad (10.8)$$

注意此方程与 LQR Riccati 方程（CARE）的对偶关系：将 $A \rightarrow A^T$ ， $B \rightarrow C^T$ ， $Q \rightarrow W$ ， $R \rightarrow V$ ，即可从 CARE 得到滤波 Riccati 方程。

§ 10.5 扩展 Kalman 滤波器

10.5.1 非线性系统的线性化

对于非线性系统

$$\mathbf{x}_{k+1} = \mathbf{f}(\mathbf{x}_k, \mathbf{u}_k) + \mathbf{w}_k, \quad \mathbf{y}_k = \mathbf{h}(\mathbf{x}_k) + \mathbf{v}_k$$

EKF 在每个时间步计算 Jacobian 矩阵（局部线性化）：

$$F_{k-1} = \left. \frac{\partial \mathbf{f}}{\partial \mathbf{x}} \right|_{\hat{\mathbf{x}}_{k-1|k-1}, \mathbf{u}_{k-1}}, \quad H_k = \left. \frac{\partial \mathbf{h}}{\partial \mathbf{x}} \right|_{\hat{\mathbf{x}}_{k|k-1}}$$

然后直接套用线性 Kalman 滤波的预测-更新公式。

扩展 Kalman 滤波器

1. 预测步骤:

$$\hat{\mathbf{x}}_{k|k-1} = \mathbf{f}(\hat{\mathbf{x}}_{k-1|k-1}, \mathbf{u}_{k-1})$$

$$\mathbf{P}_{k|k-1} = \mathbf{F}_{k-1} \mathbf{P}_{k-1|k-1} \mathbf{F}_{k-1}^\top + \mathbf{W}$$

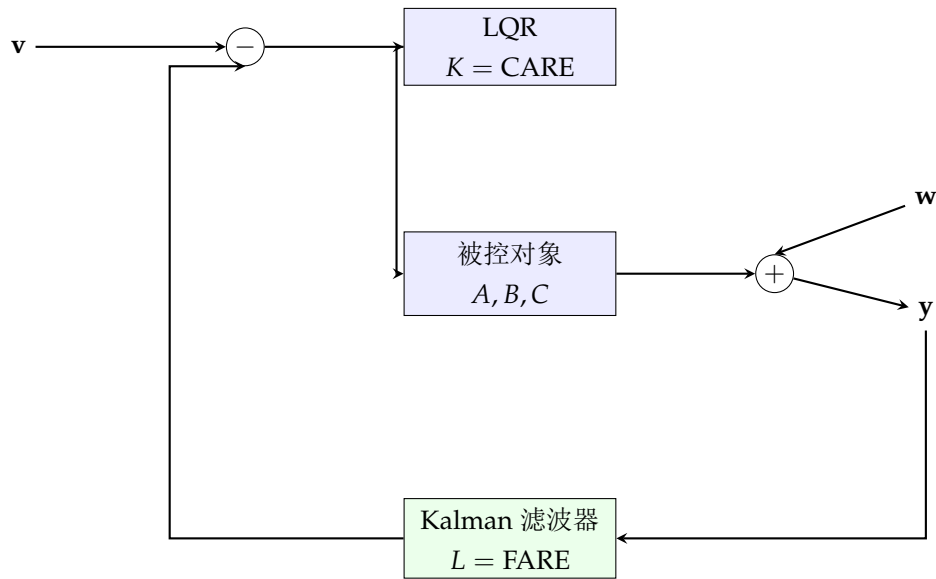
2. 更新步骤:

$$\mathbf{K}_k = \mathbf{P}_{k|k-1} \mathbf{H}_k^\top (\mathbf{H}_k \mathbf{P}_{k|k-1} \mathbf{H}_k^\top + \mathbf{V})^{-1}$$

$$\hat{\mathbf{x}}_{k|k} = \hat{\mathbf{x}}_{k|k-1} + \mathbf{K}_k (\mathbf{y}_k - \mathbf{h}(\hat{\mathbf{x}}_{k|k-1}))$$

$$\mathbf{P}_{k|k} = (\mathbf{I} - \mathbf{K}_k \mathbf{H}_k) \mathbf{P}_{k|k-1}$$

注. EKF 的局限性包括: (1) 当系统非线性较强时, 一阶 Taylor 展开近似不够精确, 可能导致滤波器发散; (2) Jacobian 矩阵的计算在高维情况下计算量大; (3) EKF 给出的状态估计不一定是无偏的, 估计协方差可能严重低估真实误差。作为改进, 无迹 Kalman 滤波 (UKF) 和粒子滤波 (PF) 在强非线性场景下表现更优。



$$\mathbf{LQG} = \mathbf{LQR} + \mathbf{Kalman}$$

图 10.4: LQG 控制结构: LQR 最优控制器与 Kalman 滤波器级联。分离定理保证了二者的独立设计和组合使用。

§ 10.6 Python 实现

10.6.1 LQR 求解与仿真

Python LQR 求解器

```

1 \begin{lstlisting}[language=Python]
2 import numpy as np
3 from scipy import linalg
4 import matplotlib.pyplot as plt
5
6 def lqr(A, B, Q, R):
7     """求解连续时间LQR问题，返回增益矩阵K和解P"""
8     # 求解连续时间代数Riccati方程
9     P = linalg.solve_continuous_are(A, B, Q, R)
10    # 计算最优反馈增益
11    K = linalg.solve(R, B.T @ P)
12    return K, P
13
14 # 例：倒立摆LQR
15 g, L, m = 9.81, 0.5, 0.2
16 A = np.array([[0, 1],
17               [g/L, 0]])
18 B = np.array([[0], [1/(m*L**2)]])
19
20 # 权重矩阵
21 Q = np.diag([100, 10]) # 重视角度和角速度
22 R = np.array([[0.1]]) # 适度惩罚控制
23
24 K, P = lqr(A, B, Q, R)
25 print(f"LQR增益 K = {K}")
26 print(f"Riccati解 P =\n{P}")
27 print(f"闭环极点: {linalg.eigvals(A - B @ K)}")
28
29 # 仿真
30 dt = 0.01
31 t = np.arange(0, 5, dt)
32 n_steps = len(t)
33 x = np.zeros((2, n_steps))
34 x[:, 0] = [0.2, 0] # 初始角度0.2 rad
35
36 for i in range(n_steps - 1):
37     u = -K @ x[:, i]
38     xdot = A @ x[:, i] + B.flatten() * u
39     x[:, i+1] = x[:, i] + xdot * dt
40
41 fig, axes = plt.subplots(2, 1, figsize=(8, 6))
42 axes[0].plot(t, x[0])
43 axes[0].set_ylabel(r'$\theta$ (rad)')
44 axes[0].grid(True)
45 axes[1].plot(t, -K @ x)
46 axes[1].set_ylabel('Control u')
47 axes[1].set_xlabel('Time [s]')
48 axes[1].grid(True)
49 plt.show()

```


10.6.2 Kalman 滤波仿真

Python Kalman 滤波器实现

```

1 \begin{lstlisting}[language=Python]
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 class KalmanFilter:
6     def __init__(self, A, B, C, W, V, x0, P0):
7         self.A = A
8         self.B = B
9         self.C = C
10        self.W = W # 过程噪声协方差
11        self.V = V # 测量噪声协方差
12        self.x = x0 # 状态估计
13        self.P = P0 # 误差协方差
14        self.n = A.shape[0]
15
16        def predict(self, u=0):
17            self.x = self.A @ self.x + self.B @ u
18            self.P = self.A @ self.P @ self.A.T + self.W
19            return self.x
20
21        def update(self, y):
22            S = self.C @ self.P @ self.C.T + self.V
23            K = self.P @ self.C.T @ np.linalg.inv(S)
24            self.x = self.x + K @ (y - self.C @ self.x)
25            self.P = (np.eye(self.n) - K @ self.C) @ self.P
26            return self.x
27
28 # 一维匀加速运动的Kalman滤波
29 dt = 0.1
30 A = np.array([[1, dt, 0.5*dt**2],
31               [0, 1, dt],
32               [0, 0, 1]])
33 B = np.zeros((3, 1))
34 C = np.array([[1, 0, 0]]) # 只测位置
35
36 W = np.diag([0.01, 0.01, 0.01]) # 过程噪声
37 V = np.array([[1.0]]) # 测量噪声
38
39 # 真实轨迹
40 n_steps = 200
41 x_true = np.zeros((3, n_steps))
42 x_true[:, 0] = [0, 1, 0.05]
43 for k in range(n_steps - 1):
44     w = np.random.multivariate_normal([0, 0, 0], W)
45     x_true[:, k+1] = A @ x_true[:, k] + w
46
47 # 测量值
48 y_meas = C @ x_true + np.random.randn(n_steps) * np.sqrt(V[0, 0])
49
50 # Kalman滤波
51 kf = KalmanFilter(A, B, C, W, V, np.zeros(3), np.eye(3))
52 x_est = np.zeros((3, n_steps))
53 for k in range(n_steps):

```


10.6.3 EKF 示例——非线性振荡器

Python 扩展 Kalman 滤波器

```

1 \begin{lstlisting}[language=Python]
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 class ExtendedKalmanFilter:
6     def __init__(self, f, h, F_jac, H_jac, W, V, x0, P0):
7         self.f = f          # 状态转移函数
8         self.h = h          # 观测函数
9         self.F_jac = F_jac  # 状态转移Jacobian
10        self.H_jac = H_jac  # 观测Jacobian
11        self.W = W
12        self.V = V
13        self.x = x0
14        self.P = P0
15        self.n = len(x0)
16
17        def predict(self, u=0):
18            self.x = self.f(self.x, u)
19            F = self.F_jac(self.x, u)
20            self.P = F @ self.P @ F.T + self.W
21            return self.x
22
23        def update(self, y):
24            H = self.H_jac(self.x)
25            S = H @ self.P @ H.T + self.V
26            K = self.P @ H.T @ np.linalg.inv(S)
27            self.x = self.x + K @ (y - self.h(self.x))
28            self.P = (np.eye(self.n) - K @ H) @ self.P
29            return self.x
30
31 # Van der Pol 振荡器
32 mu = 1.0
33 dt = 0.05
34
35 def f(x, u):
36     """x = [x1, x2]^T"""
37     x1, x2 = x[0], x[1]
38     return np.array([x1 + dt * x2,
39                     x2 + dt * (mu * (1 - x1**2) * x2 - x1)])
40
41 def h(x):
42     return np.array([x[0]]) # 只测x1
43
44 def F_jac(x, u):
45     x1, x2 = x[0], x[1]
46     return np.array([
47         [1, dt],
48         [-dt * (2 * mu * x1 * x2 + 1), 1 + dt * mu * (1 - x1**2)]
49     ])
50
51 def H_jac(x):
52     return np.array([[1, 0]])

```


10.6.4 离散 LQR 与 Kalman 结合的 LQG 仿真

Python 离散 LQG 控制器

```

1 \begin{lstlisting}[language=Python]
2 import numpy as np
3 from scipy import linalg
4 import matplotlib.pyplot as plt
5
6 # 离散系统
7 dt = 0.1
8 A_d = np.array([[1, dt], [0, 1]])
9 B_d = np.array([[0.5*dt**2], [dt]])
10 C = np.array([[1, 0]])
11
12 # LQR设计
13 Q = np.diag([10, 1])
14 R = np.array([[0.01]])
15 P_dare = linalg.solve_discrete_are(A_d, B_d, Q, R)
16 K_lqr = linalg.solve(B_d.T @ P_dare @ B_d + R,
17                      B_d.T @ P_dare @ A_d)
18 print(f"离散LQR增益 K = {K_lqr}")
19
20 # Kalman滤波器
21 W = np.diag([0.001, 0.001])
22 V = np.array([[0.1]])
23 S_dare = linalg.solve_discrete_are(A_d.T, C.T, W, V)
24 L_kf = linalg.solve(C @ S_dare @ C.T + V,
25                     C @ S_dare).T
26 print(f"稳态Kalman增益 L =\n{L_kf}")
27
28 # LQG仿真
29 n_steps = 200
30 x_true = np.zeros((2, n_steps))
31 x_true[:, 0] = [1, 0]
32 x_est = np.zeros((2, n_steps))
33 x_est[:, 0] = [0, 0]
34 u = np.zeros(n_steps)
35 y = np.zeros(n_steps)
36
37 for k in range(n_steps - 1):
38     # 控制
39     u[k] = -K_lqr @ x_est[:, k]
40     # 过程噪声
41     w = np.random.multivariate_normal([0, 0], W)
42     # 状态更新
43     x_true[:, k+1] = A_d @ x_true[:, k] + B_d.flatten() * u[k] + w
44     # 测量
45     v = np.random.randn() * np.sqrt(V[0, 0])
46     y[k+1] = C @ x_true[:, k+1] + v
47     # Kalman预测
48     x_pred = A_d @ x_est[:, k] + B_d.flatten() * u[k]
49     # Kalman更新
50     x_est[:, k+1] = x_pred + L_kf.flatten() * (y[k+1] - C @ x_pred)
51
52 fig, axes = plt.subplots(3, 1, figsize=(10, 8))
53 axes[0].plot(x_true[0, :], label='x1')
54 axes[1].plot(x_true[1, :], label='x2')
55 axes[2].plot(y, label='y')
56 axes[0].set_xlabel('Time')
57 axes[1].set_xlabel('Time')
58 axes[2].set_xlabel('Time')
59 axes[0].set_ylabel('x1')
60 axes[1].set_ylabel('x2')
61 axes[2].set_ylabel('y')
62 axes[0].legend()
63 axes[1].legend()
64 axes[2].legend()
65 plt.show()

```

§ 10.7 从 LQR 到 MPC 的联系

LQR 为线性系统的最优控制提供了完整的开/闭环解。然而，实际的物理系统总存在输入和状态的约束（如执行器饱和、安全边界等），LQR 本身无法直接处理约束。模型预测控制（MPC）可以看作是“带约束的递归 LQR”——在每个采样时刻求解一个有限时域的最优控制问题（通常转化为二次规划），只实施第一步控制，然后滚动到下一时刻重新求解。这种“滚动时域”策略使得 MPC 既能像 LQR 一样利用最优化理论，又能灵活处理约束条件。有关 MPC 的详细介绍将在第 12 章展开。

同时，LQR 给出的最优增益 K 与闭环系统的 Riccati 解之间存在密切关系。在鲁棒控制理论中，由 Riccati 方程构造的 LQR 控制器天然具有优良的增益裕度和相位裕度，这一性质引出了回路传递恢复（Loop Transfer Recovery, LTR）设计方法——通过调节 Kalman 滤波器的噪声参数，使基于观测器的 LQG 控制器恢复 LQR 的鲁棒特性。

§ 10.8 小结

本章围绕线性二次型最优控制这条主线，系统介绍了最优性能指标的定义、连续和离散时间代数 Riccati 方程的导出、Kalman 滤波器的推导与实现、以及 LQG 分离定理。核心结论：

1. LQR 通过求解 Riccati 方程给出最优线性状态反馈，闭环系统具有优越的稳定裕度；
2. Kalman 滤波器是最小方差意义下的最优状态估计器，为不可测状态的重构提供了精确算法；
3. LQG 分离定理将最优状态估计与最优控制解耦，是随机最优控制理论的基石；
4. EKF 通过对非线性系统的局部线性化将 Kalman 滤波推广至非线性领域，尽管存在局限性，但在工程实际中仍应用广泛。

最优控制理论的进一步发展催生了鲁棒控制（ H_∞ 控制）、模型预测控制（MPC）和强化学习控制等现代方法，将在后续章节中逐一介绍。

前面各章的研究对象是线性系统，其理论基础建立在叠加原理之上。然而，绝大多数实际物理系统本质上是非线性的——从机械臂的多轴耦合到飞机的高攻角气动特性，从化学反应的指数动力学到电力系统的频率振荡。非线性系统呈现出线性系统不具备的丰富动力学行为：多平衡点、极限环、分岔和混沌。本章系统介绍非线性控制的基本理论和方法，包括 Lyapunov 稳定性理论、滑模控制、反馈线性化、Backstepping 和非线性观测器设计。

§11.1 非线性系统的特性

11.1.1 非线性系统的基本描述

一般形式的非线性系统可写为

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u}, t), \quad \mathbf{y} = \mathbf{h}(\mathbf{x}, \mathbf{u}, t)$$

其中 $\mathbf{f}$ 和 $\mathbf{h}$ 是非线性向量函数。当 $\mathbf{f}$ 不显含时间 t 时，系统称为自治系统：

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x})$$

仿射非线性系统是控制理论中最常研究的一类：

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}) + \sum_{i=1}^m \mathbf{g}_i(\mathbf{x}) u_i = \mathbf{f}(\mathbf{x}) + \mathbf{G}(\mathbf{x}) \mathbf{u} \quad (11.1)$$

其中 $\mathbf{f}(\mathbf{x})$ 是漂移向量场， $\mathbf{g}_i(\mathbf{x})$ 是控制向量场。

11.1.2 叠加原理的失效与多平衡点

线性系统 $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x}$ 的平衡点只有原点（当 $\mathbf{A}$ 非奇异时）。而非线性系统 $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x})$ 可能有多个孤立平衡点，每个平衡点周围的动力学行为可能完全不同。例如，单摆方程

$$\ddot{\theta} + \frac{g}{\ell} \sin \theta = 0$$

具有无穷多个平衡点 $\theta = k\pi$ ($k \in \mathbb{Z}$)：偶数倍 π 处是稳定焦点，奇数倍 π 处是鞍点。

注. 叠加原理的失效意味着：不能将非线性系统的响应分解为不同输入分别响应的叠加，也不能将复杂运动分解为模态的组合。这是非线性控制比线性控制更具挑战性的根本原因。

11.1.3 极限环、混沌与分岔

极限环是非线性系统独有的孤立周期轨道，在线性系统中不可能出现。典型的例子是 Van der Pol 振荡器：

$$\ddot{x} + \mu(x^2 - 1)\dot{x} + x = 0, \quad \mu > 0$$

对于任何非零初始条件，解轨线都收敛到同一个极限环。

混沌表现为对初值的极端敏感性（“蝴蝶效应”）、非周期长期行为和奇怪吸引子（Strange Attractor）。Lorenz 系统是最经典的混沌示例：

$$\begin{aligned}\dot{x} &= \sigma(y - x) \\ \dot{y} &= x(\rho - z) - y \\ \dot{z} &= xy - \beta z\end{aligned}$$

分岔（Bifurcation）是指系统参数穿过临界值时，相图拓扑结构发生的定性改变。常见类型包括鞍结分岔（saddle-node）、Hopf 分岔和叉式分岔（pitchfork）。

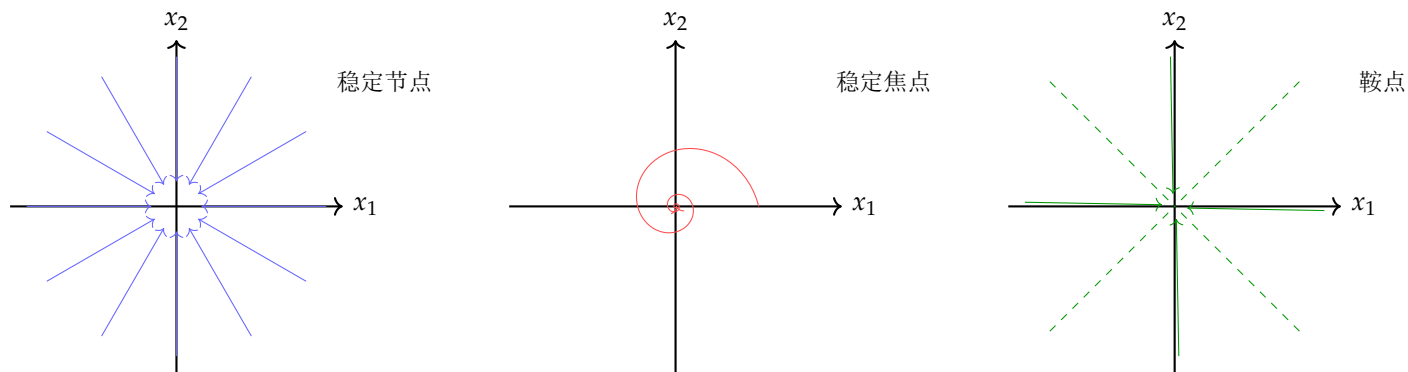


图 11.1: 典型相平面轨迹类型。左：稳定节点（特征值为负实根），中：稳定焦点（特征值为共轭复根），右：鞍点（特征值为一正一负实根）。

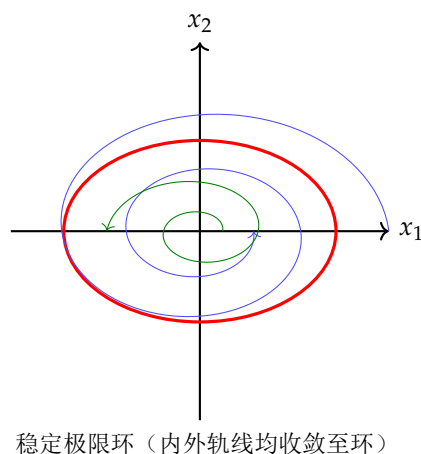


图 11.2: 极限环示意图。外部轨线向内螺旋收敛，内部轨线向外螺旋膨胀，最终都趋向红色椭圆所示的极限环。

§ 11.2 Lyapunov 稳定性理论

11.2.1 自治系统的稳定性概念

考虑自治系统 $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x})$ ，设 $\mathbf{x} = \mathbf{0}$ 是一个平衡点（通过坐标平移总可达到），即 $\mathbf{f}(\mathbf{0}) = \mathbf{0}$ 。

Lyapunov 意义下的稳定性

1. 稳定： $\forall \varepsilon > 0, \exists \delta(\varepsilon) > 0$ 使得 $\|\mathbf{x}(0)\| < \delta \Rightarrow \|\mathbf{x}(t)\| < \varepsilon, \forall t \geq 0$;
2. 渐近稳定：稳定，且 $\lim_{t \rightarrow \infty} \|\mathbf{x}(t)\| = 0$;
3. 指数稳定： $\exists \alpha, \beta > 0$ 使得 $\|\mathbf{x}(t)\| \leq \beta \|\mathbf{x}(0)\| e^{-\alpha t}$;
4. 全局渐近稳定：渐近稳定，且对任意初始条件 $\mathbf{x}(0) \in \mathbb{R}^n$ 均收敛到原点。

注. 与线性系统不同，非线性系统的稳定性往往是局部性质——一个平衡点可能在某些初始条件下稳定，而在另一些初始条件下不稳定。因此非线性稳定性理论中，“吸引域”（Region of Attraction）的概念至关重要。

11.2.2 Lyapunov 直接法（能量法）

Lyapunov 直接法的核心思想是：如果能够找到一个“能量函数” $V(\mathbf{x})$ ，它沿系统轨线的导数始终非正（表示能量不断衰减），那么系统的平衡点就是稳定的。

Lyapunov 稳定性定理

设 $\mathbf{x} = \mathbf{0}$ 是系统 $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x})$ 的一个平衡点。若存在连续可微函数 $V : D \rightarrow \mathbb{R}$ ($D \subset \mathbb{R}^n$ 包含原点) 满足

1. $V(\mathbf{0}) = 0$ ，且 $V(\mathbf{x}) > 0$ 对任意 $\mathbf{x} \in D \setminus \{\mathbf{0}\}$ （正定性）；
2. $\dot{V}(\mathbf{x}) = \nabla V(\mathbf{x}) \cdot \mathbf{f}(\mathbf{x}) \leq 0$ 对任意 $\mathbf{x} \in D$ （半负定性）。

则原点是稳定的。若进一步 $\dot{V}(\mathbf{x}) < 0$ 对任意 $\mathbf{x} \in D \setminus \{\mathbf{0}\}$ ，则原点是渐近稳定的。

Lyapunov 稳定性定理的简要证明思路. 正定性保证 V 的等值面构成一族围绕原点的闭合曲面。 $\dot{V} \leq 0$ 保证轨线不会穿越等值面向外发展——轨线始终停留在初始 V 值对应的等值面包围的区域内。通过选取充分小的初始 V 值，可使轨线始终保持在任意给定的 ε 邻域内。□

11.2.3 Lyapunov 函数的构造方法

构造合适的 Lyapunov 函数是非线性稳定性分析中的主要挑战。常用方法包括：

1. **能量法**：对于物理系统，总能量（动能 + 势能）往往是天然的 Lyapunov 函数候选。例如，对于质量-弹簧-阻尼系统，总机械能 $V = \frac{1}{2}m\dot{x}^2 + \frac{1}{2}kx^2$ 的导数为 $\dot{V} = -c\dot{x}^2 \leq 0$ ；
2. **平方和法**：对于二次型 $V = \mathbf{x}^\top \mathbf{P} \mathbf{x}$ ， $\dot{V} = \mathbf{x}^\top (\mathbf{A}^\top \mathbf{P} + \mathbf{P} \mathbf{A}) \mathbf{x}$ ，这对应于线性情况下的 Lyapunov 方程；
3. **Krasovskii 方法**：取 $V = \mathbf{f}^\top \mathbf{P} \mathbf{f}$ ，适用于某些特殊结构；
4. **反推法（Backstepping）**：对级联系统逐级构造 Lyapunov 函数（参见第 5 节）。

【例题 11.1】(单摆的 Lyapunov 分析) 考虑带阻尼的单摆方程： $\ddot{\theta} + b\dot{\theta} + \frac{g}{\ell} \sin \theta = 0$ ($b > 0$)。取状态变量 $x_1 = \theta$, $x_2 = \dot{\theta}$, 则状态方程：

$$\dot{x}_1 = x_2, \quad \dot{x}_2 = -\frac{g}{\ell} \sin x_1 - bx_2$$

取 Lyapunov 函数为总能量：

$$V = \frac{1}{2}x_2^2 + \frac{g}{\ell}(1 - \cos x_1)$$

求导：

$$\dot{V} = x_2\dot{x}_2 + \frac{g}{\ell} \sin x_1 \cdot \dot{x}_1 = x_2 \left(-\frac{g}{\ell} \sin x_1 - bx_2 \right) + \frac{g}{\ell} \sin x_1 \cdot x_2 = -bx_2^2 \leq 0$$

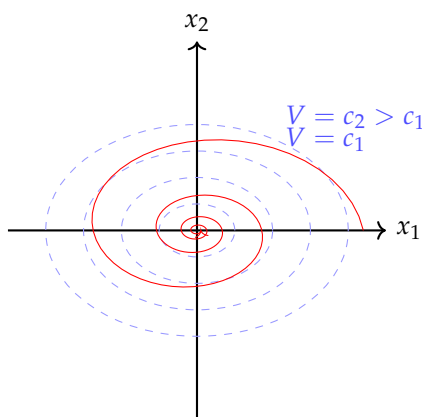
由 Lyapunov 定理，原点 $(0,0)$ 是稳定的。再利用 Lasalle 不变原理可以进一步证明渐近稳定性。

11.2.4 Lasalle 不变原理

Lasalle 不变原理推广了 Lyapunov 定理，使我们即使在 $\dot{V} \leq 0$ (非严格负定) 时也能推断渐近稳定性。

Lasalle 不变原理

设 $\Omega \subset D$ 是一个紧致正不变集 (即系统轨线一旦进入 Ω 就永不离开)。设 $V : D \rightarrow \mathbb{R}$ 连续可微，在 Ω 内满足 $\dot{V} \leq 0$ 。令 $E = \{\mathbf{x} \in \Omega \mid \dot{V}(\mathbf{x}) = 0\}$, M 是 E 中的最大不变集。则当 $t \rightarrow \infty$ 时，任何起始于 Ω 的轨线都趋于 M 。



轨线不断穿越 V 的等值面 (向内的方向), $\dot{V} \leq 0$

图 11.3: Lyapunov 函数的等值面与系统轨线的关系。轨线从外向内穿越递减的 V 等值面，体现了“能量”不断耗散的过程。

§ 11.3 滑模控制

11.3.1 滑模控制的基本原理

滑模控制 (Sliding Mode Control, SMC) 是一种鲁棒非线性控制方法，其核心思路是：在状态空间中设计一个“滑模面” $s(\mathbf{x}) = 0$ ，然后用不连续的控制律将系统状态“驱动”到该滑模面上，并使其沿滑模面滑向平衡点。

对于单输入仿射非线性系统：

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}) + \mathbf{g}(\mathbf{x})u$$

设计滑模面 $s(\mathbf{x})$ ，使 $s = 0$ 是一个 $(n - 1)$ 维超曲面，且在该面上的“滑模动态”是渐近稳定的。

11.3.2 滑模面的设计

常用的滑模面设计方法包括：

(1) 线性滑模面： $s = \mathbf{c}^\top \mathbf{x}$ ，其中 $\mathbf{c} \in \mathbb{R}^n$ 是待设计的常数向量。这种方式等价于将滑模运动配置为 $n - 1$ 维线性系统的稳定运动。对于可化为正则形式的系统， $\mathbf{c}$ 的选择对应着滑模极点配置。

(2) 终端滑模面：引入分数幂项以实现在有限时间内收敛：

$$s = \dot{x} + \beta x^{q/p}, \quad p > q > 0, \quad p, q \text{ 为奇数}$$

(3) 积分滑模面：引入误差的积分项以消除稳态误差：

$$s = \dot{e} + \lambda_1 e + \lambda_2 \int_0^t e(\tau) d\tau$$

11.3.3 趋近律设计

滑模控制的关键是设计控制律使 $s \rightarrow 0$ （到达阶段）并保持 $s = 0$ （滑动阶段）。趋近律 $\dot{s}$ 的设计决定了收敛到滑模面的速度和质量。

常用趋近律

1. 等速趋近律： $\dot{s} = -\eta \operatorname{sgn}(s)$ ， $\eta > 0$ 为趋近速度；
2. 指数趋近律： $\dot{s} = -\eta \operatorname{sgn}(s) - ks$ ， $k, \eta > 0$ 。指数项 $-ks$ 在远离滑模面时提供快速趋近，等速项 $-\eta \operatorname{sgn}(s)$ 保证在 $s = 0$ 附近不会“滑出”；
3. 幂次趋近律： $\dot{s} = -k|s|^\alpha \operatorname{sgn}(s)$ ， $k > 0$ ， $0 < \alpha < 1$ ；
4. 一般趋近律： $\dot{s} = -\eta \operatorname{sgn}(s) - f(s)$ ， $f(0) = 0$ 且 $sf(s) > 0$ 当 $s \neq 0$ 。

控制律的构造基于 Lyapunov 函数 $V = \frac{1}{2}s^2$ ：

$$\dot{V} = s\dot{s} = s(\nabla s \cdot \dot{\mathbf{x}}) = s \left(\frac{\partial s}{\partial \mathbf{x}} \mathbf{f} + \frac{\partial s}{\partial \mathbf{x}} \mathbf{g} u \right)$$

取控制律

$$u = - \left(\frac{\partial s}{\partial \mathbf{x}} \mathbf{g} \right)^{-1} \left(\frac{\partial s}{\partial \mathbf{x}} \mathbf{f} + \eta \operatorname{sgn}(s) \right)$$

则 $\dot{V} = -\eta|s|$ ，保证有限时间内到达滑模面。

11.3.4 抖振问题与边界层方法

理想滑模控制要求控制器在 $s = 0$ 处瞬间切换符号（符号函数 $\operatorname{sgn}(s)$ ），这在物理上不可能实现（执行器带宽有限），直接实施会产生高频振荡——称为抖振（Chattering）。

边界层方法将不连续的 sgn 函数替换为饱和函数 sat 以平滑控制信号：

$$\operatorname{sat}(s/\Phi) = \begin{cases} \operatorname{sgn}(s), & |s| > \Phi \\ s/\Phi, & |s| \leq \Phi \end{cases}$$

其中 $\Phi > 0$ 是边界层厚度。在边界层内使用连续控制消除了抖振，但付出的代价是滑模运动的理想鲁棒性在边界层内有所丧失（系统状态不再严格保持 $s = 0$ ，而是保持在 $|s| \leq \Phi$ 内）。

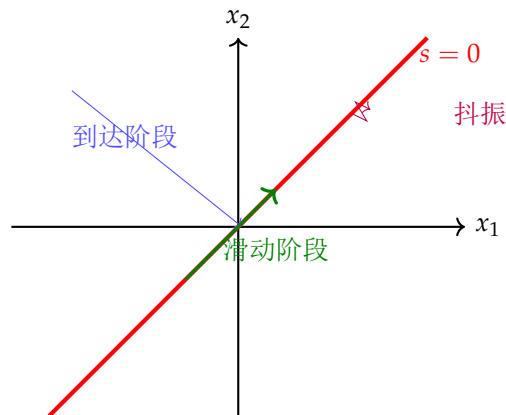


图 11.4: 滑模控制的阶段划分。蓝色轨迹为“到达阶段”，系统状态从初始位置被驱动到滑模面 $s = 0$ 上；绿色轨迹为“滑动阶段”，状态沿滑模面滑向原点。紫色放大部分示意抖振现象。

§ 11.4 反馈线性化

11.4.1 输入-状态线性化

反馈线性化（Feedback Linearization）的核心思想是通过坐标变换和非线性反馈将非线性系统精确地转化为线性系统（而非像 Jacobian 线性化那样只做近似）。考虑仿射非线性系统：

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}) + G(\mathbf{x})\mathbf{u}$$

如果存在微分同胚 $T: \mathbb{R}^n \rightarrow \mathbb{R}^n$ （坐标变换 $\mathbf{z} = T(\mathbf{x})$ ）和状态反馈

$$\mathbf{u} = \boldsymbol{\alpha}(\mathbf{x}) + \boldsymbol{\beta}(\mathbf{x})\mathbf{v}$$

使得在新坐标 $\mathbf{z}$ 下系统变为线性可控形式（Brunovsky 标准型）：

$$\dot{\mathbf{z}} = A_c \mathbf{z} + B_c \mathbf{v}$$

则称该系统可输入-状态线性化。

11.4.2 Lie 导数与 Lie 括号

精确线性化的数学基础是微分几何中的 Lie 导数和 Lie 括号运算。

Lie 导数与 Lie 括号

设 $h: \mathbb{R}^n \rightarrow \mathbb{R}$ 是光滑标量函数, $\mathbf{f}: \mathbb{R}^n \rightarrow \mathbb{R}^n$ 是光滑向量场。 h 沿 $\mathbf{f}$ 的 **Lie 导数** 定义为

$$L_{\mathbf{f}}h(\mathbf{x}) = \frac{\partial h}{\partial \mathbf{x}} \mathbf{f}(\mathbf{x}) = \sum_{i=1}^n \frac{\partial h}{\partial x_i} f_i(\mathbf{x})$$

高阶 Lie 导数迭代定义: $L_{\mathbf{f}}^k h = L_{\mathbf{f}}(L_{\mathbf{f}}^{k-1} h)$ 。

两个向量场 $\mathbf{f}$ 和 $\mathbf{g}$ 的 **Lie 括号** (对易子) 定义为

$$[\mathbf{f}, \mathbf{g}](\mathbf{x}) = \frac{\partial \mathbf{g}}{\partial \mathbf{x}} \mathbf{f}(\mathbf{x}) - \frac{\partial \mathbf{f}}{\partial \mathbf{x}} \mathbf{g}(\mathbf{x})$$

其中 $\frac{\partial \mathbf{g}}{\partial \mathbf{x}}$ 和 $\frac{\partial \mathbf{f}}{\partial \mathbf{x}}$ 分别是向量场 $\mathbf{g}$ 和 $\mathbf{f}$ 的 Jacobian 矩阵。

11.4.3 相对阶与输入-输出线性化

对于单输入单输出 (SISO) 非线性系统:

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}) + \mathbf{g}(\mathbf{x})u, \quad y = h(\mathbf{x})$$

相对阶 r 定义为满足 $L_{\mathbf{g}}L_{\mathbf{f}}^{r-1}h(\mathbf{x}) \neq 0$ 的最小整数。

【例题 11.2】(相对阶的计算) 对于系统 $\ddot{y} + \sin y = u$, 有

$$\dot{x}_1 = x_2 = f_1 + g_1 u$$

$$\dot{x}_2 = -\sin x_1 + u = f_2 + g_2 u$$

$$y = x_1 = h(\mathbf{x})$$

计算: $L_{\mathbf{g}}h = 0$, $L_{\mathbf{g}}L_{\mathbf{f}}h = L_{\mathbf{g}}(x_2) = 1 \neq 0$ 。因此相对阶 $r = 2$ 。

若 $r = n$ (系统的阶数), 则系统可完全的输入-状态线性化, 没有零动态。若 $r < n$, 则存在 $(n - r)$ 维的**零动态** (Zero Dynamics) ——即当输出被强制为零时内部状态的动态行为。系统的内部稳定性取决于零动态的稳定性。

零动态与最小相位性

若系统的零动态是渐近稳定的, 则称该系统为**最小相位系统**。输入-输出线性化控制器可以稳定整个系统当且仅当零动态是渐近稳定的。这与线性系统中“不稳定的零点限制可实现的闭环性能”具有深刻的类比关系。

注. 反馈线性化与经典的 Jacobian 线性化有本质区别: 前者是精确的代数变换, 适用面更广但条件也更苛刻 (需要验证对合性条件); 后者只是局部的一阶近似。在工程实践中, 若精确线性化条件满足, 应当优先使用反馈线性化以获得全局或大范围的有效性。

§ 11.5

Backstepping 设计方法**11.5.1 Backstepping 的基本思想**

Backstepping (反推法) 是一种针对严格反馈 (Strict-Feedback) 系统的递归 Lyapunov 设计方法。其核心思路是将高阶系统分解为一系列级联的一阶子系统, 从最内层的子系统开始, 逐层向外“反推”设计虚拟控制律和 Lyapunov 函数。

11.5.2 严格反馈形式

严格反馈系统的标准形式为：

$$\begin{aligned}\dot{x}_1 &= f_1(x_1) + g_1(x_1)x_2 \\ \dot{x}_2 &= f_2(x_1, x_2) + g_2(x_1, x_2)x_3 \\ &\vdots \\ \dot{x}_{n-1} &= f_{n-1}(x_1, \dots, x_{n-1}) + g_{n-1}(x_1, \dots, x_{n-1})x_n \\ \dot{x}_n &= f_n(x_1, \dots, x_n) + g_n(x_1, \dots, x_n)u\end{aligned}$$

其中 $g_i \neq 0$ (能控性条件)。注意这种结构：每个状态方程只受下一状态的仿射影响，形成了“下三角”结构。

11.5.3 二阶系统的 Backstepping 设计范例

考虑二阶严格反馈系统：

$$\begin{aligned}\dot{x}_1 &= f_1(x_1) + g_1(x_1)x_2 \\ \dot{x}_2 &= f_2(x_1, x_2) + g_2(x_1, x_2)u\end{aligned}$$

Step 1: 将 x_2 视为对子系统 x_1 的“虚拟控制”。选择期望的虚拟控制律 $\alpha(x_1)$ 使得 x_1 子系统稳定。定义跟踪误差 $z_1 = x_1$, $z_2 = x_2 - \alpha(x_1)$ 。

选取 Lyapunov 函数 $V_1 = \frac{1}{2}z_1^2$, 则

$$\dot{V}_1 = z_1(f_1 + g_1(z_2 + \alpha)) = z_1(f_1 + g_1\alpha) + g_1z_1z_2$$

取 $\alpha = -\frac{1}{g_1}(f_1 + k_1z_1)$ ($k_1 > 0$), 则 $\dot{V}_1 = -k_1z_1^2 + g_1z_1z_2$ 。

Step 2: 构造增广的 Lyapunov 函数 $V_2 = V_1 + \frac{1}{2}z_2^2$:

$$\begin{aligned}\dot{V}_2 &= -k_1z_1^2 + g_1z_1z_2 + z_2(f_2 + g_2u - \dot{\alpha}) \\ &= -k_1z_1^2 + z_2(g_1z_1 + f_2 + g_2u - \dot{\alpha})\end{aligned}$$

取控制律

$$u = \frac{1}{g_2}(-g_1z_1 - f_2 + \dot{\alpha} - k_2z_2)$$

则 $\dot{V}_2 = -k_1z_1^2 - k_2z_2^2 < 0$, 系统全局渐近稳定。

注. Backstepping 设计有两个显著优势：(1) 灵活性——在每个“反推”步骤中可以选择任意的 Lyapunov 函数 V_i 和控制 Lyapunov 函数 (CLF), 从而引入对非线性项的系统化补偿；(2) 避免了反馈线性化中“精确消除全部非线性”的要求，只需处理对稳定性不利的非线性项，有利于提高控制器的鲁棒性。

11.5.4 自适应 Backstepping

当系统参数 θ 未知时，将 Backstepping 与参数自适应律结合，在每个反推步骤中同时设计参数估计更新律。典型结构：状态方程为

$$\dot{x}_i = f_i(\bar{x}_i) + \varphi_i^\top(\bar{x}_i)\theta + g_i(\bar{x}_i)x_{i+1}$$

Lyapunov 函数中增加参数估计误差的二次项：

$$V_i = V_{i-1} + \frac{1}{2}z_i^2 + \frac{1}{2}\tilde{\theta}_i^\top \Gamma_i^{-1}\tilde{\theta}_i$$

其中 $\tilde{\theta}_i = \hat{\theta}_i - \theta$ 是参数估计误差, Γ_i 是自适应增益矩阵。在每个步骤中适当选择虚拟控制律和参数更新律，保证 $\dot{V}_i \leq 0$ 。这种“调节函数” (tuning function) 方法确保了最终的参数估计和状态跟踪同时收敛。

§ 11.6 非线性观测器

11.6.1 高增益观测器

对于可化为标准观测形式的非线性系统，可以设计高增益观测器。其基本思想是：利用足够大的观测器增益克服非线性不确定性。以 SISO 系统为例，若系统可写成

$$\begin{aligned}\dot{x}_1 &= x_2 + \varphi_1(x_1, u) \\ \dot{x}_2 &= x_3 + \varphi_2(x_1, x_2, u) \\ &\vdots \\ \dot{x}_n &= \varphi_n(\mathbf{x}, u) \\ y &= x_1\end{aligned}$$

则可构造高增益观测器：

$$\begin{aligned}\dot{\hat{x}}_1 &= \hat{x}_2 + \varphi_1(\hat{x}_1, u) + (\gamma_1/\varepsilon)(y - \hat{x}_1) \\ \dot{\hat{x}}_2 &= \hat{x}_3 + \varphi_2(\hat{x}_1, \hat{x}_2, u) + (\gamma_2/\varepsilon^2)(y - \hat{x}_1) \\ &\vdots \\ \dot{\hat{x}}_n &= \varphi_n(\hat{\mathbf{x}}, u) + (\gamma_n/\varepsilon^n)(y - \hat{x}_1)\end{aligned}$$

其中 $\varepsilon > 0$ 充分小（高增益），系数 γ_i 使多项式 $s^n + \gamma_1 s^{n-1} + \cdots + \gamma_n$ 为 Hurwitz。 ε 越小，估计误差的收敛速度越快，但对测量噪声的敏感性也越强。

11.6.2 滑模观测器

将滑模技术应用于观测器设计，利用不连续的注入项使估计误差滑向零。滑模观测器的基本结构为

$$\dot{\hat{\mathbf{x}}} = A\hat{\mathbf{x}} + \mathbf{b}u + \mathbf{g}$$

其中 $\mathbf{g}$ 是不连续的注入项，通常取 $\mathbf{g} = \mathbf{L} \operatorname{sgn}(y - C\hat{\mathbf{x}})$ 。在滑模面上，等效注入等价于一个线性的 Luenberger 观测器，从而在滑模期间实现精确的状态重构。

滑模观测器的优势在于：

1. 对匹配不确定性和扰动具有完全鲁棒性（在滑动阶段）；
2. 可以实现有限时间状态估计（区别于渐近收敛）；
3. 在故障检测与隔离（FDI）中有广泛应用。

§11.7

Python 实现：倒立摆的滑模控制仿真

滑模控制器实现

```

1 \begin{lstlisting}[language=Python]
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 # 倒立摆参数
6 g, L, m = 9.81, 0.5, 0.2
7
8 def pend_dyn(x, u):
9     """倒立摆非线性动力学  $x = [\theta, \dot{\theta}]$ """
10    theta, dtheta = x[0], x[1]
11    ddtheta = (g/L) * np.sin(theta) + u / (m * L**2)
12    return np.array([dtheta, ddtheta])
13
14 class SlidingModeController:
15     def __init__(self, lam=5.0, eta=0.5, k=8.0, phi=0.05):
16         self.lam = lam      # 滑模面参数  $s = \dot{\theta} + \text{lam} * \theta$ 
17         self.eta = eta      # 等速趋近律系数
18         self.k = k          # 指数趋近律系数
19         self.phi = phi      # 边界层厚度
20
21     def sliding_surface(self, x):
22         return x[1] + self.lam * x[0]
23
24     def control(self, x):
25         s = self.sliding_surface(x)
26         theta, dtheta = x[0], x[1]
27
28         # 等效控制 (当  $s = 0$  且  $\dot{s} = 0$  时)
29         u_eq = -m * L**2 * (self.lam * dtheta + (g/L) * np.sin(theta))
30
31         # 趋近律 (带边界层的饱和函数)
32         b = m * L**2
33         if abs(s) > self.phi:
34             u_sw = -b * (self.k * np.sign(s) + self.eta * s)
35         else:
36             u_sw = -b * (self.k * s / self.phi + self.eta * s)
37
38         return u_eq + u_sw
39
40 # 仿真
41 dt = 0.001
42 t = np.arange(0, 5, dt)
43 n = len(t)
44
45 smc = SlidingModeController(lam=5.0, eta=0.5, k=8.0, phi=0.05)
46
47 x = np.zeros((2, n))
48 u_arr = np.zeros(n)
49 s_arr = np.zeros(n)
50 x[:, 0] = [0.5, 0.0] # 初始角30度
51

```

不同趋近律的对比

```

1 \begin{lstlisting}[language=Python]
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 def simulate_smc(controller, x0, dt=0.001, T=5):
6     n = int(T / dt)
7     t = np.linspace(0, T, n)
8     x = np.zeros((2, n))
9     s = np.zeros(n)
10    x[:, 0] = x0
11    for i in range(n - 1):
12        s[i] = controller.sliding_surface(x[:, i])
13        u = controller.control(x[:, i])
14        xdot = pend_dyn(x[:, i], u)
15        x[:, i+1] = x[:, i] + xdot * dt
16    return t, x, s
17
18 class SMC_Exponential:
19     def __init__(self, lam=5.0, eta=0.5, k=8.0):
20         self.lam, self.eta, self.k = lam, eta, k
21     def sliding_surface(self, x):
22         return x[1] + self.lam * x[0]
23     def control(self, x):
24         s = self.sliding_surface(x)
25         u_eq = -m*L**2 * (self.lam*x[1] + (g/L)*np.sin(x[0]))
26         u_sw = -m*L**2 * (self.k * np.sign(s) + self.eta * s)
27         return u_eq + u_sw
28
29 class SMC_PowerRate:
30     def __init__(self, lam=5.0, k=5.0, alpha=0.5):
31         self.lam, self.k, self.alpha = lam, k, alpha
32     def sliding_surface(self, x):
33         return x[1] + self.lam * x[0]
34     def control(self, x):
35         s = self.sliding_surface(x)
36         u_eq = -m*L**2 * (self.lam*x[1] + (g/L)*np.sin(x[0]))
37         u_sw = -m*L**2 * self.k * np.abs(s)**self.alpha * np.sign(s)
38         return u_eq + u_sw
39
40 # 对比仿真
41 smc_exp = SMC_Exponential()
42 smc_pwr = SMC_PowerRate()
43 t1, x1, s1 = simulate_smc(smc_exp, [0.5, 0.0])
44 t2, x2, s2 = simulate_smc(smc_pwr, [0.5, 0.0])
45
46 fig, axes = plt.subplots(2, 2, figsize=(12, 8))
47 axes[0, 0].plot(t1, x1[0], label='Exponential')
48 axes[0, 0].plot(t2, x2[0], '--', label='Power Rate')
49 axes[0, 0].set_ylabel(r'$\theta$ (rad)')
50 axes[0, 0].legend()
51 axes[0, 0].grid(True)
52
53 axes[0, 1].plot(t1, s1, label='Exponential')
54 axes[0, 1].plot(t2, s2, '--', label='Power Rate')
55 axes[0, 1].set_ylabel(r'$s$')
56 axes[0, 1].legend()
57 axes[0, 1].grid(True)
58
59 axes[1, 0].plot(t1, x1[1], label='Exponential')
60 axes[1, 0].plot(t2, x2[1], '--', label='Power Rate')
61 axes[1, 0].set_ylabel(r'$\dot{\theta}$ (rad/s)')
62 axes[1, 0].legend()
63 axes[1, 0].grid(True)
64
65 axes[1, 1].plot(t1, s1, label='Exponential')
66 axes[1, 1].plot(t2, s2, '--', label='Power Rate')
67 axes[1, 1].set_ylabel(r'$\dot{s}$')
68 axes[1, 1].legend()
69 axes[1, 1].grid(True)
70
71 plt.tight_layout()
72 plt.show()
73 \end{lstlisting}

```

§ 11.8

小结

非线性控制是一个广阔而活跃的研究领域。本章介绍了其基本框架和核心方法：

1. **非线性特性**：多平衡点、极限环、混沌和分岔使非线性系统远较线性系统复杂；
2. **Lyapunov 稳定性理论**：以“能量函数”的概念为核心，是分析非线性系统稳定性的最基本工具；Lasalle 不变原理推广了 Lyapunov 方法的应用范围；
3. **滑模控制**：利用不连续控制将状态驱动到预设滑模面，对匹配扰动具有完全鲁棒性，但需注意抖振问题；
4. **反馈线性化**：利用微分几何工具通过坐标变换和反馈将非线性系统精确转化为线性系统；Lie 导数、Lie 括号和相对阶是其中的核心概念；
5. **Backstepping**：针对严格反馈系统的递归 Lyapunov 设计方法，可灵活处理非线性和参数不确定性；
6. **非线性观测器**：高增益观测器和滑模观测器为非线性系统的状态估计提供了有效方案。

现代非线性控制理论仍在持续发展，包括事件触发控制、预设性能控制、基于障碍 Lyapunov 函数的约束控制等新方向。下一章将进一步介绍模型预测控制、鲁棒控制和智能控制等高级方法。

前几章介绍的极点配置、LQR 和滑模控制等方法构成了经典现代控制理论的基础。然而，随着工业过程日益复杂和对控制系统性能要求的不断提高，一系列更为先进的控制方法应运而生。本章系统介绍模型预测控制（MPC）、鲁棒控制（ H_∞ ）、自适应控制、强化学习与控制、以及智能控制方法。这些方法各有侧重：MPC 擅长处理约束与多变量耦合，鲁棒控制保障不确定性下的性能，自适应控制在线调节参数，强化学习在未知环境中探索最优策略，智能控制借鉴人类推理和生物学习机制。理解各方法的原理、优势和局限，是综合运用它们解决实际工程问题的前提。

§ 12.1 模型预测控制

12.1.1 三大基本原理

模型预测控制（Model Predictive Control, MPC）并非单一算法，而是一类基于在线优化的控制策略，其核心由三大原理构成。

MPC 的三大原理

- 预测模型：**利用系统的数学模型预测未来一段时间（预测时域 N_p ）内的输出响应。模型可以是状态空间模型、传递函数、卷积模型（FIR/FSR）或非线性模型；
- 滚动优化：**在每个采样时刻求解一个有限时域最优控制问题，计算最优控制序列 $\mathbf{u}^*(k), \mathbf{u}^*(k+1), \dots, \mathbf{u}^*(k+N_c-1)$ （ $N_c \leq N_p$ 为控制时域），但仅实施第一步 $\mathbf{u}^*(k)$ 。下一采样时刻重新测量状态，重复优化过程——即“滚动”；
- 反馈校正：**利用实时测量信息修正预测偏差，补偿模型失配和未知扰动的影响，构成闭环机制。

12.1.2 MPC 的数学表述——二次规划形式

考虑离散时间线性时不变系统：

$$\mathbf{x}_{k+1} = A\mathbf{x}_k + B\mathbf{u}_k, \quad \mathbf{y}_k = C\mathbf{x}_k$$

MPC 在每个采样时刻 k 求解以下优化问题：

$$\min_{\mathbf{U}_k} \sum_{i=0}^{N_p-1} (\mathbf{x}_{k+i|k}^\top \mathbf{Q} \mathbf{x}_{k+i|k} + \mathbf{u}_{k+i}^\top \mathbf{R} \mathbf{u}_{k+i}) + \mathbf{x}_{k+N_p|k}^\top \mathbf{Q}_f \mathbf{x}_{k+N_p|k} \quad (12.1)$$

满足约束条件：

$$\begin{aligned} \mathbf{x}_{k+i+1|k} &= \mathbf{A} \mathbf{x}_{k+i|k} + \mathbf{B} \mathbf{u}_{k+i}, \quad i = 0, \dots, N_p - 1 \\ \mathbf{x}_{k|k} &= \mathbf{x}_k \\ \mathbf{x}_{k+i|k} &\in \mathcal{X}, \quad i = 1, \dots, N_p \\ \mathbf{u}_{k+i} &\in \mathcal{U}, \quad i = 0, \dots, N_c - 1 \end{aligned}$$

其中 $\mathbf{U}_k = [\mathbf{u}_k^\top, \dots, \mathbf{u}_{k+N_c-1}^\top]^\top$ 是决策变量， $\mathcal{X}$ 和 $\mathcal{U}$ 分别为状态约束集和输入约束集（通常为多面体）， $\mathbf{Q}_f$ 是终端代价矩阵。

12.1.3 预测方程的推导

利用递推关系，未来 N_p 步的状态预测可以紧凑地表示为当前状态和未来控制的函数：

$$\begin{bmatrix} \mathbf{x}_{k+1|k} \\ \mathbf{x}_{k+2|k} \\ \vdots \\ \mathbf{x}_{k+N_p|k} \end{bmatrix} = \underbrace{\begin{bmatrix} \mathbf{A} \\ \mathbf{A}^2 \\ \vdots \\ \mathbf{A}^{N_p} \end{bmatrix}}_{\Phi} \mathbf{x}_k + \underbrace{\begin{bmatrix} \mathbf{B} & 0 & \cdots & 0 \\ \mathbf{A}\mathbf{B} & \mathbf{B} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{A}^{N_p-1}\mathbf{B} & \mathbf{A}^{N_p-2}\mathbf{B} & \cdots & \mathbf{B} \end{bmatrix}}_{\Gamma} \begin{bmatrix} \mathbf{u}_k \\ \mathbf{u}_{k+1} \\ \vdots \\ \mathbf{u}_{k+N_c-1} \end{bmatrix}$$

代入目标函数，可得紧凑的二次规划（QP）形式：

$$\min_{\mathbf{U}_k} \frac{1}{2} \mathbf{U}_k^\top \mathbf{H} \mathbf{U}_k + \mathbf{x}_k^\top \mathbf{F}^\top \mathbf{U}_k$$

其中 $\mathbf{H} \succ 0$ （正定 Hessian 矩阵），约束为线性不等式 $\mathbf{G} \mathbf{U}_k \leq \mathbf{h}$ 。这种标准的凸 QP 问题可以用内点法或有效集法高效求解。

12.1.4 约束处理

MPC 相较于 LQR 的根本优势在于直接处理约束。典型约束包括：

- 输入约束： $\mathbf{u}_{\min} \leq \mathbf{u}_k \leq \mathbf{u}_{\max}$ （执行器饱和）；
- 输入速率约束： $\Delta \mathbf{u}_{\min} \leq \mathbf{u}_k - \mathbf{u}_{k-1} \leq \Delta \mathbf{u}_{\max}$ ；
- 状态约束： $\mathbf{x}_{\min} \leq \mathbf{x}_k \leq \mathbf{x}_{\max}$ （安全边界）。

这些约束在 QP 中被编码为线性矩阵不等式。

12.1.5 可行性与稳定性

MPC 的稳定性分析比 LQR 复杂，因为终端约束和终端代价的选择影响闭环稳定性。保证稳定性的常用策略包括：

1. 终端等式约束：强制 $\mathbf{x}_{k+N_p|k} = \mathbf{0}$ （但可能损害可行性）；
2. 终端代价 + 终端约束集：选取 $\mathbf{Q}_f$ 为 LQR 的 Riccati 解 $\mathbf{P}$ ，并令终端约束集为 LQR 的最大不变集——保证递归可行性和渐近稳定性；

3. 无限预测时域等价：通过恰当的终端条件使有限时域 MPC 等价于无限时域 LQR。

注. MPC 的工业应用极其广泛，涵盖炼油、化工、电力、交通等领域。据估计，全球 90% 以上的先进过程控制（APC）应用基于某种形式的 MPC 技术。其计算负担在嵌入式平台上面临挑战，但专用 QP 求解器（如 qpOASES、OSQP）和显式 MPC 技术正推动 MPC 向更快采样频率的应用拓展。

12.1.6 非线性 MPC 与分布式 MPC

非线性 MPC（NMPC）直接使用非线性模型预测，每次求解非线性规划（NLP）而非 QP。NMPC 对强非线性过程可获得更好的预测精度，但计算代价更高，且可能受局部极小的困扰。

分布式 MPC 将大规模系统（如电网、交通网络）分解为多个相互耦合的子系统，各子系统分别求解局部 MPC 问题，并通过通信交换信息实现协调控制。这种方法兼顾了全局最优性和计算可行性。

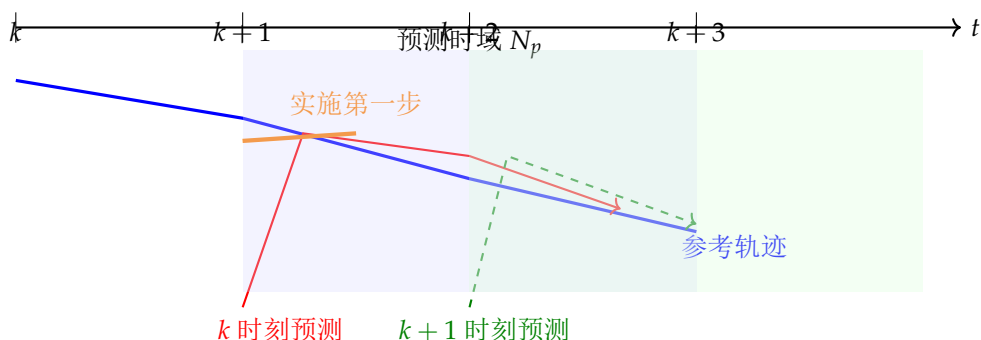


图 12.1: MPC 滚动优化原理。在每个采样时刻求解有限时域的最优控制问题（红色/绿色虚线），但仅实施第一步控制（橙色实线）。采样时刻推进后，预测时域窗口向前滚动，重新优化。

§ 12.2 鲁棒控制

12.2.1 不确定性的描述

实际系统与数学模型之间总存在差异，这些差异统称为“不确定性”。鲁棒控制的目标是设计控制器，使得闭环系统在存在不确定性时仍能保持稳定并满足一定的性能要求。

不确定性的主要描述方式包括：

1. 加性不确定性： $G_p(s) = G(s) + \Delta_a(s)$ ，其中 $\|\Delta_a(j\omega)\|$ 有界；
2. 乘性不确定性： $G_p(s) = G(s)(I + \Delta_m(s))$ ，更适合描述高频建模误差；
3. 互质因子不确定性： $G_p = (M + \Delta_M)^{-1}(N + \Delta_N)$ ，覆盖更广的不确定性类型。

小增益定理

设 $M(s)$ 为稳定传递函数矩阵。若 $\|M\|_\infty < 1$ ，则对于任何稳定且满足 $\|\Delta\|_\infty \leq 1$ 的不确定性 Δ ，闭环系统 $M \star \Delta$ 对所有 Δ 是稳定的。这里 $\|\cdot\|_\infty$ 表示 H_∞ 范数：

$$\|G\|_\infty = \sup_{\omega} \bar{\sigma}(G(j\omega))$$

小增益定理是鲁棒稳定性分析的基石。当 $M(s)$ 与 $\Delta(s)$ 的 H_∞ 范数乘积小于 1 时，任何范数有界的扰动都不会破坏系统稳定性。

12.2.2 H_∞ 控制问题

H_∞ 控制的基本问题是：设计控制器 $K(s)$ 使得闭环系统的 H_∞ 范数最小——即最小化扰动到输出的最坏情况能量增益。

考虑广义被控对象 $P(s)$ ：

$$\begin{bmatrix} \mathbf{z} \\ \mathbf{y} \end{bmatrix} = \begin{bmatrix} P_{11} & P_{12} \\ P_{21} & P_{22} \end{bmatrix} \begin{bmatrix} \mathbf{w} \\ \mathbf{u} \end{bmatrix}$$

其中 $\mathbf{w}$ 是外生输入（参考、扰动、噪声）， $\mathbf{u}$ 是控制输入， $\mathbf{z}$ 是被控输出（跟踪误差、控制代价）， $\mathbf{y}$ 是测量输出。反馈连接 $\mathbf{u} = K\mathbf{y}$ 后，从 $\mathbf{w}$ 到 $\mathbf{z}$ 的闭环传递函数为

$$T_{zw} = P_{11} + P_{12}K(I - P_{22}K)^{-1}P_{21}$$

H_∞ 最优控制问题是求解

$$\min_{K \text{ 稳定化}} \|T_{zw}\|_\infty$$

而 H_∞ 次优控制问题是：给定 $\gamma > 0$ ，寻找稳定化控制器 K 使得 $\|T_{zw}\|_\infty < \gamma$ 。

12.2.3 标准 H_∞ 框架与 Riccati 方法

对于正则的 H_∞ 问题，Glover 和 Doyle（1988）给出了基于两个 Riccati 方程的解。设广义被控对象的状态空间实现为

$$\begin{aligned} \dot{\mathbf{x}} &= A\mathbf{x} + B_1\mathbf{w} + B_2\mathbf{u} \\ \mathbf{z} &= C_1\mathbf{x} + D_{11}\mathbf{w} + D_{12}\mathbf{u} \\ \mathbf{y} &= C_2\mathbf{x} + D_{21}\mathbf{w} + D_{22}\mathbf{u} \end{aligned}$$

在标准假设下（ (A, B_1) 能稳， (A, B_2) 能稳， (C_1, A) 能检测， (C_2, A) 能检测； $D_{12}^\top D_{12} = I$ ， $D_{21} D_{21}^\top = I$ ； $D_{11} = 0, D_{22} = 0$ ），存在次优 H_∞ 控制器的充要条件是以下两个 Riccati 方程存在正定解 X_∞ 和 Y_∞ ：

$$\begin{aligned} A^\top X_\infty + X_\infty A + C_1^\top C_1 + X_\infty(\gamma^{-2}B_1B_1^\top - B_2B_2^\top)X_\infty &= 0 \\ AY_\infty + Y_\infty A^\top + B_1B_1^\top + Y_\infty(\gamma^{-2}C_1^\top C_1 - C_2^\top C_2)Y_\infty &= 0 \end{aligned}$$

且 $\rho(X_\infty Y_\infty) < \gamma^2$ 。中心控制器为：

$$K_\infty(s) = \begin{bmatrix} A_\infty & -Z_\infty L_\infty \\ F_\infty & 0 \end{bmatrix}$$

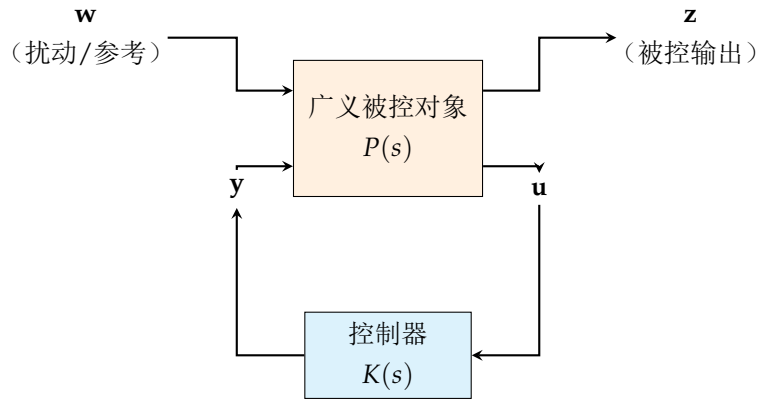
其中各项由 Riccati 解给出。

12.2.4 μ 综合简介

对于非结构不确定性， H_∞ 方法十分有效。但当不确定性具有结构化（如每个通道独立变化）时， H_∞ 条件会过度保守。 μ 综合将结构化奇异值 $\mu_\Delta(M)$ 作为鲁棒稳定性的度量：

$$\text{鲁棒稳定} \iff \sup_{\omega} \mu_\Delta(M(j\omega)) < 1$$

μ 综合通过“D-K 迭代”交替优化控制器和尺度矩阵，通常能得到比纯 H_∞ 更优的结果，但计算复杂度较高。



$$\min_K \|T_{zw}\|_\infty$$

图 12.2: H_∞ 控制的标准框架。广义被控对象 $P(s)$ 囊括了被控对象本身、加权函数和互联结构，控制器 $K(s)$ 通过测量输出 y 产生控制 u 以最小化扰动 w 到被控输出 z 的 H_∞ 增益。

§ 12.3 自适应控制

12.3.1 模型参考自适应控制

模型参考自适应控制（Model Reference Adaptive Control, MRAC）的基本思想是：设计一个参考模型来描述期望的闭环响应特性，然后通过调节控制器的参数使得被控对象的输出渐近跟踪参考模型的输出。

参考模型通常是一个稳定的线性系统：

$$\dot{\mathbf{x}}_m = A_m \mathbf{x}_m + B_m \mathbf{r}$$

被控对象含未知参数 θ ：

$$\dot{\mathbf{x}} = A(\theta) \mathbf{x} + B(\theta) \mathbf{u}$$

控制律为 $\mathbf{u} = \hat{K}(t) \mathbf{x} + \hat{L}(t) \mathbf{r}$ ，其中 $\hat{K}(t)$ 和 $\hat{L}(t)$ 由自适应律在线调整。

跟踪误差 $\mathbf{e} = \mathbf{x} - \mathbf{x}_m$ 的动态为

$$\dot{\mathbf{e}} = A_m \mathbf{e} + B(\theta)(\tilde{K} \mathbf{x} + \tilde{L} \mathbf{r})$$

取 Lyapunov 函数

$$V = \mathbf{e}^\top P \mathbf{e} + \text{tr}(\tilde{K}^\top \Gamma^{-1} \tilde{K}) + \text{tr}(\tilde{L}^\top \Gamma^{-1} \tilde{L})$$

其中 P 满足 Lyapunov 方程 $A_m^\top P + P A_m = -Q$ ($Q \succ 0$)。自适应律取为

$$\dot{\hat{K}} = -\Gamma B^\top P \mathbf{e} \mathbf{x}^\top, \quad \dot{\hat{L}} = -\Gamma B^\top P \mathbf{e} \mathbf{r}^\top$$

可证明 $\dot{V} = -\mathbf{e}^\top Q \mathbf{e} \leq 0$ ，跟踪误差渐近收敛到零。

12.3.2 自校正控制

自校正控制（Self-Tuning Control, STC）采用“辨识-控制”两步法：

1. 在线参数辨识：利用递推最小二乘（RLS）等方法实时估计系统参数；

2. **控制器设计**：将参数估计值代入某种控制设计算法（如极点配置、LQR、最小方差控制），计算控制信号。

递推最小二乘（RLS）是 STC 中最常用的参数估计算法。对于线性回归模型

$$y_k = \boldsymbol{\varphi}_k^\top \boldsymbol{\theta} + \varepsilon_k$$

RLS 的递推公式为

$$\hat{\boldsymbol{\theta}}_k = \hat{\boldsymbol{\theta}}_{k-1} + \mathbf{L}_k(y_k - \boldsymbol{\varphi}_k^\top \hat{\boldsymbol{\theta}}_{k-1}) \quad (12.2)$$

$$\mathbf{L}_k = \frac{P_{k-1} \boldsymbol{\varphi}_k}{\lambda + \boldsymbol{\varphi}_k^\top P_{k-1} \boldsymbol{\varphi}_k} \quad (12.3)$$

$$P_k = \frac{1}{\lambda} (I - \mathbf{L}_k \boldsymbol{\varphi}_k^\top) P_{k-1} \quad (12.4)$$

其中 $0 < \lambda \leq 1$ 为遗忘因子（ $\lambda = 1$ 对应标准 RLS， $\lambda < 1$ 赋予近期数据更高的权重，以适应时变系统）。

注. MRAC 关注跟踪误差的直接稳定性（直接自适应），其理论分析基于 Lyapunov 方法；STC 遵循“确定性等价原理”——将参数估计视同真值用于控制器设计（间接自适应）。MRAC 通常收敛速度可调，但需要匹配条件（如被控对象与参考模型同阶）；STC 更灵活，但辨识阶段的持续激励条件和瞬态性能保障是设计的难点。

12.3.3 MIT 规则与归一化自适应律

除 Lyapunov 方法外，MIT 规则（以 MIT Instrumentation Lab 命名）是一种基于梯度下降的参数调节方法。定义损失函数 $J = \frac{1}{2}e^2$ ，参数调整方向为

$$\dot{\boldsymbol{\theta}} = -\gamma \frac{\partial J}{\partial \boldsymbol{\theta}} = -\gamma e \frac{\partial e}{\partial \boldsymbol{\theta}}$$

其中 $\frac{\partial e}{\partial \boldsymbol{\theta}}$ 是“灵敏度导数”。MIT 规则直观简单，但缺乏稳定性保证，在非匹配不确定性下可能发散。

归一化自适应律通过在自适应增益中引入 $\mathbf{x}^\top \mathbf{x}$ 的正则项来增强鲁棒性：

$$\dot{\hat{\mathbf{K}}} = -\frac{\Gamma \mathbf{B}^\top \mathbf{P} \mathbf{e} \mathbf{x}^\top}{1 + \mathbf{x}^\top \mathbf{x}}$$

归一化可以防止参数漂移，但足以保证跟踪误差的收敛。

§ 12.4 强化学习与控制

12.4.1 MDP 与最优控制的等价性

马尔可夫决策过程（MDP）是强化学习（RL）的基本数学框架。有趣的是，确定性 LQR 问题可以完全嵌入 RL 的框架：

- **状态**： $\mathbf{x}_k$ （对应 MDP 状态 s_k ）；
- **动作**： $\mathbf{u}_k$ （对应 MDP 动作 a_k ）；
- **奖励**： $r_k = -(\mathbf{x}_k^\top \mathbf{Q} \mathbf{x}_k + \mathbf{u}_k^\top \mathbf{R} \mathbf{u}_k)$ （性能代价的负值）；
- **价值函数**： $V(\mathbf{x}_k) = \mathbf{x}_k^\top \mathbf{P} \mathbf{x}_k$ （恰好是 Riccati 方程的解）。

LQR 与最优价值函数的等价性

对于离散 LQR 问题，最优价值函数是状态的二次型 $V^*(\mathbf{x}) = \mathbf{x}^\top P \mathbf{x}$ ，其中 P 是离散 Riccati 方程的解。最优策略为 $\mathbf{u}^* = -K\mathbf{x}$ ，其中 $K = (R + B^\top P B)^{-1} B^\top P A$ 。这精确对应了 RL 中的 Bellman 最优方程。

12.4.2 Policy Iteration 求解 LQR

当系统矩阵 A 和 B 未知时，可以使用 Policy Iteration 方法在线求解 LQR；在 RL 语境中这被称为“策略迭代”：

面向 LQR 的 Policy Iteration（离线/在线）

1. 初始化：选取初始稳定化增益 K_0 （即 $\rho(A - BK_0) < 1$ ）；

2. 策略评估：解 Lyapunov 方程求 P_j ：

$$(A - BK_j)^\top P_j (A - BK_j) - P_j + Q + K_j^\top R K_j = 0$$

3. 策略改进：更新增益

$$K_{j+1} = (R + B^\top P_j B)^{-1} B^\top P_j A$$

4. 重复步骤 2-3 直到收敛。

当模型 (A, B) 未知时，策略评估可通过政策评估（On-policy）或 Q-学习（Off-policy）的 RL 方法在线收集数据完成，实现无模型的最优控制设计。

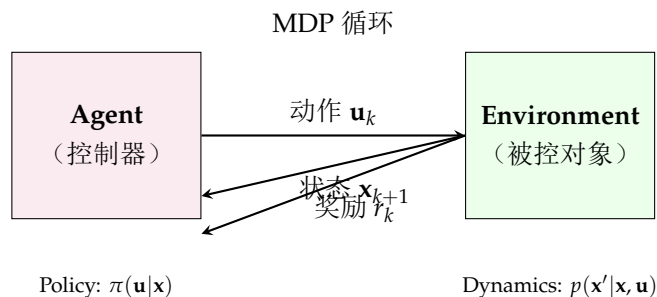
12.4.3 自适应动态规划（ADP）

自适应动态规划（Adaptive Dynamic Programming, ADP）是控制与 RL 交汇的重要分支。ADP 使用函数逼近器（如神经网络）来近似最优价值函数，然后通过策略迭代或值迭代求解近似最优控制律。

基本的启发式动态规划（HDP）结构包括三个网络：

- 模型网络：近似系统动力学 $\hat{\mathbf{x}}_{k+1} = \hat{f}(\mathbf{x}_k, \mathbf{u}_k)$ ；
- 评价网络（Critic）：近似价值函数 $\hat{V}(\mathbf{x})$ ；
- 执行网络（Actor）：近似最优策略 $\hat{\mathbf{u}} = \hat{\pi}(\mathbf{x})$ 。

训练过程交替进行 Critic 更新（最小化 Bellman 残差）和 Actor 更新（沿价值函数的负梯度方向改进策略）。



$$\text{Bellman 方程: } V^*(\mathbf{x}) = \max_{\mathbf{u}} [r + \gamma V^*(\mathbf{x}')]]$$

图 12.3: 强化学习与控制问题的 MDP 框架。控制器 (Agent) 根据状态 $\mathbf{x}_k$ 选择动作 $\mathbf{u}_k$ ，被控对象 (Environment) 转移到新状态 $\mathbf{x}_{k+1}$ 并反馈奖励信号 r_k 。策略优化即最大化长期累积奖励的期望。

§ 12.5 智能控制

12.5.1 模糊控制

模糊控制基于模糊集合理论和模糊逻辑推理，最早由 Zadeh (1965) 提出理论，Mamdani (1974) 首次应用于蒸汽机控制。其三大组成部分为：

1. **模糊化**：将精确的输入量（如“误差 = 0.7”）映射到模糊集合（如“正大”，“正中”，“零”等），用隶属度函数 $\mu_A(x) \in [0, 1]$ 量化隶属程度；
2. **模糊推理**：基于 IF-THEN 规则库进行推理。典型规则形式为：

$$\text{IF } e \text{ is NB AND } \dot{e} \text{ is PB THEN } u \text{ is ZE}$$

规则库编码了领域专家的控制经验和知识；

3. **去模糊化**：将模糊推理的输出（模糊集合）转化为精确的控制信号。常用方法有重心法 (COG) 和最大隶属度法。

模糊控制器的核心优势在于：不需要精确的数学模型，能直接利用人类专家的定性经验，对参数变化和干扰具有较好的鲁棒性。其局限性在于缺乏严格的稳定性分析和系统的设计方法。

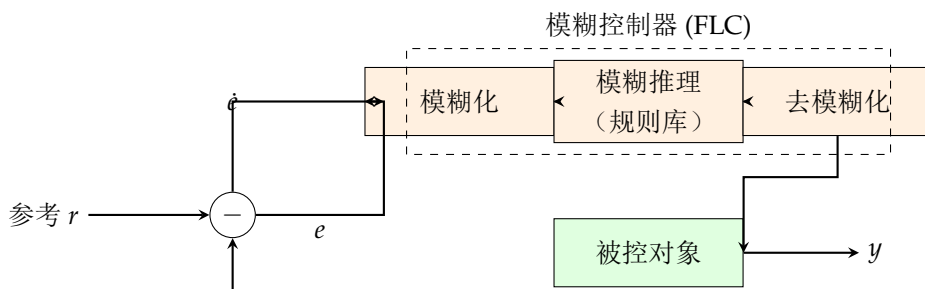


图 12.4: 模糊控制系统的基本架构。误差 e 和误差变化率 $\dot{e}$ 经模糊化转换为模糊量，通过 IF-THEN 规则库进行模糊推理，再经去模糊化转换为精确控制量作用于被控对象。

12.5.2 神经网络控制

神经网络控制利用人工神经网络的万能逼近能力进行控制设计。主要范式包括：

1. **系统辨识**：用神经网络逼近未知动力学：

$$\mathbf{x}_{k+1} = \hat{f}_{NN}(\mathbf{x}_k, \mathbf{u}_k; \mathbf{w})$$

通过最小化预测误差训练网络权重 $\mathbf{w}$ ；

2. **逆模控制**：训练神经网络学习系统的逆动力学映射，使控制器输出恰好产生期望的系统响应；
3. **NARMA-L2 控制**：一种基于线性化近似的神经网络控制方案。将系统近似为 NARMA 模型后，设计一个简单的线性控制律，利用神经网络计算控制所需的系统 Jacobian 信息。

NARMA-L2 模型的数学表述为

$$y_{k+d} = f(y_k, \dots, y_{k-n+1}, u_{k-1}, \dots, u_{k-n+1}) + g(\dots) \cdot u_k$$

控制律直接取为

$$u_k = \frac{y_{k+d}^{\text{des}} - \hat{f}(\cdot)}{\hat{g}(\cdot)}$$

其中 $\hat{f}$ 和 $\hat{g}$ 是两个并行训练的神经网络。

注. 模糊控制与神经网络各有侧重：模糊控制擅长利用定性知识，解释性强；神经网络擅长从数据中学习，拟合能力强。将两者融合的模糊神经网络（ANFIS）兼具两者的优势，通过神经网络自动调节模糊系统的隶属度参数，近年来在工业过程控制中获得广泛应用。

§ 12.6 前沿方向

12.6.1 学习型 MPC

传统 MPC 依赖于精确的数学模型，而学习型 MPC（Learning-based MPC）利用机器学习——尤其是高斯过程（GP）和贝叶斯优化——从数据中在线学习模型的剩余不确定性，然后将学习到的不确定性边界纳入 MPC 的约束处理中，实现“安全 + 最优”的权衡。其核心公式为：在标准 MPC 代价中加入数据驱动的约束收紧项：

$$\mathbf{x}_{k+i|k} \in \mathcal{X} \ominus \mathcal{B}_{\text{learned}}(\mathbf{x}_{k+i|k})$$

其中 $\mathcal{B}_{\text{learned}}$ 是从数据习得的不确定性边界。

12.6.2 Safe RL

强化学习在游戏和仿真中取得了瞩目成就，但在物理系统中的部署面临严重的安全挑战——探索过程中的随机动作可能导致灾难性后果。Safe RL 旨在：

- 利用控制屏障函数（Control Barrier Function, CBF）约束 RL 的策略空间；
- 将安全约束编码为 CMDP（Constrained MDP）框架下的期望约束或概率约束；
- 结合 MPC 的在线优化确保每个决策都安全可执行——形成 RL-MPC 混合架构。

12.6.3 基于控制理论的深度学习解释

近年来，深度学习与控制理论的交叉成为热点。代表性成果包括：

- **神经微分方程 (Neural ODE)**：将 ResNet 的层间映射解释为 ODE 的离散化，用伴随灵敏度方法高效计算梯度；
- **Lipschitz 网络**：利用控制理论中的耗散性和无源性概念约束神经网络的 Lipschitz 常数，提高鲁棒性；
- **Pontryagin 的神经网络**：将最优控制中的必要条件嵌入神经网络结构，使得网络结构本身就编码了最优性。

Neural ODE 的数学形式简洁而深刻：

$$\frac{d\mathbf{h}(t)}{dt} = \mathbf{f}_\theta(\mathbf{h}(t), t), \quad \mathbf{h}(t_0) = \mathbf{x} \quad (12.5)$$

输出 $\mathbf{y} = \mathbf{h}(t_1)$ 。训练采用伴随方法 (Adjoint Method) ——这正是最优控制理论中 Pontryagin 极小值原理的协态方程。

§ 12.7 Python 实现

12.7.1 简单 MPC 的 QP 求解

Python 线性 MPC 实现

```

1 \begin{lstlisting}[language=Python]
2 import numpy as np
3 from scipy import linalg
4 import matplotlib.pyplot as plt
5
6 dt = 0.1
7 A = np.array([[1, dt], [0, 1]])
8 B = np.array([[0.5*dt**2], [dt]])
9 n_states, n_inputs = 2, 1
10
11 N = 10 # 预测时域
12 Q = np.diag([10, 1])
13 R = np.array([[0.1]])
14
15 # 构造QP矩阵
16 def build_mpc_qp(A, B, Q, R, N):
17     n = A.shape[0]
18     m = B.shape[1]
19
20     Phi = np.vstack([np.linalg.matrix_power(A, i+1) for i in range(N)])
21     Gamma = np.zeros((N*n, N*m))
22     for i in range(N):
23         for j in range(i+1):
24             Gamma[i*n:(i+1)*n, j*m:(j+1)*m] = (
25                 np.linalg.matrix_power(A, i-j) @ B)
26
27     Qbar = linalg.block_diag(*([Q]*(N-1) + [Q]))
28     Rbar = linalg.block_diag(*([R]*N))
29
30     H = Gamma.T @ Qbar @ Gamma + Rbar
31     F = Gamma.T @ Qbar @ Phi
32     return H, F, Phi, Gamma
33
34 H, F, Phi, Gamma = build_mpc_qp(A, B, Q, R, N)
35
36 # MPC 仿真
37 x0 = np.array([1.0, 0.0])
38 n_sim = 100
39 x_hist = np.zeros((2, n_sim+1))
40 u_hist = np.zeros(n_sim)
41 x_hist[:, 0] = x0
42
43 for k in range(n_sim):
44     xk = x_hist[:, k]
45     u = -np.linalg.solve(H, F.T @ xk)
46     u_hist[k] = u[0]
47     x_hist[:, k+1] = A @ xk + B.flatten() * u[0]
48
49 fig, axes = plt.subplots(3, 1, figsize=(10, 8))

```


12.7.2 RL 求解 LQR

Python Policy Iteration 求解 LQR

```

1 \begin{lstlisting}[language=Python]
2 import numpy as np
3 from scipy import linalg
4
5 def policy_iteration_lqr(A, B, Q, R, KO=None, max_iter=100, tol=1e-6):
6     n, m = A.shape[0], B.shape[1]
7     if KO is None:
8         KO = np.zeros((m, n))
9     K = KO.copy()
10    cost_history = []
11
12    for iteration in range(max_iter):
13        # 策略评估：解Lyapunov方程
14        A_cl = A - B @ K
15        P = linalg.solve_discrete_lyapunov(A_cl.T, Q + K.T @ R @ K)
16
17        # 策略改进
18        K_new = linalg.solve(R + B.T @ P @ B, B.T @ P @ A)
19
20        cost = np.trace(P)
21        cost_history.append(cost)
22
23        if np.linalg.norm(K_new - K) < tol:
24            K = K_new
25            break
26        K = K_new
27
28    return K, P, cost_history
29
30 # 数值示例
31 dt = 0.1
32 A = np.array([[1, dt], [0, 1]])
33 B = np.array([[0.5*dt**2], [dt]])
34 Q = np.diag([10, 1])
35 R = np.array([[0.1]])
36
37 K_star, P_star, costs = policy_iteration_lqr(A, B, Q, R)
38
39 # 与解析解对比
40 P_dare = linalg.solve_discrete_are(A, B, Q, R)
41 K_dare = linalg.inv(R + B.T @ P_dare @ B) @ B.T @ P_dare @ A
42
43 print("Policy Iteration 结果:")
44 print(f"K* = {K_star.flatten()}")
45 print(f"P* = {P_star}")
46 print(f"\nDARE解析解:")
47 print(f"K = {K_dare.flatten()}")
48 print(f"\n收敛所需迭代次数: {len(costs)}")
49 print(f"每次迭代后的代价: {costs}")
50 \end{lstlisting}

```


12.7.3 模糊控制器的简单实现

Python 模糊 PID 控制器

```

1 \begin{lstlisting}[language=Python]
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 class FuzzyPID:
6     def __init__(self, Ke=1.0, Kde=1.0, Ku=1.0):
7         self.Ke = Ke
8         self.Kde = Kde
9         self.Ku = Ku
10        self.prev_e = 0.0
11
12    def membership(self, x, centers, widths):
13        return np.exp(-0.5 * ((x - centers) / widths)**2)
14
15    def inference(self, e, de):
16        e_n = self.Ke * e
17        de_n = self.Kde * de
18
19        r1 = self.membership(e_n, -1, 0.3) * self.membership(de_n, -1, 0.3)
20        r2 = self.membership(e_n, 0, 0.3) * self.membership(de_n, 0, 0.3)
21        r3 = self.membership(e_n, 1, 0.3) * self.membership(de_n, 1, 0.3)
22
23        u1 = -2; u2 = 0; u3 = 2
24        u_fuzzy = (r1*u1 + r2*u2 + r3*u3) / (r1 + r2 + r3 + 1e-10)
25        return self.Ku * u_fuzzy
26
27    # 一阶系统仿真
28    class FirstOrderSys:
29        def __init__(self, a=1.0, b=5.0):
30            self.a = a
31            self.b = b
32            self.y = 0.0
33
34        def step(self, u, dt=0.01):
35            self.y += (-self.a * self.y + self.b * u) * dt
36            return self.y
37
38    fpid = FuzzyPID(Ke=2.0, Kde=0.5, Ku=3.0)
39    plant = FirstOrderSys()
40
41    dt = 0.01
42    t = np.arange(0, 5, dt)
43    y_ref = np.ones_like(t)
44    y = np.zeros_like(t)
45    u = np.zeros_like(t)
46
47    for i in range(len(t)):
48        y[i] = plant.y
49        e = y_ref[i] - y[i]
50        de = (e - fpid.prev_e) / dt
51        u[i] = fpid.inference(e, de)
52        plant.step(u[i], dt)
53        fpid.prev_e = e

```

§ 12.8

小结

本章介绍了五种高级控制方法的理论基础、关键算法和应用特点：

1. **模型预测控制 (MPC)**: 基于滚动优化和反馈校正的在线约束最优控制方法，擅长处理多变量耦合和硬约束问题，工业应用最为广泛；
2. **鲁棒控制**: 利用 H_∞ 范数和 μ 分析来处理模型不确定性，为控制系统在最坏情况扰动下的性能提供严格保障；
3. **自适应控制**: 通过在线参数调节 (MRAC) 或在线辨识-控制 (STC) 来应对系统参数变化，适用于具有时变特性的被控对象；
4. **强化学习与控制**: MDP 与 LQR 的数学等价性架起了 RL 与传统控制的桥梁。Policy Iteration 和 ADP 为无模型最优控制提供了实用算法；
5. **智能控制**: 模糊控制利用专家经验，神经网络利用数据驱动，模糊神经网络融合两者的优势，在复杂工业场景中发挥着不可替代的作用。

这些方法的共同特点是：不再依赖单一的精确数学模型，而是通过在线优化、鲁棒设计、数据学习和知识推理等途径赋予控制系统更强的灵活性和智能性。现代控制理论正朝着“数据 + 模型 + 优化 + 智能”深度融合的方向发展，学习型 MPC、Safe RL 和控制启发的深度学习架构代表了这一趋势的最前沿。